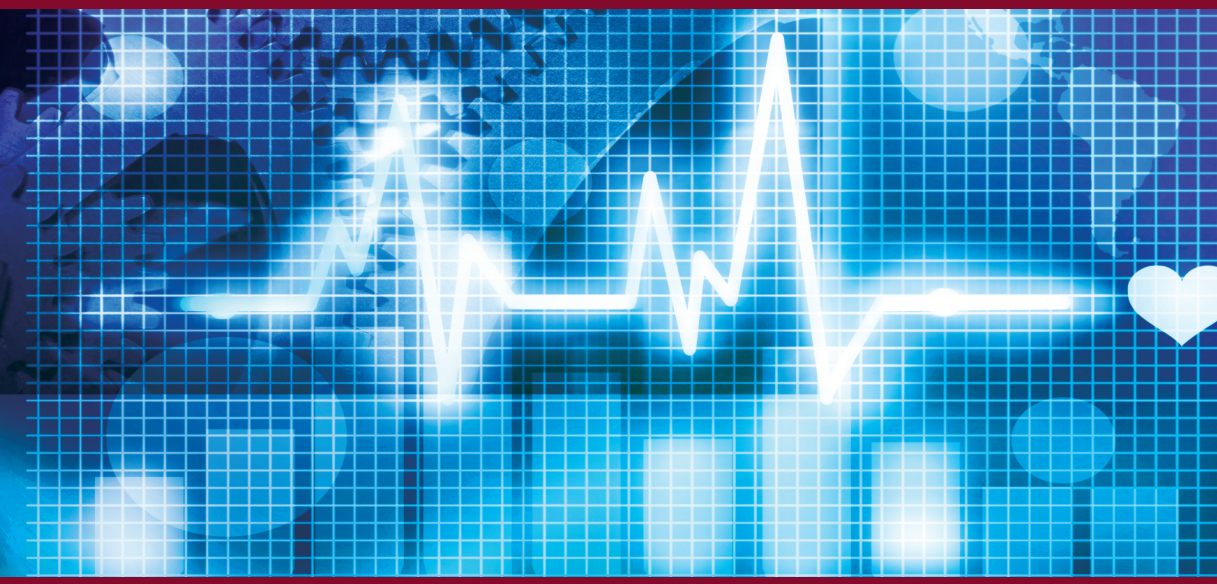
COLLECTION BIOINGÉNIERIE MÉDICALE

Série Biostatistique et sciences de la santé



Volume 1

Biostatistique et analyse informatique des données de santé avec R

Christophe Lalanne
Mounir Mesbah

iSTE editions

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Biostatistique et analyse informatique des données de santé avec R

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

First published 2016 in Great Britain by ISTE Editions Ltd.

Apart from any fair dealing for the purposes of research or private study, or criticism or review, as permitted under the Copyright, Designs and Patents Act 1988, this publication may only be reproduced, stored or transmitted, in any form or by any means, with the prior permission in writing of the publishers, or in the case of reprographic reproduction in accordance with the terms and licenses issued by the CLA. Enquiries concerning reproduction outside these terms should be sent to the publishers at the undermentioned address:

ISTE Editions Ltd
27-37 St George's Road
London SW19 4EU
UK

© ISTE Editions Ltd 2016
The rights of the authors of this work have been asserted by them in accordance with the Copyright, Designs and Patents Act 1988.

British Library Cataloguing-in-Publication Data
A CIP record for this book is available from the British Library
ISBN: 978-1-78405-119-8 (print)
ISBN: 978-1-78406-119-7 (e-book)

FSC
www.fsc.org
MIX
Paper from responsible sources
FSC® C013604

Printed and bound in Great Britain by CPI Group (UK) Ltd., Croydon, Surrey CR0 4YY, January 2016

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Série *Biostatistique et sciences de la santé*
sous la direction de Mounir Mesbah

Biostatistique et analyse informatique des données de santé avec R

Christophe Lalanne
Mounir Mesbah

iSTE editions

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Table des matières

Introduction 9

Chapitre 1. Eléments du langage 13

1.1. Avant de démarrer 13
1.1.1. Installer R 13
1.1.2. RStudio 13
1.1.3. Liste de packages utiles 14
1.1.4. Rechercher de l'aide 14
1.1.5. Scripts R 14
1.2. Représentation des données sous R 15
1.2.1. Gestion de variables numériques 15
1.2.2. Opérations sur une variable numérique 16
1.2.3. Gestion de variables catégorielles 18
1.2.4. Manipulation de variables catégorielles 19
1.3. Sélection d'observations 21
1.3.1. Sélection indexée d'observations 21
1.3.2. Sélection critériée d'observations 21
1.4. Représentation et traitement des valeurs manquantes 22
1.5. Importation et sauvegarde de données 24
1.5.1. Données univariées 24
1.5.2. Données multivariées 24
1.5.3. Sauvegarde de données dans un fichier externe 26
1.6. Gestion de données multidimensionnelles 26
1.6.1. Construction d'un tableau structuré de données 26
1.6.2. Les données *birthwt* 28
1.7. Ce qu'il faut retenir 30
1.8. Pour aller plus loin 30
1.9. Applications 31

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 2. Statistiques descriptives et estimation 35

2.1. Résumer une variable numérique 35
2.1.1. Tendance centrale et forme de la distribution 35
2.1.2. Indicateurs de dispersion 36
2.2. Résumer une variable catégorielle 37
2.3. Représenter graphiquement la distribution d'une variable 39
2.3.1. Cas des variables numériques 40
2.3.2. Cas des variables catégorielles 43
2.4. Estimation par intervalles pour une moyenne ou une proportion 44
2.4.1. Intervalles de confiance pour une moyenne 44
2.4.2. Intervalles de confiance pour une proportion 47
2.5. Ce qu'il faut retenir 48
2.6. Applications 48

Chapitre 3. Mesures et tests d'association entre deux variables 53

3.1. Statistiques descriptives bivariées 53
3.1.1. Décrire une variable numérique selon les modalités d'une variable qualitative 53
3.1.2. Décrire deux variables qualitatives 56
3.2. Comparaisons de deux moyennes de groupe 58
3.2.1. Echantillons indépendants 59
3.2.2. Echantillons non indépendants 61
3.3. Comparaisons de proportions 64
3.3.1. Cas de deux proportions 64
3.3.2. Test du chi-deux 66
3.3.3. Cas de deux échantillons non indépendants 67
3.4. Mesures de risque et odds-ratio 67
3.5. Approches non paramétriques et tests exacts 69
3.6. Ce qu'il faut retenir 70
3.7. Pour aller plus loin 71
3.8. Applications 71

Chapitre 4. Analyse de variance et plans d'expérience 77

4.1. Représentation des données et statistiques descriptives 77
4.1.1. Format de représentation des données 77
4.1.2. Statistiques descriptives et structuration des données 78
4.2. ANOVA à un facteur 80
4.2.1. Le modèle d'ANOVA à un facteur 80
4.2.2. Comparaisons par paires de traitement 83
4.2.3. Test de tendance linéaire 84

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

4.3. Approche non paramétrique de l'ANOVA à un facteur 88
4.4. ANOVA à deux facteurs 88
4.4.1. Construction du tableau d'ANOVA 89
4.4.2. Diagnostic du modèle 91
4.5. Ce qu'il faut retenir 92
4.6. Applications 92

Chapitre 5. Corrélation et régression linéaire 101

5.1. Corrélation 101
5.1.1. Statistiques descriptives 101
5.1.2. Diagramme de dispersion et courbe loess 103
5.1.3. Mesures d'association paramétrique et non paramétrique 103
5.1.4. Estimation par intervalles et test d'inférence 104
5.2. Régression linéaire simple 105
5.2.1. Droite de régression 105
5.2.2. Estimation par intervalles et tableau d'analyse de variance 108
5.2.3. Prédictions à partir du modèle de régression 109
5.2.4. Diagnostic du modèle et analyse des résidus 110
5.2.5. Lien avec l'ANOVA 112
5.3. Régression linéaire multiple 113
5.4. Ce qu'il faut retenir 114
5.5. Pour aller plus loin 114
5.6. Applications 114

Chapitre 6. Mesures d'association en épidémiologie et régression logistique 121

6.1. Tableaux de contingence et mesures d'association 121
6.1.1. Tableaux de contingence 121
6.1.2. Mesures et tests d'association 122
6.1.3. Odds-ratio et stratification 122
6.2. Etudes diagnostiques 125
6.2.1. Sensibilité et spécificité d'un test diagnostique 125
6.2.2. Valeurs prédictives positive et négative 126
6.2.3. Tableau de synthèse des propriétés diagnostiques d'un test 127
6.3. Régression logistique 128
6.3.1. Estimation des paramètres du modèle 128
6.3.2. Prédiction par intervalles 131
6.3.3. Cas des données groupées 133
6.4. Courbe ROC 134
6.5. Ce qu'il faut retenir 135
6.6. Applications 136

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 7. Analyse des données de survie 147

7.1. Représentation des données et statistiques descriptives 147
7.2. Fonction de survie et courbe de Kaplan-Meier 148
7.2.1. Table de mortalité/survie 148
7.2.2. Courbe de Kaplan-Meier 150
7.2.3. Fonction de risque cumulé 152
7.2.4. Test d'égalité des fonctions de survie 153
7.3. Régression de Cox 154
7.4. Ce qu'il faut retenir 156
7.5. Pour aller plus loin 157
7.6. Applications 158

Annexe A. Introduction à RStudio 165

Annexe B. Les graphiques avec le package lattice 169

Annexe C. Les packages Hmisc et rms 181

Bibliographie 195

Index 199

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Introduction

Une bonne partie des actions effectuées à partir d'un logiciel statistique revient à manipuler, voire transformer des données numériques représentant des données statistiques au sens propre. Il est donc primordial de bien comprendre comment sont représentées les données statistiques et comment elles peuvent être exploitées par un logiciel tel que R. Après l'importation, le recodage et la transformation éventuelle de ces données, la description des variables d'intérêt et le résumé de leur distribution sous forme numérique et graphique constitue une étape préalable et fondamentale à toute modélisation statistique, d'où l'importance de ces premières étapes dans le déroulement d'un projet d'analyse statistique. Dans un second temps, il est essentiel de bien maîtriser les commandes permettant de calculer les principales mesures d'association en recherche médicale et de savoir mettre en œuvre les modèles explicatifs et prédictifs classiques : analyse de la variance, régression linéaire et logistique et modèle de Cox. A quelques exceptions près, on préfèrera recourir aux commandes R disponibles lors de l'installation du logiciel (commandes de base), plutôt qu'à des librairies spécialisées de commandes, encore appelées packages sous R. Les packages à installer pour suivre les applications proposées dans cet ouvrage sont listés dans le premier chapitre, à la section « Avant de démarrer ».

Ce livre suppose que le lecteur est déjà familier avec les notions statistiques de base, en particulier le calcul des indicateurs de tendance centrale et de dispersion pour une variable continue, les tableaux de contingence, l'analyse de la variance et les modèles de régression classique. L'objectif est ici de mettre en application ces connaissances sur des jeux de données décrits dans de nombreux autres ouvrages, même si l'interprétation des résultats reste minimale, afin de se familiariser rapidement avec l'usage de R sur des données réelles. En particulier, l'accent est mis sur la gestion et la manipulation de données structurées puisqu'il s'avère que cela constitue 60 à 80 % du travail du statisticien. Il existe de nombreux ouvrages en français ou en anglais sur R, tant du point de vue technique que statistique. Certains de ces ouvrages sont plutôt à caractère général [SHA 12], d'autres au contraire sont

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

beaucoup plus spécialisés [BIL 14] ou abordent des notions plus avancées [HOT 09]. Le but de ce livre est de permettre au lecteur de se familiariser rapidement avec R afin qu'il puisse réaliser ses propres analyses et continuer son apprentissage de manière autonome dans le domaine des statistiques médicales.

Dans le premier chapitre, on introduira les commandes de base pour la gestion de données sous R. Il s'agit principalement de la création et de la manipulation de variables quantitatives et qualitatives (recodage de valeurs individuelles, comptage des observations manquantes), de l'importation de bases de données stockées sous forme de fichiers texte, ainsi que d'opérations arithmétiques élémentaires (minimum, maximum, moyenne arithmétique, différence, fréquence, etc.). On verra également comment sauvegarder des bases de données prétraitées au format texte ou R. L'objectif est de comprendre comment les données sont représentées sous R, et comment travailler à partir de celles-ci.

Le deuxième chapitre porte sur les commandes utiles pour la description d'un tableau de données constitué de variables quantitatives ou qualitatives. L'approche descriptive est strictement univariée, ce qui constitue le préalable à toute démarche statistique. Les commandes graphiques de base (histogramme, courbe de densité, diagramme en barres ou en points) seront présentées en complément des résumés descriptifs numériques usuels de tendance centrale (moyenne, médiane) et de dispersion (variance, quartiles). On abordera également l'estimation ponctuelle et par intervalle à l'aide d'une moyenne arithmétique et d'une proportion empirique. L'objectif est de se familiariser avec l'emploi de commandes R simples opérant sur une variable, éventuellement en précisant certaines options pour le calcul, et à la sélection d'unités statistiques parmi l'ensemble des observations disponibles.

Le troisième chapitre est consacré à la comparaison de deux échantillons, pour des mesures quantitatives ou qualitatives. On aborde les tests d'hypothèse suivants : test de Student pour échantillons indépendants ou appariés, test non paramétrique de Wilcoxon, test du χ^2 et test exact de Fisher, test de NcNemar, à partir des principales mesures d'association pour deux variables (différence de moyennes, odds-ratio et risque relatif). A partir de ce chapitre, on insistera moins sur la description univariée de chaque variable, mais il est conseillé de toujours procéder aux étapes de description des données abordées dans le deuxième chapitre. L'objectif est de maîtriser les principaux tests statistiques dans le cas où l'on s'intéresse à la relation entre une variable quantitative et une variable qualitative, ou pour deux variables qualitatives.

Le quatrième chapitre est une introduction à l'analyse de variance dans laquelle on cherche à expliquer la variabilité observée au niveau d'une variable réponse numérique par la prise en compte d'un facteur de groupe ou de classification et à l'estimation par intervalle de différences de moyennes. On mettra l'accent sur la construction d'un tableau d'ANOVA résumant les différentes sources de variabilité et sur les méthodes graphiques permettant de résumer la distribution des données individuelles

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

ou aggrégées. On discutera également le test de tendance linéaire lorsque le facteur de classification peut être considéré comme naturellement ordonné. L'objectif est de comprendre comment construire un modèle explicatif dans le cas où l'on a un, voire deux, facteurs explicatifs, et comment présenter numériquement et graphiquement les résultats d'un tel modèle à l'aide de R.

Le cinquième chapitre porte sur l'analyse de la relation linéaire entre deux variables quantitatives continues. Dans l'approche de corrélation linéaire, qui suppose une relation symétrique entre les deux variables, on s'intéressera à quantifier la force et la direction de l'association de manière paramétrique (corrélation de Pearson) ou non paramétrique (corrélation de Spearman basée sur les rangs) et à la représentation graphique de cette relation. La régression linéaire simple sera utilisée dans le cas où l'une des deux variables numériques joue le rôle d'une variable réponse et l'autre celui d'une variable explicative. On présentera les commandes utiles pour l'estimation des coefficients de la droite de régression, la construction du tableau d'ANOVA associé à la régression, et la prédiction. L'objectif de ce chapitre reste identique à celui du quatrième chapitre, à savoir présenter les commandes R nécessaires à la construction d'un modèle statistique simple entre deux variables, dans une optique explicative ou prédictive.

Dans le sixième chapitre seront abordées les principales mesures d'association rencontrées dans les études épidémiologiques : odds-ratio, risque relatif, prévalence, etc. Les commandes R permettant l'estimation (ponctuelle et par intervalle) et les tests d'hypothèse associés seront illustrées sur des données de cohorte ou d'études cas-témoins. La mise en œuvre d'un modèle de régression logistique simple permet de compléter l'éventail des méthodes statistiques permettant d'expliquer la variabilité observée au niveau d'une variable réponse binaire. L'objectif est de comprendre les commandes R à utiliser dans le cas où les variables sont binaires, soit pour résumer un tableau de contingence sous forme d'indicateurs d'association soit pour modéliser la relation entre une réponse binaire (malade/non-malade) et une variable explicative qualitative à partir de données dites groupées.

Le septième et dernier chapitre constitue une introduction à l'analyse de données censurées, aux principaux tests associés à la construction d'une courbe de survie (test du log-rank ou de Wilcoxon) et enfin au modèle de régression de Cox. La spécificité des données censurées impose un soin particulier dans le codage des données sous R, et l'objectif est de présenter les commandes R essentielles à la bonne représentation des données de survie sous forme numérique, à leur résumé numérique (médiane de survie) et graphique (courbe de Kaplan-Meier), et à la mise en œuvre des tests courants.

A la fin de chaque chapitre, quelques applications sont proposées et des exemples de commandes permettant de répondre aux questions posées sont proposées pour la plupart des questions. Il est parfois possible d'obtenir des résultats identiques

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

par d'autres approches ou en utilisant d'autres commandes. Les sorties R ne sont pas reproduites mais le lecteur est encouragé à essayer lui-même les instructions R proposées et à essayer des instructions alternatives ou complémentaires. On supposera que les fichiers de données utilisés sont disponibles dans le répertoire de travail. L'ensemble des fichiers de données et les commandes R utilisées dans cet ouvrage peuvent être téléchargés sur le site GitHub, à l'adresse https://github.com/biostatsante.

Trois annexes permettent de mieux se familiariser avec RStudio, les packages `lattice` pour la gestion des sorties graphiques et `Hmisc` et `rms` pour la gestion de données avancée et la modélisation. Ces chapitres ne remplacent évidemment pas les excellents ouvrages de John Verzani, Paul Murrell et Franck Harrell [VER 11, MUR 05, HAR 01].

Pour des raisons de mise en page, certaines sorties R ont été tronquées ou reformatées. Par conséquent, celles-ci pourraient être amenées à différer lorsque le lecteur tente de reproduire les commandes contenues dans cet ouvrage.

Un index des commandes R utilisées dans les illustrations est disponible à la fin de l'ouvrage.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 1

Eléments du langage

Plus qu'un simple logiciel pour les statistiques, R est avant tout un langage de manipulation de données statistiques [IHA 96, VEN 02]. Cela explique en partie sa difficulté de prise en main pour les utilisateurs habitués à suivre des menus déroulants tels que ceux proposés par SPSS (même si SPSS offre également un langage basique de macro). Ce premier chapitre permet de découvrir les éléments du langage et de se familiariser avec la manière de représenter les données statistiques sous R. Dans toutes les illustrations qui suivront, les commandes R sont préfixées du signe >, qui désigne l'invite de la console R. Il est donc inutile de recopier ce symbole pour tester les instructions proposées.

1.1. Avant de démarrer

1.1.1. *Installer R*

L'installation de R est relativement aisée et les instructions se trouvent sur le site suivant : http://cran.r-project.org. Le logiciel est disponible pour Windows, Linux et Mac. Outre le programme R lui-même, l'installateur fournit un éditeur de scripts R, l'aide en ligne et un ensemble de packages, appelés packages de base. Les packages regroupent des commandes spécifiques d'un certain domaine (commandes graphiques, commandes de modélisation, etc.) et permettent d'enrichir les fonctionnalités de base de R.

1.1.2. *RStudio*

Bien que R soit suffisant pour démarrer ou effectuer des analyses statistiques, RStudio (www.rstudio.com) fournit un environnement de travail particulièrement agréable pour R. Il intègre un éditeur de scripts R puissant, une console dans laquelle exécuter les commandes (ou envoyer celles saisies dans l'éditeur de scripts), l'aide

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

en ligne, un navigateur de graphiques, un visualisateur de données, et bien d'autres choses encore [VER 11]. Une présentation succincte du logiciel est fournie en annexe.

1.1.3. *Liste de packages utiles*

Bien que les packages de base de R permettent de réaliser la plupart des analyses statistiques abordées dans ce livre, certains packages additionnels devront être installés afin de reproduire certaines des applications proposées. Outre les packages de base qui sont utilisés tout au long de ce livre (`lattice`, `foreign`, `survival`), les packages suivants pourront être installés à l'aide de la commande `install.packages()`, en indiquant entre les parenthèses le nom du package entourés de guillements, en suivant la notation anglo-saxonne, soit par exemple `install.packages("reshape2")` : `reshape2`, `gridExtra`, `vcd` (chapitre 3) ; `epiR`, `car` (chapitre 4) ; `ppcor`, `psych` (chapitre 5) ; `epicalc`, `ROCR` (chapitre 6).

Ces packages peuvent être installés directement depuis RStudio en utilisant le gestionnaire de packages. Avant d'installer un package, il est possible de consulter l'aide en ligne sur le site CRAN (http://cran.r-project.org). Le site CRAN offre également la possibilité de rechercher des packages par thème (ou « Task Views »).

Pour accéder aux commandes d'un package, on utilisera la commande `library()` en indiquant le nom du package.

1.1.4. *Rechercher de l'aide*

Comme dans tout logiciel, il est important de savoir comment rechercher de l'aide sur une commande précise ou à partir d'un mot-clé. Les commandes `help()`, et `help.search()` ou `apropos()` remplissent ces deux fonctions sous R. RStudio fournit par ailleurs un moteur de recherche intégré qui facilite grandement l'accès aux pages d'aide en ligne.

1.1.5. *Scripts R*

Le plus simple et la manière recommandée pour tirer avantage des fonctionnalités de RStudio dans la gestion d'un projet d'analyse statistique consiste à saisir les commandes dans l'éditeur de scripts, tout en prenant soin de sauvegarder ces commandes dans un fichier script R. Les commandes saisies dans l'éditeur de RStudio peuvent être envoyées directement à la console et R affichera le résultat dans cette même console. Cette approche interactive permet de construire un script de commande de manière incrémentale, en corrigeant ou adaptant les commandes au fur et à mesure.

Il est également important de documenter les principales étapes de l'analyse statistique ainsi que les commandes qui pourraient ne pas être claires pour un lecteur

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

externe. Il faut en effet toujours imaginer que le script de commandes R pourrait être réutilisé dans le futur par une tierce personne. Les commentaires sont utiles dans ce cas et il suffit de préfixer une ligne de texte par le symbole # pour que celle-ci soit traitée comme un commentaire et non une instruction par R. Il convient également d'organiser logiquement les commandes dans le script R, en distinguant bien les commandes relatives à l'importation des données, aux statistiques descriptives univariées et bivariées, aux modèles statistiques, etc. Tout cela doit figurer dans des parties bien distinctes, et reposer le moins possible sur des variables ou tableaux de données temporaires qui modifieraient significativement le tableau de données d'origine, sans aucune documentation ou sauvegarde sur disque. En effet, les données d'origine doivent toujours être accessibles à n'importe quel moment de l'analyse. Les tableaux de données intermédiaires peuvent être sauvegardés séparément. Dans le cas de gros projets d'analyse, il est préférable de créer plusieurs fichiers script pour stocker les commandes correspondant aux différentes étapes de l'analyse.

1.2. Représentation des données sous R

1.2.1. *Gestion de variables numériques*

Supposons que l'on dispose du poids de dix nouveaux-nés (variable x, en grammes) et de celui de leur mère (variable y, en kilogrammes), comme illustré dans le tableau 1.1.

x	2523	2551	2557	2594	2600	2622	2637	2637	2663	2665
y	82,7	70,5	47,7	49,1	48,6	56,4	53,6	46,8	55,9	51,4

Tableau 1.1. *Données artificielles sur les poids à la naissance*

Dans l'exemple suivant, on a créé une variable appelée `x` dans laquelle on stocke les observations de poids des nouveaux-nés reportées dans le tableau 1.1 sous la forme d'une simple liste de nombres :

```
> x <- c(2523,2551,2557,2594,2600,2622,2637,2637,2663,2665)
```

On notera l'usage du symbole `<-` (à préférer au symbole =) pour associer une série de mesures à une variable nommée. On peut à présent afficher ces valeurs sous R en utilisant la commande `print(x)` ou en tapant simplement le nom de la variable :

```
> x
[1] 2523 2551 2557 2594 2600 2622 2637 2637 2663 2665
```

On procèderait naturellement de la même manière avec `y` :

```
> y <- c(82.7,70.5,47.7,49.1,48.6,56.4,53.6,46.8,55.9,51.4)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Les espaces n'ont pas d'importance lorsque l'on saisit une liste de nombres et, à de rares exceptions près, d'autres expressions. Le séparateur décimal est obligatoirement le point puisque R suit la notation anglo-saxonne pour les nombres.

On pourra vérifier que R a gardé en mémoire les deux variables dans ce que l'on appelle l'espace de travail à l'aide de la commande `ls()` :

```
> ls()
[1] "x" "y"
```

On pourra donc toujours accéder à ces variables tant que la session R n'est pas fermée et que l'on n'a pas supprimé entre temps l'une ou l'autre de ces variables.

Le nombre d'observations pour la variable `x` correspond à la longueur de `x` (nombre d'éléments), lorsqu'il n'y a pas de valeurs manquantes :

```
> length(x)
[1] 10
```

Les valeurs d'une variable, ou en termes plus statistiques les valeurs prises par une variable pour un échantillon de taille n, peuvent être sélectionnées en précisant l'index ou le numéro de l'observation entre crochets. On verra plus en détail dans la partie suivante comment on peut sélectionner plus d'une observation à la fois, et comment sélectionner des observations sur la base d'un critère externe, et un peu plus tard encore comment on peut généraliser cette approche à la sélection d'un ensemble d'observations recueillies pour une même unité statistique. La cinquième observation s'obtient de la manière suivante :

```
> x[5]
[1] 2600
```

Les observations sont indexées de 1 à n, où n désigne le nombre total d'éléments dans la variable considérée. La première et la dernière observations seront ainsi obtenues à l'aide des commandes `x[1]` et `x[10]`.

1.2.2. *Opérations sur une variable numérique*

On aura remarqué que les poids des bébés sont exprimés en grammes, alors que ceux de la mère sont en kg. Le poids du 5e bébé en kg s'obtient à l'aide d'une simple opération arithmétique de division :

```
> x[5] / 1000
[1] 2.6
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Mais, cette même division peut être appliquée à l'ensemble des observations, sans avoir à la réaliser pour chaque éléments de `x` pris individuellement, comme le montre l'exemple suivant :

```
> x / 1000
[1] 2.523 2.551 2.557 2.594 2.600 2.622 2.637 2.637 2.663 2.665
```

L'opération précédente n'a pas modifié les valeurs de la variable `x` d'origine, mais il est très facile de créer une nouvelle variable dans laquelle on stockera les poids des bébés exprimés en kg :

```
> x2 <- x / 1000
> x
[1] 2523 2551 2557 2594 2600 2622 2637 2637 2663 2665
> x2
[1] 2.523 2.551 2.557 2.594 2.600 2.622 2.637 2.637 2.663 2.665
```

Evidemment, on pourrait également vouloir remplacer les anciennes valeurs de `x` par les nouvelles valeurs. Dans ce cas, il suffit de remplacer l'instruction `x2 <- x / 1000` par `x <- x / 1000`. Cependant, il ne sera plus possible de revenir aux anciennes valeurs de `x` après avoir effectué cette opération. De même que la suppression d'une variable avec la commande `rm()`, toute mise à jour des éléments d'une variable est définitive pour la session en cours.

Outre les opérateurs arithmétiques de base (addition, soustraction, multiplication et division), R dispose de la plupart des fonctions que l'on trouve sur un calculateur : logarithme, exponentielle, valeur absolue, etc. Ainsi, `log(x)` renverrait les valeurs de `x` après transformation par le logarithme népérien, alors que pour le logarithme en base 10 on utiliserait `log10(x)`.

Enfin, R offre une large gamme de commandes plus spécifiquement orientées sur le traitement statistique des données, par exemple résumer la distribution des valeurs observées (tendance centrale, dispersion, étendue, etc.). La moyenne arithmétique des valeurs de `x` s'obtient simplement avec la commande `mean()` :

```
> mean(x)
[1] 2604.9
```

D'autres commandes, sur lesquelles nous reviendrons dans le deuxième chapitre, permettent d'obtenir les informations essentielles concernant la forme de la distribution d'une variable continue, telle que l'étendue ou la variance, par exemple :

```
> range(x)
[1] 2523 2665
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> c(min(x), max(x))
[1] 2523 2665
> var(x)
[1] 2376.767
```

Comme on peut le constater dans les instructions précédentes, il est tout à fait possible de combiner deux commandes qui renvoient le même type de résultat, par exemple `c(min(x), max(x))`. Comme dans le cas de la création d'une liste de nombres, la commande `c()` permet d'associer dans une même liste les résultats de commandes renvoyant des nombres.

Les illustrations suivantes reposent sur la même idée que l'opération de division que l'on a évoquée plus haut : chaque opération (mise au carré des valeurs de `x` et centrage de ces valeurs sur leur moyenne) est appliquée élément par élément :

```
> sum(x)
[1] 26049
> sum(x^2)
[1] 67876431
> sum(x^2 - mean(x))
[1] 67850382
```

Dans l'exemple précédent, `mean(x)` est une constante (qui dépend des données mais qui ne varie pas dans ce cas de figure) que l'on soustrait de chaque x_i^2 (i dénotant le numéro d'observation ou index dans la variable `x`), ce qui revient à manipuler onze valeurs différentes au total. Dans l'expression `x - x^2`, en revanche, on soustrait de chaque élément de `x` sa propre valeur au carré (dix opérations au total, avec chaque fois dix paires de nombres différents).

Les commandes `sort()`, `order()` et `rank()` permettent de trier les éléments d'une variable ou de travailler avec les rangs des observations, c'est-à-dire leur position dans la variable.

1.2.3. *Gestion de variables catégorielles*

Les variables catégorielles ou qualitatives possèdent un statut à part dans la plupart des logiciels statistiques : leurs modalités sont représentées par des nombres (1, 2, 3, etc.) mais on leur associe généralement des étiquettes, appelées `labels` sous R. Dans l'exemple précédent on disposait de données sur le poids des bébés à leur naissance (`x`) et le poids de leur mère (`y`). Supposons que l'on sache également si la mère fumait pendant le premier trimestre de grossesse et appelons cette variable `z`. Lorsque la mère ne fumait pas pendant cette période, la variable vaut 1 ; lorsque la mère fumait, la variable vaut 2. Les données augmentées de la variable `z` sont reportées dans le tableau 1.2.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

x	2523	2551	2557	2594	2600	2622	2637	2637	2663	2665
y	82,7	70,5	47,7	49,1	48,6	56,4	53,6	46,8	55,9	51,4
z	1	1	2	2	2	1	1	1	2	2

Tableau 1.2. *Données artificielles augmentées pour les poids à la naissance*

On saisira les données sous R comme on l'a fait pour les variables `x` et `y`, soit :

```
> z <- c(1, 1, 2, 2, 2, 1, 1, 1, 2, 2)
> z
[1] 1 1 2 2 2 1 1 1 2 2
```

La commande `factor(z)` permet de convertir cette variable numérique en variable catégorielle. Pour associer des étiquettes aux codes numériques on utilise l'option `labels=`. L'option `levels=` permet de spécifier les codes numériques des niveaux de la variable. La valeur 1 sera associée à la modalité NF et la valeur 2 à la modalité F :

```
> factor(z, levels = c(1,2), labels=c("NF","F"))
 [1] NF NF F  F  F  NF NF NF F  F
Levels: NF F
```

On notera que lorsque R affiche le contenu de la variable `z`, il est précisé qu'il s'agit bien d'une variable de type `factor`, avec deux niveaux, dans ce cas non ordonnés, NF et F. Certaines opérations, comme le calcul d'une moyenne arithmétique, n'ont généralement aucun sens sur ce type de variable et c'est ce qui se passe avec R également. En revanche, les opérations de tri à plat ou croisé restent tout à fait valides. Autre aspect important : les étiquettes associées à une variable qualitative peuvent être retrouvées à l'aide de la commande `levels`, tandis que `nlevels` renvoit le nombre total de modalités :

```
> z <- factor(z, labels=c("NF","F"))
> levels(z)
[1] "NF" "F"
> nlevels(z)
[1] 2
```

1.2.4. *Manipulation de variables catégorielles*

Il arrive fréquemment que l'on ait besoin de recoder une variable qualitative à k classes en $j < k$ classes, ou que l'on souhaite associer des étiquettes (*labels*) aux modalités numériques d'une variable qualitative. Par exemple, supposons que l'on dispose des données de dix patients sur le volume expiratoire maximal seconde

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

(variable `vems`) sous forme d'une variable à trois niveaux ordonnés : critique (1), bas (2), normal (3). Ici, on utilisera la commande `sample()` qui permet de générer des données aléatoires par rééchantillonnage à partir d'une liste de valeurs. Il s'agit d'une commande R qui attend trois options : la première indique la liste de valeurs admissibles, la deuxième le nombre d'observations à générer et la dernière le type de tirage que l'on souhaite effectuer, avec (`replace = TRUE`) ou sans (`replace = FALSE`) remise :

```
> vems <- sample(1:3, 10, replace=TRUE)
> vems
 [1] 3 2 1 3 3 3 2 2 1 2
```

Notons que les valeurs obtenues sont aléatoires et ne seront pas nécessairement identiques si l'on utilise cette commande dans une session R. Pour cela, il faudrait utiliser la commande `set.seed()` pour assurer la reproductibilité de ces simulations.

Dans un premier temps, on va remplacer les valeurs numériques par les étiquettes décrites précédemment (1 = critique, 2 = bas, 3 = normal) :

```
> vems <- factor(vems, levels=c(1,2,3),
                 labels=c("critique","bas","normal"))
> vems
 [1] normal   bas      critique normal   normal   normal   bas
 [8] bas critique bas
Levels: critique bas normal
```

On peut très rapidement vérifier la distribution des effectifs par classe à l'aide de la commande `table()` qui permet de réaliser des tris à plat ou des tris croisés qui seront abordés au chapitre suivant :

```
> table(vems)
vems
critique      bas   normal
       2        4        4
```

Si l'on souhaite recoder la variable `vems` à trois classes en une variable à deux classes en aggrégeant les deux premières modalités ou niveaux, on peut utiliser la commande `levels()` de la manière suivante :

```
> levels(vems)
[1] "critique" "bas"      "normal"
> levels(vems)[1:2] <- "critique ou bas"
> levels(vems)
[1] "critique ou bas" "normal"
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> table(vems)
vems
critique ou bas          normal
              6               4
```

On remarquera que les modalités de la variable `vems` ont bien été modifiées et que les effectifs des deux premiers niveaux d'origine se retrouvent à présent dans le premier niveau intitulé `critique ou bas`. Il est conseillé de toujours vérifier que les opérations de recodage des niveaux d'une variable catégorielle ont bien été effectuées comme attendu, en utilisant les commandes `levels()` ou `table()`.

1.3. Sélection d'observations

1.3.1. *Sélection indexée d'observations*

Avec les données de poids discutées plus haut, on pourrait vouloir sélectionner plus d'une observation, par exemple la 3e et la 5e :

```
> x[c(3,5)]
[1] 2557 2600
```

ou de la 3e à la 5e :

```
> x[3:5]
[1] 2557 2594 2600
```

La syntaxe un peu particulière `3:5` signifie en fait la séquence des nombres entiers débutant à 3 et se terminant à 5. Ceci est donc strictement équivalent à `c(3,4,5)`. Pour accéder à des valeurs particulières d'une variable, on se contente donc de fournir une liste de numéros d'observation (ou index). Le même principe d'indexation s'applique bien entendu aux variables qualitatives :

```
> vems[2]
[1] critique ou bas
Levels: critique ou bas normal
```

1.3.2. *Sélection critériée d'observations*

Supposons que l'on veuille à présent obtenir le poids des bébés dont la mère pèse moins de 50 kg :

```
> x[y < 50]
[1] 2557 2594 2600 2637
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

La commande précédente se lit : sélectionner les valeurs de `x` pour lesquelles la condition (logique) `y < 50` est vérifiée. Cette condition peut se visualiser en tapant directement `y < 50` sous R, ce qui produit le résultat suivant :

```
> y < 50
 [1] FALSE FALSE  TRUE  TRUE  TRUE FALSE FALSE  TRUE FALSE FALSE
```

Ce principe de sélection sur la base d'un critère externe reste valable lorsque le critère est une variable catégorielle. Si l'on souhaite obtenir la liste des poids des bébés dont la mère est non-fumeur, on tapera simplement la commande :

```
> x[z == "NF"]
[1] 2523 2551 2622 2637 2637
```

On notera l'usage du double signe égal (==) pour signifier la condition logique « `z` valant `NF` ». Enfin, on peut combiner des critères portant sur des variables du même type ou de différents types à l'aide d'opérateurs logiques, `&` (et) et `|` (ou), principalement. La commande suivante retourne les valeurs de `x` telles que `z` vaut `NF` *et* $y < 55$ (poids des bébés dont la mère de fume pas et pesant 55 kg ou moins) :

```
> x[z == "NF" & y <= 55]
[1] 2637 2637
```

1.4. Représentation et traitement des valeurs manquantes

Dans la réalité, il est rare que l'on dispose de données sans valeurs manquantes. Quelle que soit la manière dont on décide de les traiter sur le plan statistique, il est important de s'assurer qu'elles sont bien représentées comme telles par le logiciel statistique.

Reprenons l'exemple précédent. Supposons que la 3e observation pour le poids des bébés soit en fait manquante, ou que l'on souhaite la traiter comme telle. Cette donnée est représentée par un point (.) dans le tableau 1.3.

x	2523	2551	.	2594	2600	2622	2637	2637	2663	2665
y	82,7	70,5	47,7	49,1	48,6	56,4	53,6	46,8	55,9	51,4

Tableau 1.3. *Données artificielles sur les poids à la naissance avec données manquantes*

On aurait très bien pu représenter cette valeur manquante par une absence de valeur (comme une cellule vide sous Microsoft Excel), ou n'importe quel autre symbole. En

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

utilisant la même approche que lorsque l'on a défini la variable `x` pour la première fois, une façon de saisir les poids des bébés consisterait à remplacer la valeur manquante par le symbole `NA`, qui est le symbole réservé par R pour coder les valeurs manquantes :

```
> c(2523, 2551, NA, 2594, 2600, 2622, 2637, 2637, 2663, 2665)
```

Puisque la variable `x` a déjà été saisie, on peut simplement la mettre à jour, en l'occurence remplacer le troisième élément par `NA` :

```
> x[3] <- NA
> x
 [1] 2523 2551   NA 2594 2600 2622 2637 2637 2663 2665
```

On prendra garde au fait que la commande `length(x)` renvoit toujours 10 : il y a bien dix observations, mais l'une d'elle est manquante. La commande `is.na()` permet de vérifier quelles sont les valeurs manquantes présentes dans une variable :

```
> is.na(x)
 [1] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
> which(is.na(x))
[1] 3
```

Comme on l'a déjà vu dans le cas de la sélection d'observation, les valeurs renvoyées par la commande `is.na()` sont des valeurs booléennes valant vrai (`TRUE`) lorsque la valeur de `x` est manquante (`NA`), faux (`FALSE`) sinon. Cette commande renverra donc autant de valeurs que la variable `x` en contient. Pour faciliter l'identification des numéros d'observations manquantes et au fond limiter l'information aux seules informations utiles, on peut préférer combiner `is.na()` avec la commande `which()`. On notera que les commandes se combinent de manière intuitive : la commande `which()` utilise le résultat renvoyé par `is.na()`, sans que l'on ait besoin de créer une variable auxiliaire pour stocker ce résultat. Une alternative consiste à dénombrer les cas dits complets avec la commande `complete.cases` :

```
> complete.cases(x)
 [1]  TRUE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

En utilisant le principe de sélection d'observations vu précédemment, la commande suivante renverrra donc les valeurs de `y` pour lesquelles `x` a une valeur non manquante :

```
> y[complete.cases(x)]
[1] 82.7 70.5 49.1 48.6 56.4 53.6 46.8 55.9 51.4
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

1.5. Importation et sauvegarde de données

1.5.1. *Données univariées*

Les trois séries de dix mesures discutées dans les parties précédentes étaient de taille suffisamment modérée pour permettre leur saisie directement dans R. Cependant, dans la plupart des cas, les données auront souvent déjà été saisies dans un fichier externe et la première étape consiste généralement à les importer dans R.

Voici, par exemple, le contenu du fichier `poids.dat`, qui est un simple fichier texte que l'on peut visualiser avec n'importe quel éditeur de texte :

```
2523 2551 2557 2594 2600 2622 2637 2637 2663 2665
```

Dans un cas simple où l'on désire importer une seule série de mesures, la commande `scan()` est suffisante. Il suffit d'indiquer l'endroit où se trouve le fichier de données. Par « endroit » on entend la localisation précise du fichier dans l'arborescence des fichiers de l'ordinateur. Ici, on supposera que le répertoire de travail (modifiable avec la commande `setwd()` ou en utilisant les utilitaires de RStudio) est le répertoire courant :

```
> x <- scan("poids.dat")
Read 10 items
> head(x, n=3)
[1] 2523 2551 2557
```

La commande `head()` permet d'afficher les premières observations d'une variable. L'option `n=` permet d'en préciser le nombre (par défaut, `n=6`).

1.5.2. *Données multivariées*

Dans les cas plus complexes, mais plus proches de la réalité, les données sont multivariées avec, en général, les variables organisées en colonnes et les observations en ligne [WIC 14] : chaque ligne du fichier représente donc une unité statistique sur laquelle on a collecté plusieurs mesures ou données (variables ou champs), ces dernières étant séparées l'une de l'autre soit par une virgule, un point-virgule, des espaces ou des tabulations. Dans tous les cas, la commande à utiliser est `read.table()` (le séparateur de champ est un espace ou une tabulation), ou l'un de ses raccourcis `read.csv()` (séparateur de type virgule) et `read.csv2()` (séparateur de type point-virgule).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Considérons le fichier `birthwt.dat`, dont les trois premières lignes sont reproduites ci-après :

```
0 19 182 2 0 0 0 1 0 2523
0 33 155 3 0 0 0 0 3 2551
0 20 105 1 1 0 0 0 1 2557
```

Comme on le voit, il y a dix colonnes (soit dix variables) et chaque valeur est séparée de la suivante par un espace. Le nom des variables ne figure nulle part, mais l'on sait qu'il s'agit des variables suivantes : statut pondéral du bébé à la naissance `low` (= 1 si poids $< 2,5$ kg, 0 sinon), `age` de la mère (années), `lwt` poids de la mère (en livres), `race` ethnicité de la mère (codée en trois classes, 1 = white, 2 = black, 3 = other), `smoke` (= 1 si consommation de tabac durant la grossesse, 0 sinon), `ptl` (nombre d'accouchements préterme antérieurs), `ht` (= 1 si antécédent d'hypertension, 0 sinon), `ui` (= 1 si manifestation d'irritabilité utérine, 0 sinon), `ftv` (nombre de consultations chez le gynécologue durant le premier trimestre de grossesse), `bwt` pour le poids des bébés à la naissance (en grammes). Voici comment ces données peuvent être importées sous R, toujours en supposant que le fichier de données se trouve dans le répertoire de travail de R :

```
> bt <- read.table("birthwt.dat", header=FALSE)
> varnames <- c("low","age","lwt","race","smoke","ptl","ht",
                "ui","ftv","bwt")
> names(bt) <- varnames
> head(bt)
  low age lwt race smoke ptl ht ui ftv  bwt
1   0  19 182    2     0   0  0  1   0 2523
2   0  33 155    3     0   0  0  0   3 2551
3   0  20 105    1     1   0  0  0   1 2557
4   0  21 108    1     1   0  0  1   2 2594
5   0  18 107    1     1   0  0  1   0 2600
6   0  21 124    3     0   0  0  0   0 2622
```

Si le fichier de données avait eu la forme suivante :

```
0,19,182,2,0,0,0,1,0,2523
0,33,155,3,0,0,0,0,3,2551
0,20,105,1,1,0,0,0,1,2557
```

qui est typiquement le format d'exportation offert par un tableur comme Excel, alors on aurait simplement remplacé la commande `read.table()` par `read.csv()` (ou modifier l'option `sep=` de `read.table()`).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

1.5.3. *Sauvegarde de données dans un fichier externe*

Dans le cas univarié comme dans le cas multivarié, on pourra exporter les données traitées sous R à l'aide des commandes `write.table()` ou `write.csv()` qui produisent des fichiers texte identiques à ceux discutés plus haut. Par exemple :

```
> write.csv(bt, file="bt.csv")
```

permettra de sauvegarder le fichier importé avec la commande `read.table()` sous la forme d'un fichier où les valeurs des variables sont séparées par des virgules, et le fichier `bt.csv` pourra être visualisé à l'aide d'un tableur de type Microsoft Excel.

Si l'on souhaite sauvegarder les données au format R directement, on utilisera plutôt `save(bt, file="bt.RData")`, `RData` (ou `rda`) constituant l'extension réservée aux fichiers de données R.

1.6. Gestion de données multidimensionnelles

1.6.1. *Construction d'un tableau structuré de données*

Jusqu'à présent, nous avons crée et manipulé différentes variables (`x`, `y` et `z`, principalement). Une manière plus naturelle de rendre compte de cette association entre les observations est de regrouper ces variables dans une même structure de données, ou tableau, que l'on appelle « data frame » sous R.

La structure générale d'un data frame est illustrée dans la figure 1.1. Les données hypothétiques sont supposées regrouper dans un data frame nommé `a` des données recueillies sur sept unités statistiques, arrangées en lignes, et les quatre variables correspondent à un score numérique (`score`), le sexe de l'individu (`gender`), son quotient intellectuel (`IQ`) et son niveau socio-économique (`SES`). Un data frame est donc un tableau rectangulaire permettant d'arranger des variables de type différent (nombres, caractères, facteurs) mais de même taille en colonnes. Une ligne désignera une unité statistique. La première unité a ainsi un score de 1, est de sexe masculin, a un quotient intellectuel de 92 et son statut socio-économique est C. Les colonnes d'un data frame peuvent être nommées, ce qui permet d'accéder aux variables par leur nom.

On peut appliquer ce principe aux variables `x`, `y` et `z` utilisées dans les paragraphes précédents. Ces trois variables seront stockées dans trois colonnes distinctes, à l'aide de la commande `data.frame()`. Voici comment l'on peut procéder sous R :

```
> d <- data.frame(x, y, z)
> names(d) <- c("poids.bébé", "poids.mère", "cig.mère")
> head(d, n=3)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
  poids.bébé poids.mère cig.mère
1       2523       82.7       NF
2       2551       70.5       NF
3       2557       47.7        F
```

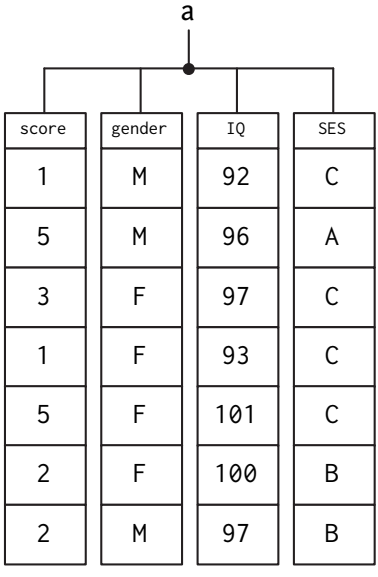


Figure 1.1. *Structure d'un data frame*

On remarquera que l'on en a profité pour renommer le nom des variables. Le premier individu pèse donc 2 523 g et sa mère qui pèse 82,7 kg ne fume pas. Pour afficher ces seules informations, on indexera le tableau nommé `d` par le numéro de ligne correspondante, par exemple :

```
> d[1,]
  poids.bébé poids.mère cig.mère
1       2523       82.7       NF
```

Le fait de ne rien préciser pour les numéros de colonne signifie que l'on désire toutes les sélectionner. Au contraire, pour afficher la valeur de la deuxième variable pour les deux premières observations, on utilisera :

```
> d[c(1,2), 2]
[1] 82.7 70.5
```

Plutôt que d'utiliser les numéros de colonnes, on peut tout à fait utiliser le nom de la variable. Par exemple, une instruction telle que `d$poids.bébé` renverra

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

toutes les valeurs de la variable `poids.bébé`, de sorte que `d$poids.mère[1:2]` renverrait comme dans le cas précédent les deux premières observations de la variable `poids.mère`.

1.6.2. *Les données* birthwt

Les données complètes sur le poids des bébés discutées précédemment figurent parmi les exemples de base fournis avec R, et on peut les importer grâce à la commande `data()`. Les données proviennent en fait d'une étude épidémiologique des années 1980 [HOS 89] où l'on s'intéressait aux facteurs de risque de naissance avec un poids inférieur à la norme chez des enfants américains. Ce jeu de données sera utilisé systématiquement dans les chapitres suivants, à l'exception de celui portant sur l'analyse des données de survie (chapitre 7). Une fois importé, ce tableau de données est disponible sous le nom `birthwt` :

```
> data(birthwt, package="MASS")
> c(nrow(birthwt), ncol(birthwt))
[1] 189  10
> names(birthwt)
 [1] "low"   "age"   "lwt"   "race"  "smoke" "ptl"   "ht"
 [8] "ui"    "ftv"   "bwt"
> head(birthwt, n=2)
   low age lwt race smoke ptl ht ui ftv  bwt
85   0  19 182    2     0   0  0  1   0 2523
86   0  33 155    3     0   0  0  0   3 2551
```

La variable `birthwt` est un tableau (data frame) regroupant dix variables, chacune ayant 189 observations. On dispose donc d'informations sur 189 unités statistiques dans cette étude prospective. Pour accéder aux valeurs d'une variable particulière, on utilisera la notation suivante : nom du tableau (`birthwt`) suivi du nom de la variable préfixée du signe dollar (`$`). Par exemple, les cinq premières valeurs pour le poids des bébés (`bwt`) sont ainsi :

```
> birthwt$bwt[1:5]
[1] 2523 2551 2557 2594 2600
```

La signification de chaque variable a été donnée plus haut. On sait que certaines variables sont strictement binaires (0/1), comme `low`, `smoke`, `ht` et `ui`, alors que les autres variables sont numériques, soit à valeurs discrètes comme `ftv` soit à valeurs supposées continues comme `lwt` ou `bwt`. On sait que dans le cas des variables binaires, une valeur de 1 signifie que le signe est présent (la mère fume, la mère a des antécédents d'hypertension, etc.). Cela ne coûte rien d'ajouter des étiquettes plus informatives à ces variables. Le cas du facteur ethnicité (`race`) est plus particulier car

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

il s'agit d'une variable qualitative mais qui est pour l'instant traitée par R comme une variable numérique :

```
> summary(birthwt$race)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.000   1.000   1.000   1.847   3.000   3.000
```

La commande `summary()` permet de résumer la distribution des variables numériques et catégorielles, et s'applique aussi bien à une variable seule qu'à un tableau de données. Voici une manière de recoder les modalités des variables qualitatives pour le tableau `birthwt` :

```
> ouinon <- c("No", "Yes")
> birthwt$smoke <- factor(birthwt$smoke, labels=ouinon)
> birthwt$race <- factor(birthwt$race, levels=c(1, 2, 3),
                         labels=c("White", "Black", "Other"))
```

On utilisera la même démarche pour `low`, `ht`, `ui` (utiliser les étiquettes `ouinon`) et on vérifiera que les données finales sont bien dans le format attendu à l'aide de `summary()` qui permet de repérer rapidement les variables considérées comme des variables numériques (résumé numérique avec min, max, moyenne) ou qualitatives (tableau d'effectifs) :

```
> birthwt <- within(birthwt, {
    low <- factor(low, labels=ouinon)
    ht <- factor(ht, labels=ouinon)
    ui <- factor(ui, labels=ouinon)
  })
```

Plutôt que de répéter chaque fois le nom du data frame suivi du nom de la variable, les opérations ont été regroupées à l'aide de la commande `within()`. Il est inutile de préciser les codes numériques (`levels=`) pour ces trois variables puisqu'elles sont toutes binaires.

A ce stade, voici ce que donne le recodage des variables lorsque l'on utilise la commande `summary()` sur les cinq premières variables :

```
> summary(birthwt[,1:5])
  low           age             lwt           race     smoke
 No :130   Min.   :14.00   Min.   : 80.0   White:96   No :115
 Yes: 59   1st Qu.:19.00   1st Qu.:110.0   Black:26   Yes: 74
           Median :23.00   Median :121.0   Other:67
           Mean   :23.24   Mean   :129.8
           3rd Qu.:26.00   3rd Qu.:140.0
           Max.   :45.00   Max.   :250.0
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

On peut à présent facilement répondre aux questions suivantes : quel est le poids moyen des femmes qui fumait durant leur grossesse, combien d'antécédents d'hypertension dénombre-t-on chez les femmes pesant plus de 60 kg (sachant que les mesures dans le tableau sont exprimées en livres), quel est le poids minimal des bébés chez les mères n'ayant pas manifesté d'irritabilité utérine. Voici une solution possible pour chacune de ces questions :

```
> mean(birthwt$lwt[birthwt$smoke == "Yes"]) ## poids en livres
[1] 128.1351
> table(birthwt$ht[birthwt$lwt/2.2 > 60])   ## poids en kg

 No Yes
 55   7
> min(birthwt$bwt[birthwt$ui == "No"])
[1] 1135
```

Par la suite, plutôt que de répéter systématiquement le nom du data frame suivi de `$` pour accéder à une ou plusieurs variables, comme dans l'exemple précédent, on préférera utiliser la commande `with()` qui permet d'indiquer dans quel data frame on souhaite chercher des variables :

```
> with(birthwt, lwt[1:5])
[1] 182 155 105 108 107
> with(birthwt, mean(lwt[smoke == "Yes"]))
[1] 128.1351
```

1.7. Ce qu'il faut retenir

– Une variable sera représentée sous la forme d'une liste de nombres ou de caractères (simples ou multiples).

– Les variables seront généralement arrangées en colonnes dans un tableau rectangulaire (data frame).

– Les opérations arithmétiques usuelles opèrent sur chaque élément (observation) d'une variable.

– Il est possible d'indexer des observations par leur numéro ou position dans une liste de valeurs, ou en utilisant des expressions logiques.

1.8. Pour aller plus loin

Le livre de Phil Spector [SPE 08] fournit des informations complémentaires sur l'importation de sources de données externes (incluant le cas des bases de données relationnelles) et la fusion de sources avec la commande `merge()`, le traitement

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

des chaînes de caractères et la représentation des dates sous R. D'autres ouvrages sont disponibles en ligne, gratuitement ou pour un prix modéré, par exemple *R Programming for Data Science* de Roger Peng (https://leanpub.com/rprogramming) ou *The Elements of Data Analytic Style* de Jeff Leek (https://leanpub.com/datastyle).

1.9. Applications

1) La charge virale plasmatique permet de décrire la quantité de virus (par exemple, VIH) dans un échantillon de sang. Ce marqueur virologique qui permet de suivre la progression de l'infection et de mesurer l'efficacité des traitements est rapporté en nombre de copies par millilitre, et la plupart des instruments de mesure ont un seuil de détectabilité de 50 copies/ml. Voici une série de mesures, X, exprimées en logarithmes (base 10) collectées sur vingt patients :

```
3.64 2.27 1.43 1.77 4.62 3.04 1.01 2.14 3.02 5.62 5.51 5.51 1.01
1.05 4.19 2.63 4.34 4.85 4.02 5.92
```

Pour rappel, une charge virale de 100 000 copies/ml équivaut à 5 log.

a) Indiquer combien de patients ont une charge virale considérée comme non-détectable.

b) Le chercheur réalise que la valeur 3,04 correspond à une erreur de saisie et doit être changée en 3,64. De même, il a un doute sur la septième mesure et décide de la considérer comme une valeur manquante : effectuer les transformations correspondantes.

c) Quelle est le niveau de charge virale médian, en copies/ml, pour les données considérées comme valides ?

Avant toute chose, il est nécessaire d'exprimer la limite de détection (50 copies/ml) en unités logarithmiques ; celle-ci vaut en fait :

```
> log10(50)
```

Ensuite, il suffit de filtrer les observations qui ne remplissent pas la condition $X > 1,70$ (on utilisera le résultat numérique exact, pas la valeur approchée) :

```
> X <- c(3.64,2.27,1.43,1.77,4.62,3.04,1.01,2.14,3.02,5.62,5.51,
         5.51,1.01,1.05,4.19, 2.63,4.34,4.85,4.02,5.92)
> length(X[X <= log10(50)])
```

Pour remplacer l'observation valant 3,04, on se contentera de réaliser un test logique simple dans la mesure où cette observation n'est pas dupliquée dans la série d'observations :

```
> X[X == 3.04] <- 3.64
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Quant à la septième observation, on procédera de la même manière en mettant à jour la valeur de l'observation par indexation de la variable :

```
> X[7] <- NA
```

Finalement, la charge virale médiane pour les patients ayant une mesure considérée comme valide peut se calculer comme suit :

```
> Xc <- X[X > log10(50)]
> round(median(10^Xc), 0)
```

2) Le fichier `dosage.txt` contient une série de quinze dosages biologiques, stockés au format numérique avec trois décimales, comme suit :

```
6.379 6.683 5.120 ...
```

a) Utiliser `scan` pour lire ces données (bien lire l'aide en ligne concernant l'usage de cette commande, en particulier l'option `what=`).

b) Corriger la série de mesures afin de pouvoir calculer la moyenne arithmétique.

c) Enregistrer les données corrigées dans fichier texte appelé `data.txt`.

Lorsque les données stockées dans un fichier texte ne sont pas trop volumineuses, il est recommandé d'y jeter un œil à l'aide d'un simple éditeur de texte avant de les importer dans R. Dans le cas présent, il s'avère qu'il y a eu un problème de codage du séparateur décimal : pour une des mesures, une virgule a été utilisée à la place d'un point pour séparer la partie entière de la partie décimale des nombres. Si l'on se contente de l'instruction suivante :

```
> x <- scan("dosage.txt")
```

R retournera un message d'erreur du type :

```
Error in scan(file, what, nmax, sep, dec, quote, skip, nlines, :
  scan() attendait 'a real' et a reçu '2,914'
```

puisque par défaut R tente de lire des nombres (au format anglo-saxon). Pour corriger cela, il faut donc préciser le format sous lequel on souhaite stocker les données :

```
> x <- scan("dosage.txt", what="character")
> str(x)
> head(x)
```

Toutefois, il n'est pas possible de calculer directement la moyenne sur les données importées de cette manière puisqu'il s'agit de caractères (`character`). On corrigera donc l'observation mal enregistrée, avant de convertir les mesures en nombres exploitables par R :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> x[x == "2,914"] <- "2.914"
> x <- as.numeric(x)
> head(x)
> round(mean(x), 3)
```

Enfin, pour sauvegarder les données corrigées dans un nouveau fichier, on peut utiliser la commande générale `write.table()`, qui possède beaucoup d'options, mais qui en général fonctionne de la manière suivante :

```
> write.table(x, file="data.txt", row.names=FALSE)
```

Un aperçu du fichier texte ainsi sauvegardé est fourni ci-après :

```
"x"
6.379
6.683
5.12
6.707
6.149
5.06
```

Si l'on ne souhaite pas faire apparaître le nom de la variable sur la première ligne, on rajoutera l'option `col.names=FALSE`.

3) Le fichier `anorexia.dat` contient les données d'une étude clinique chez des patientes anorexiques ayant reçu l'une des trois thérapies suivantes : thérapie comportementale, thérapie familiale, thérapie contrôle [HAN 93].

a) Combien y a-t-il de patientes au total ? Combien y a-t-il de patientes par groupe de traitement ?

b) Les mesures de poids sont en livres. Les convertir en kilogrammes.

c) Créer une nouvelle variable contenant les scores de différences (`After - Before`).

d) Indiquer la moyenne et l'étendue (min/max) des scores de différences par groupe de traitement.

Un aperçu des données contenues dans le fichier `anorexia.dat` est fourni ci-après (cinq premières lignes du fichier) :

```
Group Before After
g1 80.5  82.2
g1 84.9  85.6
g1 81.5  81.4
g1 82.6  81.9
```

On voit donc que la première ligne est une ligne d'en-tête indiquant le nom des variables, et chacune des lignes suivantes représente une unité statistique pour laquelle

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

on a le groupe d'appartenance, la mesure de poids avant le début de la prise en charge et la mesure de poids à la fin de la thérapie. Pour importer ce type de données, on utilisera la commande `read.table()` en précisant l'option `header=TRUE` pour bien prendre en compte la ligne d'en-tête :

```
> anorex <- read.table("anorexia.dat", header=TRUE)
> names(anorex)
> head(anorex)
```

Le nombre total de patientes correspond au nombre de lignes du tableau de données :

```
> nrow(anorex)
```

Il y a donc au total 72 patientes. Pour trouver la répartition des effectifs par groupe, le plus simple est de faire un tri à plat de la variable qualitative `Group` :

```
> table(anorex$Group)
```

Pour convertir les poids exprimés en livres en kilogrammes, il suffit de diviser chaque mesure par 2,2 (approximativement). On réalise cette opération pour chacune des deux variables `Before` et `After` :

```
> anorex$Before <- anorex$Before/2.2
> anorex$After <- anorex$After/2.2
```

On notera que dans ce cas-là, les valeurs d'origine de ces deux variables seront simplement remplacées. Si l'on souhaite y avoir accès de nouveau, il faudra recharger le fichier. Une autre solution aurait consisté à créer deux nouvelles variables, comme ceci :

```
> anorex$Before.kg <- anorex$Before/2.2
> anorex$After.kg <- anorex$After/2.2
```

Pour calculer les scores de différences `After - Before`, il suffit de soustraire les valeurs des deux variables, en se rappelant que ce genre d'opérations opère élément par élément (c'est-à-dire pour chaque unité statistique). Ici, on ajoutera la variable nouvellement créée au tableau de données `anorex` :

```
> anorex$poids.diff <- anorex$After - anorex$Before
> head(anorex)
```

Enfin, pour calculer la moyenne et l'étendue des scores de différence par groupe de traitement, on peut procéder de deux manières : soit on isole chaque groupe et on calcule les statistiques demandées, soit on « factorise » l'opération de calcul en opérant par modalité de la variable qualitative. La première solution s'obtiendrait ainsi, pour le premier groupe par exemple :

```
> mean(anorex$poids.diff[anorex$Group == "g1"])
> range(anorex$poids.diff[anorex$Group == "g1"])
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 2

Statistiques descriptives et estimation

A partir de ce chapitre et jusqu'à celui traitant de la régression logistique, on travaillera avec le même jeu de données sur les poids à la naissance [HOS 89]. L'objectif de cet ouvrage est de présenter les principaux outils disponibles sous R pour la description univariée d'une variable numérique ou catégorielle. Le résumé d'une variable peut se faire de manière numérique à l'aide des indicateurs de tendance centrale et de dispersion pour une variable numérique, ou d'un tableau d'effectifs ou de fréquences pour une variable catégorielle. La distribution des valeurs observées pour une variable peut également être visualisée sous forme d'un graphique, soit en utilisant les données individuelles (diagramme de dispersion univariée) soit en utilisant une forme de résumé de l'information (histogramme, boîte à moustaches, diagrammes en barres ou en points).

2.1. Résumer une variable numérique

2.1.1. *Tendance centrale et forme de la distribution*

Considérons la variable `bwt` qui représente les poids à la naissance des nouveaux-nés. Les poids sont exprimés en grammes et il n'y a pas de valeurs manquantes. On peut le vérifier rapidement en comptant le nombre de valeurs pour lesquelles la commande `is.na()` renvoie vrai (`TRUE`) :

```
> sum(is.na(birthwt$bwt))
[1] 0
```

Dans l'expression précédente, on combine deux commandes : la commande `is.na()` renvoie la valeur `TRUE` si l'un des éléments de `bwt` est considéré comme une valeur manquante, `FALSE` sinon. Le résultat de cette commande contient donc autant d'éléments que la variable `bwt` et on dénombre le nombre de valeurs valant `TRUE`, celles-ci étant représentées de manière interne comme des nombres valant 1 (d'où l'usage de la commande `sum()`).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Les principaux indicateurs de tendance centrale, moyenne et médiane, peuvent être obtenus avec les commandes `mean()` et `median()`. Elles supposent qu'il n'y a pas de valeur manquante, ce qui est le cas ici ; le cas échéant il est nécessaire de rajouter l'option `na.rm=TRUE` :

```
> mean(birthwt$bwt)
[1] 2944.587
> median(birthwt$bwt)
[1] 2977
```

Comme on l'a vu dans le chapitre précédent, on peut également utiliser directement la commande `summary()` qui a l'avantage d'afficher l'étendue des valeurs observées (minimum et maximum, voir les commandes `min()`, `max()` et `range()`) et les quartiles (la médiane constituant le deuxième quartile) :

```
> summary(birthwt$bwt)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    709    2414    2977    2945    3487    4990
```

Les quartiles peuvent être retrouvés de la manière suivante :

```
> quantile(birthwt$bwt)
  0%  25%  50%  75% 100%
 709 2414 2977 3487 4990
```

L'instruction suivante :

```
> quantile(birthwt$bwt, probs=seq(0.1, 1, by=0.1))
```

permet d'obtenir la valeur des déciles de la variable `bwt`. Notons que plutôt que d'écrire chaque décile (10 %, 20 %, etc.) on a préféré laisser R générer lui-même la séquence de déciles à l'aide de la commande `seq()`.

En ce qui concerne l'estimation de l'asymétrie ou de l'aplatissement de la distribution, il n'existe pas de commandes de base sous R, mais elles sont disponibles dans le package `e1071` (`skewness()` et `kurtosis()`). Ce package doit être installé et chargé au préalable avant de pouvoir utiliser ces commandes.

2.1.2. *Indicateurs de dispersion*

Pour les mesures de dispersion, on dispose déjà des quartiles qui permettent d'estimer l'intervalle interquartile (50 % des observations). On pourrait donc très bien calculer cet intervalle comme suit :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> quantile(birthwt$bwt, probs=c(0.75))
  - quantile(birthwt$bwt, probs=c(0.25))
 75%
1073
```

ou plus simplement :

```
> diff(quantile(birthwt$bwt, probs=c(0.25, 0.75)))
 75%
1073
```

puisque la commande `diff()` renvoie les différences itérées des valeurs d'une variable. L'affichage 75% au-dessus de la valeur de l'intervalle interquartile est dû au fait que R « nomme » explicitement les quantiles comme on a pu le constater en utilisant `quantile(birthwt$bwt`. Une alternative consisterait à utiliser directement la commande `IQR()`.

R fournit également un estimateur de l'écart-type et de la variance (avec un dénominateur à $n-1$) *via* les commandes `sd()` et `var()` :

```
> var(birthwt$bwt)
[1] 531753.5
> sd(birthwt$bwt)
[1] 729.2143
> sd(birthwt$bwt)^2
[1] 531753.5
```

Les mêmes précautions d'usage concernant les valeurs manquantes s'appliquent, et il faudra ajouter l'option `na.rm=TRUE` en présence de données manquantes.

2.2. Résumer une variable catégorielle

Concernant les variables qualitatives, que les modalités de la variable soient ordonnées ou non, on peut toujours fournir un tableau d'effectifs ou de fréquences. La commande `table()` qui a été utilisée au chapitre précédent permet de fournir un tableau d'effectifs. Comme les commandes `mean()` ou `median()`, il est nécessaire de préciser que faire en présence de valeurs manquantes. Par défaut, R ne les affichera pas comme une catégorie à part ; si l'on souhaite afficher explicitement les observations manquantes, il faut ajouter l'option `useNA="always"` :

```
> table(birthwt$race)

White Black Other
   96    26    67
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Il est tout à fait possible de sauvegarder les résultats intermédiaires, y compris le résultat de commandes telles que `table()`, dans des variables R. Cela facilite leur réutilisation et améliore la lisibilité des commandes utilisant ces mêmes résultats. Dans l'exemple suivant, on combinera l'usage de `table()` avec `prop.table()` pour dresser un tableau de fréquences relatives plutôt que d'effectifs :

```
> tab <- table(birthwt$race)
> tab

White Black Other
   96    26    67
> prop.table(tab)

    White     Black     Other
0.5079365 0.1375661 0.3544974
```

On pourrait tout aussi bien remplacer ajouter `* 100` à l'expression précédente pour exprimer les résultats en pourcentages plutôt qu'en fréquences relatives. Notons que l'opération de multiplication s'effectue élément par élément, comme on l'a vu dans le cas des opérations sur les variables numériques (section 1.2.2).

A partir d'un tableau d'effectifs, on peut retrouver le mode en recherchant l'effectif le plus élevé, comme illustré ci-après :

```
> which.max(tab)
White
    1
```

Ici, `tab` représente bien le tableau construit à l'aide de `table(birthwt$race)`. Le résultat renvoyé par `which.max()` nous dit que la classe modale est en fait la première classe de la variable `race` (`White`, qui figure en première position dans la liste des modalités de la variable, voir `levels(birthwt$race)`).

Il existe une autre commande R qui permet de réaliser le même type d'opération (tris à plat et tris croisés, ces derniers étant présentés au chapitre suivant), `xtabs()` :

```
> xtabs(~ birthwt$race)
birthwt$race
White Black Other
   96    26    67
```

Cette commande fait intervenir une nouvelle notation, l'usage du symbole ~ qui est utilisé pour décrire des relations entre variables, par exemple pour exprimer une relation du type : une variable numérique, `x`, décrite par une variable catégorielle, `z`

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

(`x ~ z`). Dans le cas univarié, la notation `~ birthwt$race` indique à R de décrire les variations de la variable `race` du tableau `birthwt`, ce qui pour une variable qualitative équivaut à la distribution des effectifs par classe ou modalité de la variable. On appelle cette notation une « notation par formule », et l'on y reviendra dans les chapitres suivants.

Dans certaines situations, on peut vouloir résumer une variable numérique à l'aide d'un tableau d'effectifs en considérant différents intervalles de classe (borne inférieure et supérieure entre lesquelles les valeurs de la variable numérique se situent) : on se retrouve alors avec une variable catégorielle avec laquelle on peut utiliser `table()` ou `xtabs()`. Considérons l'âge des mères (`age`, qui varie entre 14 et 45 ans), et les intervalles de classe suivants : $[15, 19]$, $[20, 24]$, $[25, 29]$, $[30, 45]$. Tels qu'ils sont définis, ces intervalles incluent les valeurs indiquées pour leurs bornes inférieures et supérieures. Voici une manière de construire le tableau d'effectifs correspondant à cette répartition des individus dans des classes d'âge mutuellement exclusives :

```
> table(cut(birthwt$age, breaks=c(15,19,24,29,45),
            include.lowest=TRUE))

[15,19] (19,24] (24,29] (29,45]
     48      69      42      27
```

On remarquera que l'usage de la commande `cut()` nécessite d'indiquer à R comment découper les valeurs de la variable numérique, et en particulier si les bornes inférieures ou supérieures doivent être inclusives. Dans cet exemple, on indique que les intervalles doivent inclure la borne inférieure du 1er intervalle (`include.lowest=TRUE`). Une autre option utile est `right=`; celle-ci est utilisée pour indiquer si les intervalles doivent être ouverts à droite ou non (par défaut, R constitue des intervalles dont les bornes sont ouvertes à gauche et fermées à droite).

2.3. Représenter graphiquement la distribution d'une variable

Toutes les commandes graphiques utilisées dans l'ouvrage sont fournies par le package `lattice` qui est disponible avec l'installation de base du logiciel R. Il existe également des commandes graphiques basiques, mais dans la mesure où `lattice` fournit des graphiques dont les options par défaut sont relativement satisfaisantes on préférera utiliser ce dernier. Une introduction au package `lattice` est fournie en annexe.

Pour pouvoir utiliser ces commandes, il est nécessaire de taper :

```
> library(lattice)
```

avant d'utiliser une des commandes du package. Il n'est pas nécessaire de réitérer l'opération tant que l'on ne ferme pas la session R en cours.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

2.3.1. *Cas des variables numériques*

Pour représenter graphiquement l'ensemble des observations, on peut utiliser un simple diagramme de dispersion à une dimension à l'aide de `stripplot()` dont un exemple d'utilisation est fourni ci-après avec une variable supposée être appelée `poids`. Lorsque le nombre d'observations n'est pas trop important, cela permet de visualiser rapidement les données plus ou moins extrêmes et la tendance centrale :

```
> stripplot(~ poids)
```

On retrouve ici la notation par formule présentée plus haut : ici, dans un contexte univarié, on indique à R de décrire les variations de la variable `poids`, ce qui pour une variable numérique équivaut à la distribution de l'ensemble des observations, sans considération d'un quelconque facteur de classification ou de stratification. Il est également possible d'utiliser ce type de représentation graphique avec des données de taille importante, mais dans ce cas il y a un risque que les points se chevauchent rendant l'interprétation de la distribution univariée plus délicate. Une alternative consiste à ajouter un léger décalage, horizontal ou vertical selon l'axe choisi pour représenter la variable mesurée, aux points sur le graphique. Ceci est illustré dans la figure 2.1 avec les poids des bébés :

```
> stripplot(~ bwt, data=birthwt, jitter.data=TRUE, amount=.05,
            xlab="Poids (g)")
```

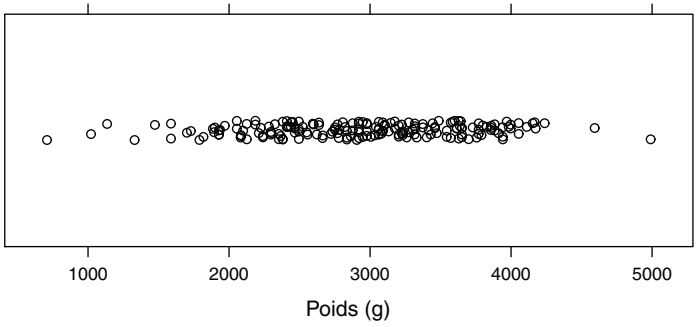


Figure 2.1. *Diagramme en points pour la distribution des poids des bébés*

Le décalage vertical des points est ici contrôlé par l'option `jitter.data=TRUE` (on ajoute en fait un aléa gaussien pour chaque point sur l'axe des ordonnées), et on peut augmenter l'amplitude du décalage avec l'option `amount=`.

Une représentation bien connue pour les variables continues est un histogramme des effectifs ou des fréquences en définissant une série d'intervalles de classe pour les

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

valeurs prises par la variable numérique. R offre différentes options pour le choix du calcul des intervalles de classe, et généralement ils sont satisfaisants. On peut toujours spécifier manuellement le nombre d'intervalles (option `nint=`) ou les classes elles-mêmes (option `breaks=`). Par exemple, pour découper les effectifs sur des intervalles de taille 500, on utiliserait une option telle que `breaks=seq(700, 5000, by=500)`, les bornes 700 et 5 000 étant légèrement en dehors de l'intervalle des valeurs observées pour la variable `bwt`. Par défaut, R affiche un histogramme de proportions, ce que l'on peut changer en ajoutant l'option `type="count"` pour obtenir un histogramme des effectifs. Voici le résultat illustré dans la figure 2.2 pour la variable `bwt` :

```
> histogram(~ bwt, data=birthwt, type="count", xlab="Poids (g)",
            ylab="Effectifs")
```

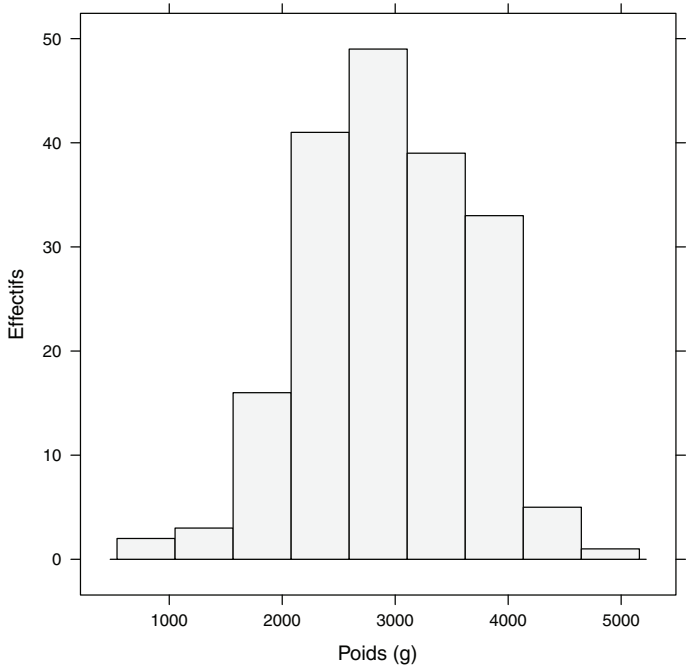


Figure 2.2. *Histogramme pour la distribution des poids des bébés*

Ici, on introduit un concept important dans R : de nombreuses commandes offrent la possibilité de désigner le data frame dans lequel se trouvent les variables à l'aide de l'option `data=`, et d'utiliser directement le nom des variables plutôt que de les préfixer systématiquement avec le nom du tableau suivi du caractère `$`. Avec la notation par formule, ces deux concepts constituent la base de la description des données statistiques sous R. Un autre intérêt de cette approche est la possibilité de

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

ne considérer qu'un sous-ensemble des observations du tableau de données spécifié par l'option `data=`. Pour cela, il suffit d'ajouter l'option `subset=` et d'indiquer une condition logique permettant de filtrer les unités statistiques, ce filtre pouvant porter sur la variable d'intérêt, par exemple `subset = bwt > 3000` (travailler uniquement sur les bébés pesant plus de 3 kg), ou sur n'importe quelle variable (ou combinaison de variables grâce aux opérateurs logiques `&` et `|`) disponibles dans le data frame, par exemple `smoke == "Yes"` (bébés dont la mère fume) :

```
> histogram(~ bwt, data=birthwt, subset=smoke == "Yes",
            type="count")
```

Une alternative à l'histogramme est la courbe de densité non paramétrique qui permet de s'affranchir du problème du choix des intervalles de classe. En l'occurence, plutôt que de regrouper les observations dans des classes définies à l'avance (manuellement ou à l'aide d'un algorithme spécifique), on peut tout à fait utiliser des méthodes plus sophistiquées pour estimer la courbe de densité, en contrôlant le degré de « lissage » de cette approximation, comme illustré dans la figure 2.3 :

```
> densityplot(~ bwt, data=birthwt, xlab="Poids (g)",
              ylab="Densité")
```

La taille de la fenêtre de lissage de la courbe se contrôle avec l'option `bw=` ; plus elle est élevée, plus les variations locales sont atténuées (par exemple, avec `bw=1000` la courbe apparaît beaucoup plus proche d'une courbe gaussienne pour les poids des bébés). L'aide en ligne pour la commande `density()` fournit des informations additionnelles sur les différents paramètres pouvant être contrôlés. Concernant `lattice` spécifiquement, l'aide en ligne correspondante s'obtient avec :

```
help(panel.densityplot)
```

Enfin, une dernière façon de représenter graphiquement la distribution d'une variable numérique est un diagramme de type boîte à moustaches (figure 2.4). Dans ce type de représentation, la « boîte » contient 50 % des observations de sorte que ces deux bords représentent le premier et le troisième quartiles. La médiane est représentée par un point à l'intérieur de cette boîte. Les « moustaches » représentent les valeurs minimale et maximale de la distribution, sauf dans le cas où certaines observations sont situées à plus de $1,5 \times$ IQR (IQR = intervalle interquartile) du premier ou du troisième quartile auquel cas ces dernières sont indiquées explicitement sur le graphique et les "moustaches" représentent alors les bornes des valeurs limites ($1.5 \times$ IQR du premier ou troisième quartile, méthode de Tukey, voir `help(boxplot.stats)`) :

```
> bwplot(~ bwt, data=birthwt, xlab="Poids (g)")
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

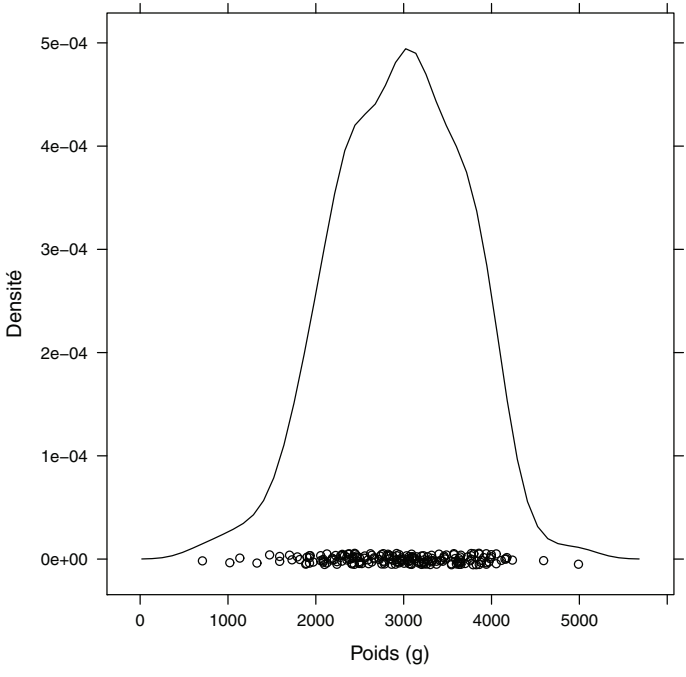


Figure 2.3. *Courbe de densité estimée pour la distribution des poids des bébés*

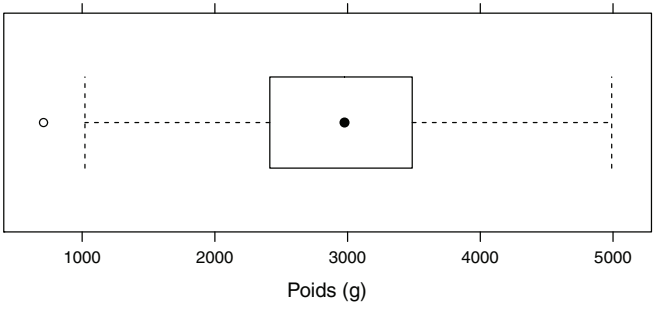


Figure 2.4. *Représentation de type boîte à moustaches pour la distribution des poids des bébés*

2.3.2. *Cas des variables catégorielles*

Dans le cas des variables catégorielles, on utilisera soit des diagrammes en barres soit des diagrammes en points (diagramme de Cleveland), en évitant dans la mesure

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

du possible les représentations circulaires de type « boîte à camembert » qui ne fournissent de toute façon pas plus d'informations que les diagrammes précédents.

Dans les deux cas, il est nécessaire de travailler avec les données tabulées, c'est-à-dire un tableau d'effectifs construit à l'aide de `table()` ou `xtabs()`. On peut également associer ces commandes à `prop.table()` pour obtenir un tableau de fréquences relatives. Comme on l'a vu plus haut, la commande `xtabs()` permet une notation par formule ce qui permet de souligner le rôle joué par les variables et de désigner directement le data frame dans lequel se trouvent les variables en question. Voici un exemple de diagramme en barres pour une seule variable (figure 2.5) :

```
> barchart(xtabs(~ race, data=birthwt), xlab="Ethncité",
           ylab="Effectifs", horizontal=FALSE)
```

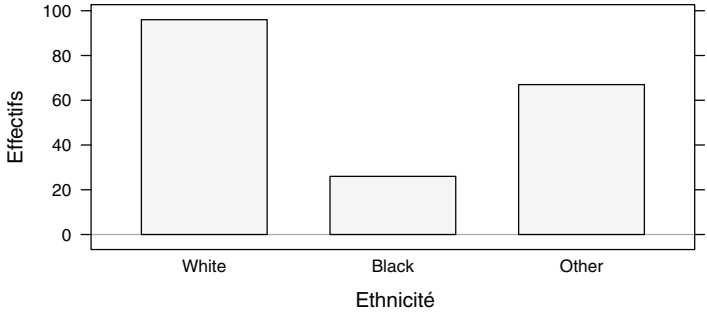


Figure 2.5. *Diagramme en barres pour la distribution des effectifs selon l'ethnicité de la mère*

Il est possible de changer l'orientation des barres (par défaut horizontales) grâce à l'option `horizontal=FALSE` (barres verticales). Les diagrammes en points, comme illustré dans la figure 2.6, sont construits exactement de la même manière, avec la commande `dotplot()` qui partage une grande partie des options de base de la commande `barchart()` (en particulier l'option `horizontal=FALSE` pour un diagramme orienté verticalement) :

```
> dotplot(xtabs(~ race, data=birthwt), ylab="Ethnicité",
          xlab="Effectifs")
```

2.4. Estimation par intervalles pour une moyenne ou une proportion

2.4.1. *Intervalles de confiance pour une moyenne*

Comme on l'a vu, les commandes `mean()` et `summary()` fournissent toutes les deux une estimation de la moyenne (arithmétique) pour une variable numérique. Pour

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

obtenir des intervalles de confiance à $100(1-\alpha)$ %, où α désigne le risque de première espèce (habituellement 5 %), et en se référant à une distribution gaussienne (loi « normale »), il est nécessaire d'avoir une estimation de l'erreur standard et de connaître la valeur du quantile de la loi normale standard correspondant. Pour ce dernier, plutôt que d'utiliser l'approximation $\pm 1,96$ (pour $\alpha = 5$ %, soit 2,5 % pour chaque queue de distribution), on peut obtenir directement sa valeur sous R en utilisant la commande `qnorm()` qui fournit les fractiles d'une loi normale standard $\mathcal{N}(0;1)$:

```
> qnorm(0.975)
[1] 1.959964
```

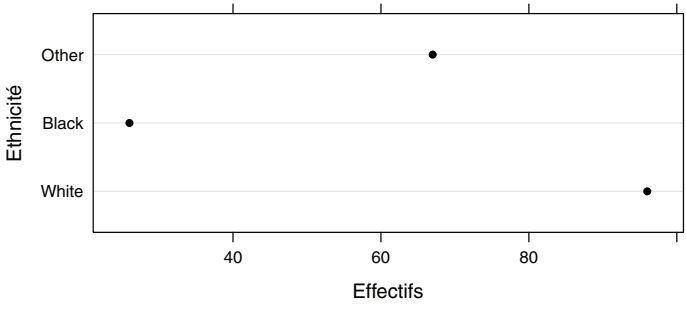


Figure 2.6. *Diagramme en points pour la distribution des effectifs selon l'ethnicité de la mère*

On pourra vérifier que la commande `1-pnorm(1.96)` renvoie l'aire sous la courbe de densité pour la queue de distribution à droite.

Quant à l'erreur standard, définie comme le rapport entre l'écart-type estimé sur les données et la racine carrée de la taille de l'échantillon, on l'obtiendra simplement comme :

```
> sd(birthwt$bwt) / sqrt(length(birthwt$bwt))
[1] 53.04254
```

Comme on l'a vu au chapitre précédent, plutôt que de répéter systématiquement le nom du data frame suivi du symbole $ pour travailler avec une variable donnée, on peut tout à fait remplacer l'expression précédente par :

```
> with(birthwt, sd(bwt) / sqrt(length(bwt)))
```

ce qui indique à R de travailler avec la variable `bwt` disponible dans le tableau `birthwt`. On rappelle qu'il n'y a pas de valeurs manquantes dans ce jeu de données et donc la commande `length()` renvoie bien la taille de l'échantillon. Dans le cas

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

contraire, on écrirait `sqrt(sum(!is.na(birthwt$bwt)))`. En combinant les deux résultats précédents, on obtient assez facilement les bornes inférieures et supérieures d'un intervalle de confiance à 95 % pour la moyenne des poids des bébés dans cet échantillon :

```
> mean(birthwt$bwt) - qnorm(0.975) * sd(birthwt$bwt) /
  sqrt(length(birthwt$bwt))
[1] 2840.626
> mean(birthwt$bwt) + qnorm(0.975) * sd(birthwt$bwt) /
  sqrt(length(birthwt$bwt))
[1] 3048.549
```

On peut stocker ces deux résultats dans des variables auxiliaires et les combiner ensemble pour faciliter la lecture :

```
> bwt.lci <- mean(birthwt$bwt) - qnorm(0.975) * sd(birthwt$bwt) /
             sqrt(length(birthwt$bwt))
> bwt.uci <- mean(birthwt$bwt) + qnorm(0.975) * sd(birthwt$bwt) /
             sqrt(length(birthwt$bwt))
> c(bwt.lci, bwt.uci)
[1] 2840.626 3048.549
```

Notons que l'on aurait très bien pu ramener le calcul des deux bornes de l'intervalle de confiance à une seule commande :

```
> mean(birthwt$bwt) + c(-1,1) * qnorm(0.975) * sd(birthwt$bwt) /
  sqrt(length(birthwt$bwt))
```

Cela dit, plutôt que de calculer manuellement de tels intervalles de confiance, il est tout à fait possible d'utiliser le package `epiR` qui dispose d'une commande spécifique pour ce type de calcul : `epi.conf()`.

Dans le cas des petits échantillons, on peut préférer recourir à une distribution de Student pour construire l'intervalle de confiance associé à une moyenne. Les commandes `pt()` et `qt()` peuvent être utilisées en lieu et place de `pnorm()` et `qnorm()` pour obtenir les valeurs de probabilité ou de fractile d'une loi t pour ν degrés de liberté. Les degrés de liberté pour la distribution de Student sont contrôlés par l'option `df=`. Le fractile associé à une probabilité de 0,025 à droite d'une distribution à 18 degrés de liberté est ainsi :

```
> qt(0.975, df=18)
[1] 2.100922
```

Comme on le verra dans le chapitre suivant, une autre alternative pour obtenir ce type d'intervalles de confiance est d'utiliser directement une procédure de test,

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

`t.test()` par exemple, qui renvoie le résultat d'un test d'hypothèse et fournit l'intervalle de confiance associé à la statistique de test. Dans le cas présent, on utiliserait un test t de Student pour un échantillon, en considérant comme hypothèse nulle $H_0 : \mu = 0$.

2.4.2. *Intervalles de confiance pour une proportion*

En utilisant une approximation normale pour l'estimation de l'intervalle de confiance d'une proportion, on procéderait exactement de la même manière que celle décrite avant, en remplaçant les estimations de la moyenne par celle de la fréquence p (obtenue à partir d'une commande `table()`, par exemple) et de l'erreur standard par $\sqrt{p(1-p)/n}$. Il s'agit, comme dans le cas précédent, d'une approximation qui est valable essentiellement pour les grands échantillons.

Avec les données sur le statut fumeur des mères (variable `smoke`), un intervalle de confiance à 95 % pour la proportion de mères fumant durant leur grossesse peut donc être formulé ainsi :

```
> tab <- prop.table(table(birthwt$smoke))
> p <- tab[2]
> n <- nrow(birthwt)
> p + c(-1,1) * qnorm(0.975) * sqrt((p*(1-p))/n)
[1] 0.3219487 0.4611201
```

La séquence de commandes précédente permet d'illustrer quelques-unes des notions évoquées depuis le premier chapitre : on peut tout à fait stocker le résultat d'une commande `table()` ou `prop.table()` dans une variable auxiliaire. Ici, il s'agit d'un tri simple de la variable `smoke` et donc la variable `p` contient deux valeurs :

```
> tab

       No       Yes
0.6084656 0.3915344
```

la deuxième correspondant à la proportion de mères fumeuses : `tab[2]` permet donc d'accéder à cette valeur. Une alternative consisterait à utiliser la syntaxe `tab["Yes"]` (nom de la colonne du tableau au lieu du numéro de colonne). Il n'y a pas de données manquantes, donc `length(birthwt$bwt)` et `nrow(birthwt)` renverront essentiellement la même information : la taille de l'échantillon. On en profite pour associer cette quantité à une variable appelée `n`. Le reste correspond à l'expression de l'intervalle de confiance $p \pm 1,96\sqrt{p(1-p)/n}$ en langage R.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

2.5. Ce qu'il faut retenir

– La commande `summary()` fournit l'essentiel des informations concernant la forme de la distribution d'une variable numérique et un tableau d'effectif pour les variables catégorielles. La plupart de ces informations peuvent être obtenues individuellement à l'aide de commandes spécifiques telles que `mean()`, `sd()`, `quantile()` ou `table()`.

– Une notation par formule est utilisée pour décrire les variations d'une variable, et celle-ci sert de point d'entrée aux commandes graphiques de base pour représenter la distribution d'une variable numérique ou catégorielle.

– Pour construire des intervalles de confiance asymptotiques pour une moyenne ou une proportion estimée à partir des données observées, on peut traduire directement leur formulation mathématique en langage R à partir de la commande `qnorm()` et du calcul de l'erreur standard associée à ces quantités.

2.6. Applications

1) Une variable quantitative X prend les valeurs suivantes sur un échantillon de 26 sujets :

```
24.9,25.0,25.0,25.1,25.2,25.2,25.3,25.3,25.3,25.4,25.4,25.4,25.4,
25.5,25.5,25.5,25.5,25.6,25.6,25.6,25.7,25.7,25.8,25.8,25.9,26.0
```

a) Calculer la moyenne, la médiane ainsi que le mode de X.

b) Quelle est la valeur de la variance estimée à partir de ces données ?

c) En supposant que les données sont regroupées en quatre classes dont les bornes sont : 24,9-25,1, 25,2-25,4, 25,5-25,7, 25,8-26,0, afficher la distribution des effectifs par classe sous forme d'un tableau d'effectifs.

d) Représenter la distribution de X sous forme d'histogramme, sans considération d'intervalles de classe *a priori*.

Une manière de représenter ce type de données consiste à utiliser saisir les observations comme ceci :

```
> x <- c(24.9,25.0,25.0,25.1,25.2,25.2,25.3,25.3,25.3,25.4,25.4,
         25.4,25.4,25.5,25.5,25.5,25.5,25.6,25.6,25.6,25.7,25.7,
         25.8,25.8,25.9,26.0)
```

La médiane et la moyenne sont obtenues à l'aide des commandes `mean()` et `median()`, ou alors directement *via* la commande `summary()` :

```
> median(x)
> mean(x)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Concernant la médiane, on peut vérifier qu'il s'agit bien de la valeur correspondant au deuxième quartile ou au 50e percentile, c'est-à-dire la valeur de X telle que 50 % des observations lui sont inférieures :

```
> quantile(x)
```

Pour le mode, il est nécessaire d'afficher la distribution des effectifs selon les valeurs de X, puis vérifier à quelle valeur de X est associé l'effectif le plus grand :

```
> table(x)
> names(table(x)[table(x)==max(table(x))])
```

On pourra vérifier que, dans ce cas précis, on retrouve bien deux modes.

La variance est obtenue à l'aide de la commande `var()` :

```
> var(x)
```

En supposant que les données sont regroupées en quatre classes dont les bornes sont : 24,9-25,1, 25,2-25,4, 25,5-25,7, 25,8-26,0, on peut recalculer la distribution des effectifs par classe :

```
> xc <- cut(x, breaks=c(24.9,25.2,25.5,25.8,26.0),
           include.lowest=TRUE, right=FALSE)
> table(xc)
```

On remarquera que R utilise la notation anglo-saxonne pour représenter les bornes des intervalles de classe : le symbole `)` à droite d'un nombre signifie que ce nombre est exclu de l'intervalle, alors que `]` signifie que l'intervalle contient ce nombre. Par défaut, R détermine automatiquement les intervalles de classe d'un histogramme produit à l'aide de la commande `histogram()` du package `lattice`. Si l'on souhaite spécifier soi-même les bornes des intervalles de classe, on utilisera l'option `breaks=`. Par exemple, pour afficher la distribution des effectifs selon les quatre classes définies plus haut, on écrirait :

```
> histogram(~ x,type="count",breaks=c(24.9,25.2,25.5,25.8,26.0))
```

2) On dispose des temps de survie de 43 patients souffrant de leucémie granulocytaire chronique, mesurés en jours depuis le diagnostique [EVE 01] :

```
7,47,58,74,177,232,273,285,317,429,440,445,455,468,495,497,532,
571,579,581,650,702,715,779,881,900,930,968,1077,1109,1314,1334,
1367,1534,1712,1784,1877,1886,2045,2056,2260,2429,2509
```

a) Calculer le temps de survie médian.

b) Combien de patients ont une survie inférieure (strictement) à 900 jours au moment de l'étude ?

c) Quelle est la durée de survie associée au 90e percentile ?

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Les données sont saisies manuellement comme dans le cas précédent :

```
> s <- c(7,47,58,74,177,232,273,285,317,429,440,445,455,468,495,
         497,532,571,579,581,650,702,715,779,881,900,930,968,
         1077,1109,1314,1334,1367,1534,1712,1784,1877,1886,2045,
         2056,2260,2429,2509)
```

Le temps de survie médian s'obtient naturellement à l'aide de `median()` :

```
> median(s)
```

Pour déterminer le nombre de patients avec une survie $\leq$ 900 jours, il suffit d'effectuer un test logique et de construire un tableau des individus remplissant ou non la condition :

```
> table(s < 900)
```

La colonne libellée `TRUE` indique le nombre d'observations vérifiant la condition précédente.

La survie associée au 90e percentile peut s'obtenir à l'aide de la commande :

```
> quantile(s, 0.9)
```

On peut rapidement vérifier que le résultat correspond bien à la valeur de survie telle que 90 % des observations ne dépassent pas cette valeur :

```
> table(s <= quantile(s, 0.9))
```

Ici, $38/43$ est bien inférieur à 0,90 tandis que $39/43 = 0,91$.

3) Le fichier `elderly.dat` contient la taille mesurée en cm de 351 personnes âgées de sexe féminin, sélectionnées aléatoirement dans la population lors d'une étude sur l'ostéoporose [EVE 01]. Quelques observations sont cependant manquantes.

a) Combien y a t-il d'observations manquantes au total ?

b) Donner un intervalle de confiance à 95 % pour la taille moyenne dans cet échantillon, en utilisant une approximation normale.

c) Représenter la distribution des tailles observées sous forme d'une courbe de densité.

Pour lire le fichier qui ne se compose que d'une série de valeurs numériques, on utilise la commande `scan()`. Attention, comme il y a des valeurs manquantes, codées dans le fichier texte par des « . », il est nécessaire de s'assurer que ces observations sont bien identifiées comme telles par R :

```
> tailles <- scan("elderly.dat", na.strings=".")
```

On pourrait afficher la distribution des observations à l'aide d'un simple histogramme (`histogram(~ tailles)`). Cela permet en particulier de vérifier la forme générale de la distribution et la présence d'éventuelles valeurs « extrêmes ».

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Le nombre d'observations non complètes (données manquantes) est déterminé en comptant le nombre de valeurs manquantes (`NA`), ce qui est équivalent à établir un tri à plat des valeurs considérées comme manquantes par R à l'aide de la commande `table()` :

```
> sum(is.na(tailles))
> table(is.na(tailles))
```

On a donc cinq observations manquantes au total.

La moyenne est obtenue à l'aide de la commande `mean()`. Cependant, comme il existe des valeurs manquantes il est nécessaire de préciser l'option `na.rm=TRUE` pour indiquer à R de calculer la moyenne arithmétique sur les cas complets. La même remarque vaut pour l'usage de `sd()`. On a également besoin d'avoir une estimation de l'écart-type. Le quantile de référence pour la loi normale (97,5 %) est généralement pris à 1,96 dans les tables statistiques mais on peut obtenir sa valeur avec R grâce à `qnorm()`, comme on l'a vu plus haut. Soit au final :

```
> m <- mean(tailles, na.rm=TRUE)    # moyenne
> s <- sd(tailles, na.rm=TRUE)      # écart-type
> n <- sum(!is.na(tailles))         # nombre d'observations
> m - qnorm(0.975) * s/sqrt(n)      # borne inf. IC 95 %
> m + qnorm(0.975) * s/sqrt(n)      # borne sup. IC 95 %
```

En présence de valeurs manquantes, il faut explicitement s'assurer que l'on travaille bien sur les cas complets. La commande `sum(!is.na(tailles))` est équivalente à la commande, peut-être plus explicite, `sum(complete.cases (tailles))`. Les deux dernières instructions peuvent se simplifier en exploitant la capacité de R à répéter le même calcul sur des séries de données stockées dans une variable.

Pour représenter la distribution des tailles sous forme d'une courbe de densité, qui ne pose pas le problème du choix de classes *a priori*, la commande à utiliser est `densityplot()` :

```
> densityplot(~ tailles)
```

Le degré de lissage de la courbe de densité estimée à partir des données peut être contrôlé à l'aide de l'option `bw=`. L'exemple suivant produirait une courbe présentant beaucoup moins de variations locales, par exemple :

```
> densityplot(~ tailles, bw=4)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 3

Mesures et tests d'association entre deux variables

Dans ce chapitre, on s'intéressera plus particulièrement à quantifier, en termes de direction et d'amplitude, et à tester le degré d'association entre deux variables jouant un rôle symétrique ou non. Dans un premier temps, on discutera de la comparaison de moyennes pour deux échantillons, indépendants ou non (une variable joue le rôle de variable réponse, l'autre de variable explicative), à l'aide du test de Student, et son alternative non paramétrique (test de Wilcoxon-Mann-Whitney) pour deux échantillons indépendants ou non. Dans un second temps, on s'intéressera aux tableaux de contingence à deux entrées (échantillons indépendants) et aux mesures d'association classique (chi-deux) ou plus spécifique de l'épidémiologie (odds-ratio, risque relatif), incluant le cas des échantillons non indépendants (test de McNemar). Dans ces situations, les variables peuvent jouer un rôle symétrique ou non (cas du risque relatif). Avant de décrire ces procédures de test, on discutera les commandes R qui peuvent être utilisées pour résumer une structure de données composée de deux variables.

3.1. Statistiques descriptives bivariées

3.1.1. *Décrire une variable numérique selon les modalités d'une variable qualitative*

Considérons les variables âge de la mère (`age`) et poids des bébés (`low`). On souhaite estimer l'âge moyen des mères selon le statut pondéral du nouveau-né ($< 2\ 500$ grammes ou dans les normes). On peut très bien calculer les moyennes de groupe en sélectionnant les observations appartenant à chacun des groupes, ou niveaux (`levels`) du facteur d'étude. Par exemple, dans l'étude sur les poids à la naissance l'âge moyen des mères de bébés nés avec un poids inférieur à 2,5 kg vaut :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> with(birthwt, mean(age[bwt < 2500]))
[1] 22.30508
```

ou de manière équivalente `mean(age[low == "Yes"])`. On procéderait de la même manière pour calculer l'âge moyen des mères ayant eu un enfant dans les normes du point de vue du poids de naissance (`mean(age[low == "No"])`). Cependant, il est souvent plus commode de grouper ce genre d'opérations (calcul d'une statistique quelconque) à l'aide de la commande `tapply()` en indiquant le facteur sur lequel on souhaite opérer :

```
> with(birthwt, tapply(age, low, mean))
      No      Yes
23.66154 22.30508
```

La commande `tapply()` fonctionne sur le principe suivant : on indique le nom de la variable réponse sur laquelle on souhaite faire des calculs, puis le nom de la variable de classification (généralement de type `factor`) et enfin la commande à utiliser pour faire les calculs. Le troisième argument de la commande `tapply()` peut être une commande R, par exemple `mean`, `sum`, `summary`, ou n'importe quelle combinaison de commandes. Cette stratégie est connue sous le nom « split-apply-combine » [WIC 11] et son principe est illustré dans la figure 3.1 : on constitue des blocs de données pour la variable `y` correspondant aux différents niveaux d'un facteur (variable `A`), on applique une même commande (`mean()`) à chacun de ces blocs, et on assemble les résultats dans une même liste.

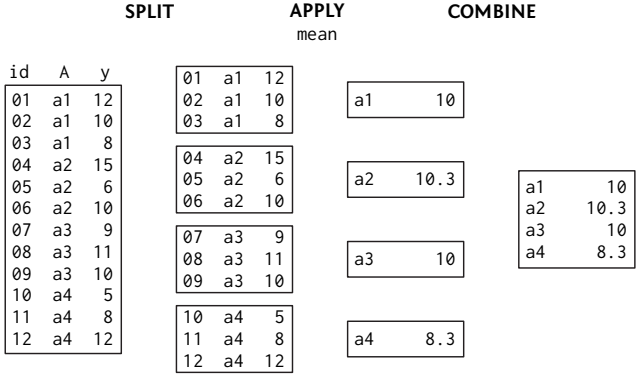


Figure 3.1. *Principe de l'approche « split-apply-combine »*

Plutôt que d'utiliser des commandes de base R, on pourrait très bien définir une « fonction » qui calcule la moyenne et l'écart-type d'une variable. Dans l'instruction suivante, on appelle notre fonction `f` et on indique à R de renvoyer une liste de deux

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

valeurs, nommées `m` et `s`, correspondant à la moyenne et à l'écart-type de n'importe quelle variable numérique, désignée par `x`, qui sera transmise à la fonction `f` :

```
> f <- function(x) c(m=mean(x), s=sd(x))
> with(birthwt, tapply(age, low, f))
$No
        m         s
23.661538  5.584522

$Yes
        m         s
22.305085  4.511496
```

On peut examiner la distribution des âges selon le statut pondéral à l'aide d'un histogramme, en indiquant la variable de stratification (`low`) à la suite de l'opérateur `|`. Il s'agit encore une fois d'une notation par formule, déjà évoquée au chapitre 2 : ici, on cherche à décrire les variations de la variable `age` selon les différents niveaux de `low`, c'est-à-dire conditionnellement à la variable `low`. Le conditionnement s'exprime à l'aide du symbole `|`. L'avantage de cette approche est que R utilisera le même système d'unités pour l'axe des ordonnées, ce qui facilite souvent la comparaison entre les distributions conditionnelles. De plus, les niveaux du facteur de classification sont automatiquement indiqués dans la partie supérieure de chaque graphique (figure 3.2) :

```
> histogram(~ age | low, data=birthwt, type="count")
```

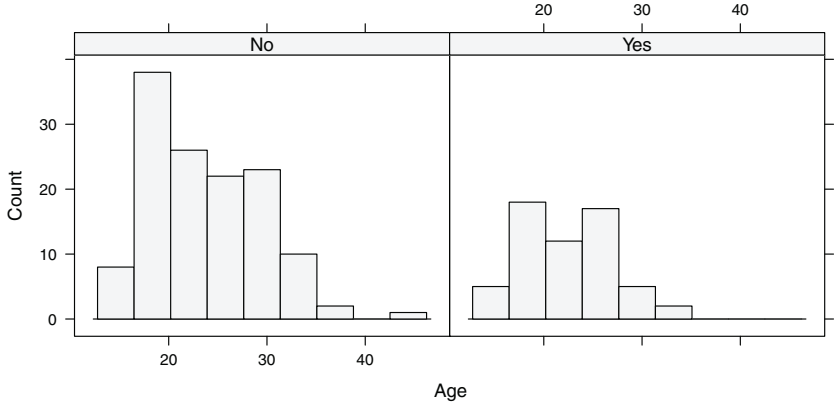


Figure 3.2. *Histogramme pour la distribution de l'âge des mères conditionnellement au statut pondéral à la naissance*

On pourrait tout aussi bien utiliser des courbes de densité au lieu d'histogrammes. La commande à utiliser est alors `densityplot()` et les options restent strictement

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

identiques à celles utilisées avec `histogram()`, à l'exception de `type="count"`. Dans l'illustration suivante, on a précisé une option `bw=2.5` pour indiquer à R de considérer une fenêtre de lissage assez grande, donc moins sensible aux variations locales des âges (figure 3.3) :

```
> densityplot(~ age | low, data=birthwt, bw=2.5)
```

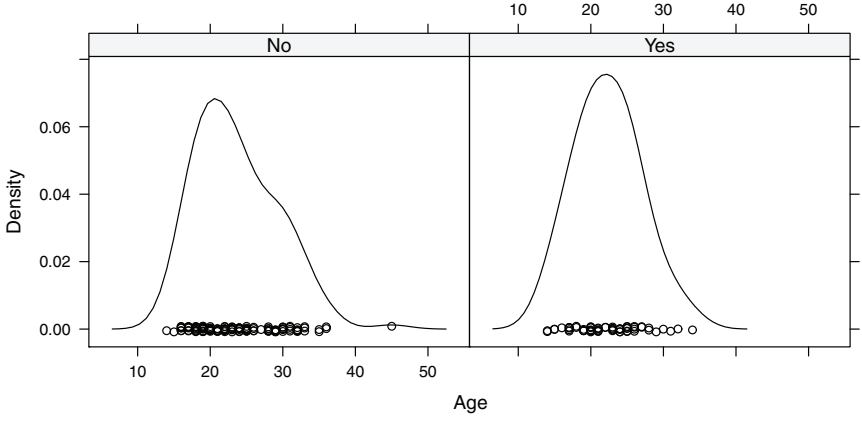


Figure 3.3. *Courbes de densité pour la distribution de l'âge des mères conditionnellement au statut pondéral à la naissance*

Une formulation alternative consiste à superposer, ou « grouper », les deux courbes de densité dans le même graphique, comme illustré dans la figure 3.4. Il est alors nécessaire d'ajouter une légende pour identifier les deux sous-échantillons, et l'option `auto.key=TRUE` fournit une légende adaptée avec un placement automatique (par défaut, dans la marge supérieure du graphique) :

```
> densityplot(~ age, data=birthwt, groups=low, bw=2.5,
              auto.key=TRUE)
```

3.1.2. *Décrire deux variables qualitatives*

Les commandes `table()` et `xtabs()`, utilisées dans les chapitres précédents pour résumer une variable qualitative sous forme d'un tableau d'effectifs, peuvent être utilisées pour construire un tableau croisant les modalités de deux variables qualitatives dans un tableau de contingence. Si l'on s'intéresse à la répartition des effectifs par modalités des variables `low` et `smoke` (statut pondéral des bébés et consommation de tabac chez les mères), on utilisera :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> with(birthwt, table(low, smoke))
     smoke
low   No Yes
  No  86  44
  Yes 29  30
```

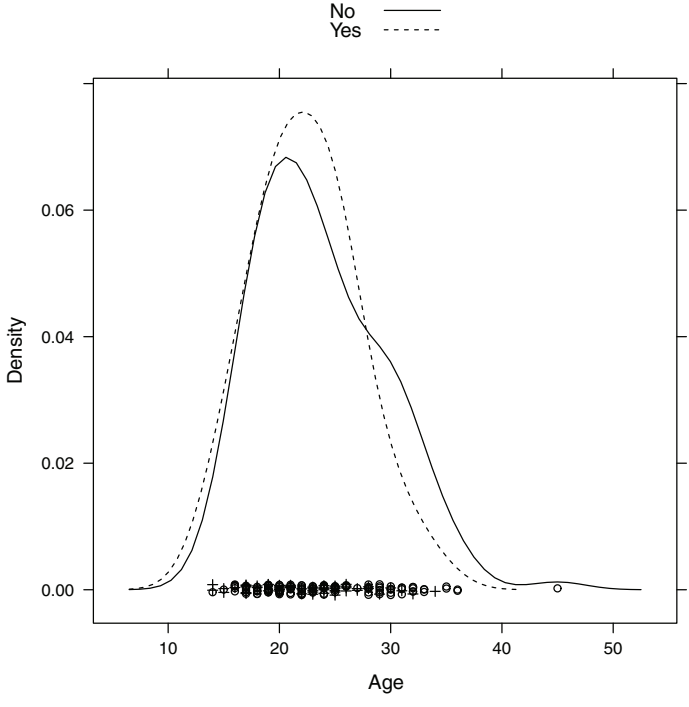


Figure 3.4. *Courbes de densité pour la distribution de l'âge des mères conditionnellement au statut pondéral à la naissance*

Avec la commande `xtabs()`, on écrirait `xtabs(~ low + smoke, data= birthwt)`. Les effectifs marginaux s'obtiennent avec la commande `margin.table()` en précisant l'option `margin=` (1, totaux lignes ; 2, totaux colonnes) :

```
> margin.table(xtabs(~ low + smoke, data=birthwt), margin=1)
low
 No Yes
130  59
```

On retrouve donc facilement la répartition des bébés dans les deux catégories de poids (`low`) avec $n = 29 + 30 = 59$ bébés ayant un poids inférieur à la norme dans cet échantillon.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

On peut toujours utiliser la commande `prop.table()` pour convertir le tableau d'effectifs en tableau de fréquences : par défaut, R remplacera chaque cellule du tableau de contingence par la valeur de l'effectif rapporté à l'effectif total (il s'agit donc de fréquences conditionnelles). L'option `margin=` peut être utilisée comme dans le cas de `margin.table()`, afin de calculer des proportions conditionnellement à l'une ou l'autre des deux variables du tableau de contingence :

```
> prop.table(xtabs(~ low + smoke, data=birthwt), margin=1)
     smoke
low          No       Yes
  No  0.6615385 0.3384615
  Yes 0.4915254 0.5084746
```

Cette commande permet donc de connaître la fréquence de mères qui fumaient pendant la grossesse parmi l'ensemble des bébés avec un poids inférieur à la norme ($44/(44+86) = 34$ %).

Comme dans le cas univarié, il est tout à fait possible de représenter graphiquement la distribution des effectifs pour deux variables qualitatives en utilisant soit un diagramme en barres (`barchart()`, soit un diagramme en points (ou de Cleveland, `dotplot()`). Dans les deux cas, ces commandes attendent un tableau d'effectifs et non une liste de variables reliées par une notation par formule (sauf dans le cas où l'on transforme le tableau en un data frame à trois colonnes dans lequel on fait apparaître explicitement les niveaux de chaque variable et les effectifs associés à leur croisement, par exemple `as.data.frame(xtabs(~ low+smoke, data=birthwt))`). Voici un exemple avec les variables `low` et `smoke` pour un diagramme en barres (figure 3.5) :

```
> barchart(xtabs(~ low + smoke, data=birthwt), stack=FALSE,
           auto.key=TRUE, ylab="low")
```

L'option `stack=FALSE` permet d'afficher des barres séparées pour les modalités de la variable `low`, rappelées en légende à l'aide de `auto.key=TRUE`. L'orientation des barres se contrôle avec l'option `horizontal=` (par défaut, elle vaut `TRUE`).

3.2. Comparaisons de deux moyennes de groupe

Les procédures de test « simples » sont généralement accessibles à partir de commandes R incluant le mot-clé `test` : `t.test()` (test de Student), `chisq.test()` (test du χ^2 de Pearson), etc. Dans certains cas, elles reposent sur une notation par formule R selon laquelle on désigne une variable réponse « décrite » par une variable explicative ; par exemple, `y ~ x` signifiera généralement la réponse `y` décrite par la variable `x` (de type `factor`).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

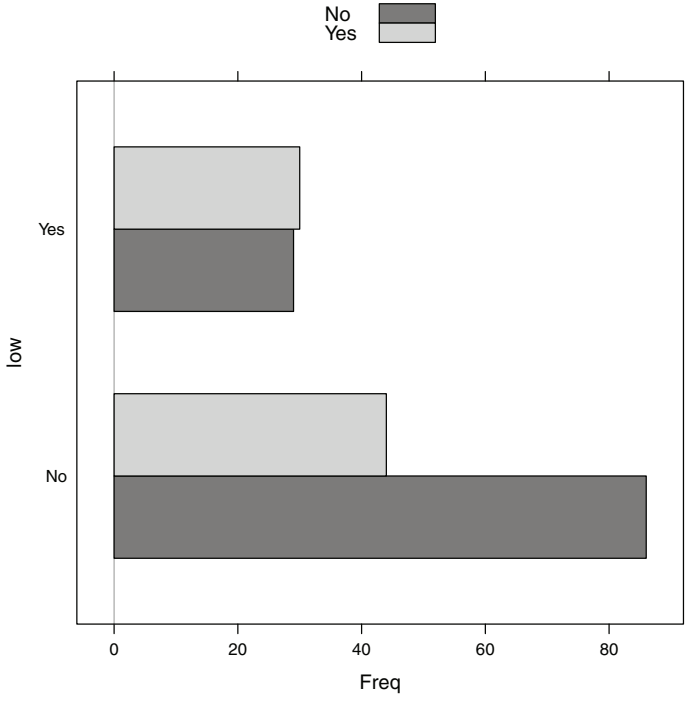


Figure 3.5. *Diagramme en barres pour un tableau de contingence*

3.2.1. *Echantillons indépendants*

Dans le cas de deux échantillons indépendants, le test de Student permettant de tester l'égalité des moyennes de groupe s'obtient avec la commande `t.test()`. Pour comparer le poids moyen des mères ayant présenté un problème d'irritabilité de l'utérus (variable `ui`) et celui des mères n'ayant aucun problème, on procèdera de la manière suivante :

```
> t.test(lwt ~ ui, data=birthwt, var.equal=TRUE)

Two Sample t-test

data:  lwt by ui
t = 2.1138, df = 187, p-value = 0.03586
alt hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
  0.8753389 25.3544748
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
sample estimates:
 mean in group No mean in group Yes
         131.7578          118.6429
```

La notation par formule permet d'indiquer à R que l'on s'intéresse à la relation entre les deux variables `lwt` et `ui`, où les variations de `lwt` sont « décrites » par `ui`. En d'autres termes, `lwt` est considérée comme une variable réponse dont on cherche à expliquer les variations (moyennes) en fonction d'une variable de classification `ui`. L'option `var.equal=TRUE` (par défaut, elle vaut `FALSE`) indique à R de ne pas appliquer une correction de type Satterwaithe (test dit de Welch) lors du calcul. R affiche le résultat du test de l'hypothèse nulle en donnant la valeur de la statistique de test (`t`), le nombre de degrés de liberté (`df`) et le degré de signification associé (`p-value`). Il fournit également un intervalle de confiance pour le paramètre d'intérêt, ici la différence de moyennes.

On peut naturellement vérifier si les conditions d'application de ce test, en particulier l'égalité des variances parentes, sont vérifiées en comparant les valeurs des variances estimées à partir de l'échantillon :

```
> with(birthwt, tapply(lwt, ui, var))
      No      Yes
940.8472 783.7196
```

Une alternative consiste à vérifier visuellement la distribution de `lwt` dans les deux groupes définis par la variable `ui`, par exemple en utilisant un diagramme de type boîte à moustaches (figure 3.6) :

```
> bwplot(lwt ~ ui, data = birthwt, pch = "|")
```

Un test plus formel pour la comparaison des deux variances est disponible *via* la commande `var.test()`. On peut également comparer les résultats précédents avec ceux que l'on obtiendrait en corrigeant les degrés de liberté de la distribution t de référence pour tenir compte de l'hétérogénéité des variances (les degrés de liberté ne sont alors généralement plus des entiers). Il suffit pour cela d'enlever l'option `var.equal=TRUE` lors de l'utilisation de `t.test()` :

```
> t.test(lwt ~ ui, data=birthwt)

Welch Two Sample t-test

data:  lwt by ui
t = 2.2547, df = 39.163, p-value = 0.02982
alt hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
  1.351128 24.878685
sample estimates:
 mean in group No mean in group Yes
        131.7578          118.6429
```

Les tests réalisés précédemment considèrent une hypothèse alternative non orientée (différence des deux moyennes non nulle, mais sans préciser le sens de la différence) et sont donc bilatéraux. Pour spécifier *a priori* le sens de la différence attendue sous l'hypothèse alternative (par exemple, $H_1 : \mu_1 > \mu_2$), il faut renseigner l'option `alternative=` dans la commande `t.test()`.

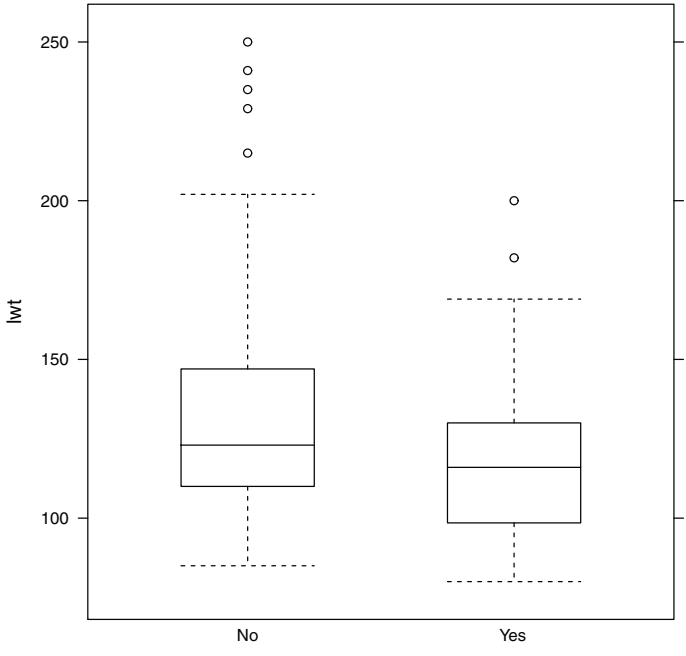


Figure 3.6. *Distribution de la variable lwt selon les niveaux de la variable ui*

3.2.2. *Echantillons non indépendants*

Dans le cas où les échantillons ne sont pas indépendants, la commande R à utiliser reste `t.test()` mais il faut ajouter l'option `paired=TRUE` pour indiquer à R que les deux échantillons ne sont pas indépendants (cas où les séries de mesure ont été recueillies sur les mêmes unités statistiques à deux reprises, par exemple).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

On profitera de cette section pour discuter l'organisation possible des données dans un tableau lorsque l'on travaille avec deux séries de mesures appariées. Considérons deux séries de scores sur une échelle de dépression (échelle de Hamilton) obtenues sur neuf patients avant (x1) et après traitement (x2) par tranquilisants [HOL 73] :

```
> x1 <- c(1.83,0.50,1.62,2.48,1.68,1.88,1.55,3.06,1.30)
> x2 <- c(0.878,0.647,0.598,2.05,1.06,1.29,1.06,3.14,1.29)
```

Une façon naturelle d'organiser ces données consisterait à arranger ces deux séries de mesure dans un tableau à neuf lignes et deux colonnes, les colonnes représentant les deux variables précédentes :

```
> d <- data.frame(x1, x2)
> dim(d)
[1] 9 2
> summary(d)
       x1              x2
 Min.   :0.500   Min.   :0.598
 1st Qu.:1.550   1st Qu.:0.878
 Median :1.680   Median :1.060
 Mean   :1.767   Mean   :1.335
 3rd Qu.:1.880   3rd Qu.:1.290
 Max.   :3.060   Max.   :3.140
```

Un test t pour comparer les moyennes entre les deux périodes peut alors se formuler de la manière suivante :

```
> with(d, t.test(x1, x2, paired=TRUE))

Paired t-test

data:  x1 and x2
t = 3.0354, df = 8, p-value = 0.01618
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 0.1037787 0.7599991
sample estimates:
mean of the differences
              0.4318889
```

On pourrait tout aussi bien adopter une autre organisation des données : dans une colonne on a l'ensemble des scores de dépression, et dans l'autre la période de mesure

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

(avant ou après traitement). Pour transformer le tableau `d` (deux colonnes, `x1` et `x2`) en un tableau contenant ces deux nouvelles variables, on peut utiliser la commande `melt()` du package `reshape2`. Ce package n'étant pas fourni avec l'installation de base de R, il sera nécessaire de l'installer *via* la commande `install.packages()` dans R. On utilise ensuite la commande `library()` pour « charger » le package dans la session R et pouvoir accéder aux commandes qu'il fournit :

```
> library(reshape2)
> dm <- melt(d)
No id variables; using all as measure variables
> dim(dm)
[1] 18  2
> summary(dm)
 variable     value
 x1:9     Min.   :0.500
 x2:9     1st Qu.:1.060
          Median :1.425
          Mean   :1.551
          3rd Qu.:1.867
          Max.   :3.140
```

On peut vérifier que l'on retrouve bien les mêmes moyennes dans chaque groupe qu'avec la commande `summary(d)` utilisée plus haut :

```
> with(dm, tapply(value, variable, mean))
      x1       x2
1.766667 1.334778
```

Le test t s'écrirait à présent `t.test(value~variable, data=dm, paired=TRUE)`.

Notons que l'on pourrait également rendre plus explicite le fait que ce sont les mêmes sujets qui sont mesurés dans les deux conditions expérimentales (`x1` puis `x2`) en ajoutant une variable codant pour l'identifiant du patient (n°) :

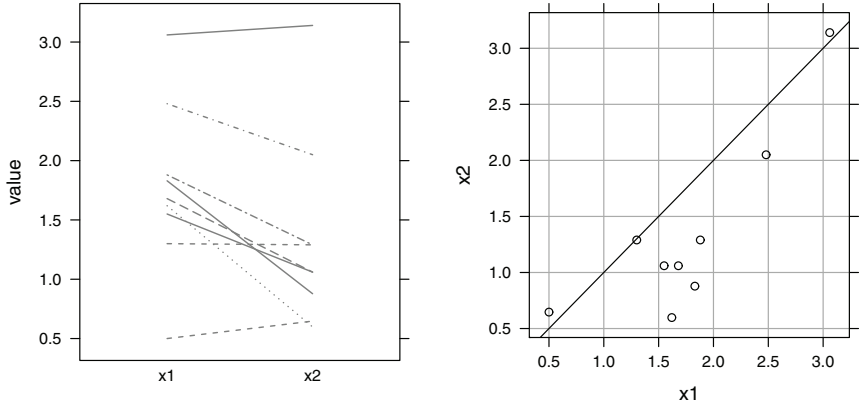
```
> dm$id <- factor(rep(1:9, 2))
> head(dm, n=3)
  variable value id
1       x1  1.83  1
2       x1  0.50  2
3       x1  1.62  3
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Pour représenter graphiquement les données individuelles tout en tenant compte de l'appariement, deux solutions graphiques sont proposées ci-après :

```
> library(gridExtra)
> p1 <- stripplot(value ~ variable, data=dm, groups=id, type="l",
                  col="grey50", lwd=1.2)
> p2 <- xyplot(x2 ~ x1, data=d, type=c("p", "g"),
               abline=list(a=0, b=1))
> grid.arrange(p1, p2, nrow=1)
```

Le package `gridExtra` (à installer) fournit la commande `grid.arrange()` qui permet d'assembler différents graphiques sur une même fenêtre graphique. En examinant la figure 3.7b, on peut vérifier que la majorité des points sont situés en dessous de la diagonale, représentée par la droite de pente unité, ce qui suggère qu'en général les patients ont un score plus bas après traitement (`x2`). Dans la figure 3.7a, cela se traduit par des pentes individuelles négatives dans la plupart des cas.



a) b)

Figure 3.7. *a) Profil d'évolution des scores individuels.*
b) Diagramme de dispersion des scores

3.3. Comparaisons de proportions

3.3.1. *Cas de deux proportions*

Supposons que l'on s'intéresse à la probabilité d'observer des enfants en sous-poids à la naissance (`low`) selon que leur mère fumait ou non durant leur grossesse (`smoke`). L'hypothèse nulle est que ces deux probabilités sont égales. La commande

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

`prop.test()` permet de tester cette hypothèse en utilisant comme distribution asymptotique la loi normale :

```
> tab <- with(birthwt, table(smoke, low))
> tab
     low
smoke No Yes
  No  86  29
  Yes 44  30
> prop.table(tab, margin=1)
     low
smoke        No       Yes
  No  0.7478261 0.2521739
  Yes 0.5945946 0.4054054
> prop.test(tab[,c(2,1)], correct=FALSE)

2-sample test for equality of proportions without continuity
correction

data:  tab[, c(2, 1)]
X-squared = 4.9237, df = 1, p-value = 0.02649
alternative hypothesis: two.sided
95 percent confidence interval:
 -0.29039121 -0.01607177
sample estimates:
   prop 1    prop 2
0.2521739 0.4054054
```

On prendra garde au fait que le tableau doit être légèrement réorganisé pour que l'événement d'intérêt (`low="Yes"`) figure bien dans la première colonne, d'où la permutation des deux colonnes du tableau avec `tab[,c(2,1)]` et les proportions recherchées 30/74 (fumeur) et 29/115 (non-fumeur). Cela ne change rien au résultat du test d'hypothèse, mais cela influence évidemment les proportions estimées et l'intervalle de confiance associé à la différence entre ces deux proportions.

Une formulation alternative consiste à indiquer le nombre de « succès » (ici, le fait que le bébé soit en sous-poids) et l'effectif total, pour chaque classe définie par la variable `smoke` :

```
> prop.test(c(29,30), c(86+29, 44+30), correct=FALSE)

2-sample test for equality of proportions without continuity
correction
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
data:  c(29, 30) out of c(86 + 29, 44 + 30)
X-squared = 4.9237, df = 1, p-value = 0.02649
alternative hypothesis: two.sided
95 percent confidence interval:
 -0.29039121 -0.01607177
sample estimates:
   prop 1    prop 2
0.2521739 0.4054054
```

On remarquera que la commande `prop.test()` inclut une option `correct=` qui permet de décider si une correction de continuité (correction de Yates) doit être appliquée ou non.

3.3.2. *Test du chi-deux*

Un autre test fréquemment utilisé lorsque l'on travaille avec des tableaux de contingence est le test du χ^2 de Pearson qui vise à tester l'association entre deux variables qualitatives, l'écart à l'indépendance étant quantifié par la différence entre les effectifs théoriques attendus sous l'hypothèse d'indépendance et les effectifs observés à partir de l'échantillon. Voici une application en considérant les mêmes variables `low` et `smoke` :

```
> chisq.test(xtabs(~ low + smoke, data=birthwt))

Pearson's Chi-squared test with Yates' continuity correction

data:  xtabs(~low + smoke, data = birthwt)
X-squared = 4.2359, df = 1, p-value = 0.03958
```

Notons que l'instruction précédente est équivalente à :

```
> with(birthwt, chisq.test(table(low, smoke)))
> summary(xtabs(~ low + smoke, data=birthwt)
```

Comme dans le cas de `prop.test()`, l'option `correct=` peut être utilisée dans le cas d'un tableau 2×2 pour réaliser une correction de continuité.

En stockant le résultat précédent dans une variable auxiliaire, il est même possible de vérifier si les conditions d'application de ce test sont vérifiées, c'est-à-dire des effectifs théoriques suffisamment larges (généralement > 5) :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> res <- chisq.test(xtabs(~ low + smoke, data=birthwt))
> res$expected
     smoke
low         No      Yes
  No  79.10053 50.89947
  Yes 35.89947 23.10053
```

Comme on peut le constater, il suffit d'ajouter `$expected` à la suite de la variable dans laquelle le résultat du test du χ^2 a été stocké. En fait, les tests statistiques réalisés par R renvoient de nombreuses valeurs (statistique de test, degrés de liberté, etc.) et R affiche automatiquement les informations utiles pour prendre une décision concernant le résultat du test. Cependant, d'autres informations restent masquées mais peuvent être retrouvées de la même manière que celle illustrée précédemment. Dans le cas du χ^2, la commande `str(res)` permet d'afficher l'ensemble des informations qui sont disponibles (effectifs observés, théoriques et résidus entre autres).

3.3.3. *Cas de deux échantillons non indépendants*

Dans le cas où le tableau de contingence est formé à partir du croisement de deux variables observées sur le même échantillon (par exemple, sévérité de la symptomatologie à deux classes en pré et post-traitement), on peut recourir au test de McNemar à l'aide de la commande `mcnemar.test()`. Cette commande accepte également les données présentées sous la forme d'un tableau généré avec `table()` ou `xtabs()`. L'aide en ligne fournit un exemple d'utilisation de cette commande, voir `help(mcnemar.test)`.

3.4. Mesures de risque et odds-ratio

Les mesures de risque usuellement calculées à partir d'un tableau de contingence croisant un facteur d'exposition (ou n'importe quelle variable binaire d'intérêt) et une variable réponse binaire (malade/pas malade, échec/réussite) sont le risque relatif et l'odds-ratio (rapport de cotes). Ce dernier peut être utilisé plus généralement pour quantifier le degré d'association entre deux variables binaires.

Les données sont généralement arrangées dans un tableau de contingence 2×2 où les effectifs associés aux « événements positifs » figurent sur la première ligne et la première colonne. Les variables `smoke` et `low` utilisées dans les illustrations précédentes ont deux niveaux, 0 et 1, et par défaut ce sont les niveaux de référence 0 (codant pour les événements négatifs, non-fumeur ou poids normal) qui apparaîtront sur la première ligne et la première colonne lorsque l'on utilise `table()` ou `xtabs()`. Cela reste valable en utilisant les étiquettes `No` (0) et `Yes` (1). En pratique, cela ne pose pas de problème pour le calcul de l'odds-ratio, mais pour le risque relatif il faut faire

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

attention. On réarrangera donc le tableau de contingence en permutant les deux lignes et les deux colonnes :

```
> tab <- xtabs(~ smoke + low, data=birthwt)
> tab <- tab[c(2,1),c(2,1)]
> tab
     low
smoke Yes No
  Yes  30 44
  No   29 86
```

On voit donc que la première ligne du tableau `tab` contient la répartition des enfants ayant un poids inférieur ou supérieur à 2,5 kg pour les mères fumeuses, et que la première colonne contient bien la répartition des mères fumeuses ou non-fumeuses pour les enfants en sous-poids à la naissance. L'odds-ratio est simplement le rapport des odds (cotes) 30/44 et 29/86, soit $\frac{30/44}{29/86} = \frac{30\times 86}{44\times 29} = 2,02$. Quant au risque relatif, il représente le rapport des risques de chaque groupe d'exposition. Ici, on considère que le fait que la mère ait fumé durant sa grossesse représente le facteur d'exposition, d'où un risque de $30/(30+44)$ chez les fumeuses comparé à $29/(29+86)$ chez les non-fumeuses, soit un risque relatif de $\frac{30/(30+44)}{29/(29+86)} = 1,61$.

Il existe de nombreuses commandes permettant de retrouver ces calculs manuels et de fournir des intervalles de confiance pour les valeurs estimées de l'odds-ratio ou du risque relatif. Pour le calcul de l'odds-ratio, on utilisera un package externe, `vcd`, qu'il sera nécessaire d'installer au préalable. Pour cela, on tapera simplement `install.packages("vcd")` dans R, et après avoir vérifié que l'installation s'est correctement déroulée, on chargera le package à l'aide de la commande `library()`, comme on l'a déjà vu pour le package `lattice` ou `reshape2` :

```
> library(vcd)
> oddsratio(tab, log=FALSE)
[1] 2.021944
```

L'option `log=FALSE` est indispensable si l'on souhaite obtenir la valeur de l'odds-ratio et non le logarithme de sa valeur. La commande `confint()` permet ensuite d'obtenir l'intervalle de confiance associé, par défaut à 95 % : `confint (oddsratio(tab, log=FALSE))`.

Une autre possibilité est d'utiliser le package `epiR`, également à installer et charger avant de pouvoir utiliser les commandes fournies dans ce package. L'avantage de ce package et de la commande `epi.2by2()` est qu'il fournit une estimation du risque relatif et de l'odds-ratio, en considérant par défaut une étude de cohorte pour laquelle ces deux indicateurs ont un sens :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> library(epiR)
> epi.2by2(tab)
            Outcome +   Outcome -   Total   Inc risk *     Odds
Exposed +          30          44      74         40.5    0.682
Exposed -          29          86     115         25.2    0.337
Total              59         130     189         31.2    0.454

Point estimates and 95 % CIs:
---------------------------------------------------------------
Inc risk ratio                              1.61 (1.06, 2.44)
Odds ratio                                  2.01 (1.03, 3.96)
Attrib risk *                               15.32 (1.61, 29.04)
Attrib risk in population *                 6.00 (-4.33, 16.33)
Attrib fraction in exposed (%)              37.80 (5.47, 59.07)
Attrib fraction in population (%)           19.22 (-0.21, 34.88)
---------------------------------------------------------------
 * Cases per 100 population units
```

3.5. Approches non paramétriques et tests exacts

Pour la comparaison de mesures numériques recueillies sur deux échantillons indépendants, une approche non paramétrique reposant uniquement sur les rangs des observations consiste à utiliser le test de Wilcoxon-Mann-Whitney, disponible dans la commande `wilcox.test()`. Comme dans le cas du test de Student, lorsque les échantillons ne sont pas indépendants, on ajoutera l'option `paired=TRUE`. Avec les deux exemples discutés précédemment (section 3.2), on procèderait de la manière suivante :

```
> wilcox.test(lwt ~ ui, data=birthwt)

Wilcoxon rank sum test with continuity correction

data:  lwt by ui
W = 2896, p-value = 0.01626
alternative hypothesis: true location shift is not equal to 0

> with(d, wilcox.test(x1, x2, paired=TRUE))

Wilcoxon signed rank test

data:  x1 and x2
V = 40, p-value = 0.03906
alternative hypothesis: true location shift is not equal to 0
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Le second test pour le cas des échantillons appariés est appelé test signé des rangs. Avec cet exemple utilisant les variables `x1` et `x2` dans le data frame `d`, une formulation équivalente en utilisant le tableau faisant apparaître distinctement variable explicative et variable réponse (`dm`) serait alors :

```
> wilcox.test(value ~ variable, data=dm, paired=TRUE)
```

En ce qui concerne les tableaux de contingence, une alternative au test du χ^2 est le test de Fisher disponible *via* la commande `fisher.test()` :

```
> fisher.test(xtabs(~ low + smoke, data=birthwt))

Fisher's Exact Test for Count Data

data:  xtabs(~low + smoke, data = birthwt)
p-value = 0.03618
alternative hypothesis: true odds ratio is not equal to 1
95 percent confidence interval:
 1.028780 3.964904
sample estimates:
odds ratio
  2.014137
```

On notera que cette commande renvoie également l'odds-ratio et son intervalle de confiance à 95 %.

En dernier lieu, R dispose de la commande `binom.test()` pour réaliser un test binomial. Celui-ci permet de tester une probabilité de succès dans le cas d'une variable binaire en considérant une distribution binomiale et non une distribution normale comme dans le cas de `prop.test()`.

3.6. Ce qu'il faut retenir

– Avant de réaliser un test, on calcule toujours quelques statistiques descriptives pour chaque niveau d'une variable explicative (`tapply()`) ou en croisant les modalités de deux variables qualitatives (`table()` ou `xtabs()`).

– Les représentations graphiques pour deux variables ne diffèrent pas vraiment de celles vues dans le cas univarié, à ceci près que l'on utilise une notation par formule pour indiquer les relations entre variables. Dans le cas des variables qualitatives, on fournit généralement un tableau d'effectifs (`barchart()` ou `dotplot()`).

– Les procédures de test renvoient généralement le résultat du test de l'hypothèse nulle associée au test et un intervalle de confiance pour le paramètre estimé.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

– Dans le cas des mesures de risque (odds-ratio, mais surtout risque relatif), on prendra garde à l'arrangement des effectifs (en lignes et en colonnes) dans un tableau de contingence.

3.7. Pour aller plus loin

Le package `psych` (www.personality-project.org/r/psych/) fournit quelques commandes spécifiques pour le calcul et la représentation graphique de moyennes conditionnelles (`describeBy()` et `error.bars.by()`). Les packages `plyr` (http://plyr.had.co.nz) et `dplyr` offrent également la possibilité de calculer des statistiques sur des données aggrégées de manière relativement efficaces. Des fonctionnalités identiques sont disponibles dans le package `Hmisc`, décrit en annexe.

Outre les commandes `power.t.test()` et `power.prop.test()`, les packages `pwr` et `clinfun` disposent de commandes spécifiques au calcul du nombre de sujets nécessaires pour mettre en évidence une différence de moyennes ou de proportions définie *a priori* pour un risque alpha et une puissance donnés.

3.8. Applications

1) La qualité de sommeil de dix patients a été mesurée avant (contrôle) et après traitement par un des deux hypnotiques suivants : (1) D. hyoscyamine hydrobromide et (2) L. hyoscyamine hydrobromide. Le critère de jugement retenu par les chercheurs était le gain moyen de sommeil (en heures) par rapport à la durée de sommeil de base (contrôle) [STU 08]. Les données figurent également parmi les jeux de données de base de R (`data(sleep)`).

Les chercheurs ont conclu que seule la deuxième molécule avait réellement un effet soporifique.

a) Estimer le temps moyen de sommeil pour chacune des deux molécules, ainsi que la différence entre ces deux moyennes.

b) Afficher la distribution des scores de différence (LHH - DHH) sous forme d'un histogramme, en considérant des intervalles de classe d'une demi-heure, et indiquer la moyenne et l'écart-type de ces scores de différence.

c) Vérifier l'exactitude des conclusions à l'aide d'un test de Student.

On remarquera qu'il s'agit des mêmes patients qui sont testés dans les deux conditions (le sujet est pris comme son propre témoin). L'aide en ligne pour le jeu de données `sleep` indique bien que les deux premières variables, `extra` (différentiel de temps de sommeil) et `group` (type de médicament), sont les variables d'intérêt :

```
> data(sleep)
> mean(sleep$extra[sleep$group == 1]) # D. hyoscyamine hydro.
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> mean(sleep$extra[sleep$group == 2]) # L. hyoscyamine hydro.
> m <- with(sleep, tapply(extra, group, mean))
> m[2] - m[1]
```

Les commandes précédentes permettent de vérifier les moyennes de groupe et la différence de gain de sommeil entre les molécules L. hyoscyamine hydrobromide et D. hyoscyamine hydrobromide, en gardant à l'esprit que la commande `with(sleep, tapply(extra, group, mean))` renvoie deux valeurs (les moyennes par type de molécule) et que la différence entre ces deux valeurs (stockées dans la variable auxiliaire `m`) renvoie bien la différence de moyennes entre les deux traitements.

Pour calculer les scores de différence, on peut procéder de la manière suivante :

```
> sdif <- sleep$extra[sleep$group == 2] -
          sleep$extra[sleep$group == 1]
> c(mean=mean(sdif), sd=sd(sdif))
```

Le calcul de la moyenne et de l'écart-type de ces scores de différence ne pose pas de difficulté particulière, pas plus que l'affichage de leur distribution sous forme d'histogramme :

```
> histogram(~ sdif, breaks=seq(0, 5, by=0.5), xlab="LHH - DHH")
```

L'option `breaks=seq(0, 5, by=0.5)` permet de construire une séquence de nombres compris entre 0 et 5 et séparés de 0,5 unité pour le découpage des classes d'effectifs.

Le résultat du test t est obtenu à partir de la commande `t.test()` en fournissant la variable réponse et le facteur de classification sous forme d'une formule (variable réponse à gauche, variable explicative à droite) :

```
> t.test(extra ~ group, data=sleep, paired=TRUE)
```

On notera qu'il est nécessaire d'indiquer à R où trouver les variables d'intérêt, d'où l'usage de `data=sleep`. L'option `paired=TRUE` est nécessaire pour rendre compte de l'appariement des observations. Notons également que, par défaut, R présente la différence de moyennes entre le premier et le second traitement, et non le second moins le premier comme calculé précédemment.

Le résultat significatif du test et le sens de la différence de moyennes observée (au niveau du gain horaire de sommeil) est bien en accord avec les conclusions des chercheurs. On peut visualiser les résultats sous forme d'un diagramme en barres (on pourrait également utiliser un diagramme en points à l'aide de la commande `dotplot()`) :

```
> barchart(with(sleep, tapply(extra, group, mean)))
```

2) Dans un essai clinique, on a cherché à évaluer un régime supposé réduire le nombre de symptômes associé à une maladie bénigne du sein. Un groupe de 229

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

femmes ayant cette maladie ont été alétoirement réparties en deux groupes. Le premier groupe a reçu les soins courants, tandis que les patientes du second groupe suivaient un régime spécial (variable B = traitement). Après un an, les individus ont été évalués et ont été classés dans l'une des deux catégories : amélioration ou pas d'amélioration (variable A = réponse) [SEL 98]. Les résultats sont résumés dans le tableau 3.1, pour une partie de l'échantillon.

a) Réaliser un test du chi-deux.

b) Quels sont les effectifs théoriques attendus sous une hypothèse d'indépendance ?

c) Comparer les résultats obtenus en (a) avec ceux d'un test de Fisher.

d) Donner un intervalle de confiance pour la différence de proportion d'amélioration entre les deux groupes de patientes.

	Régime	Pas de régime	Total
Amélioration	26	21	47
Pas d'amélioration	38	44	82
Total	64	65	129

Tableau 3.1. *Régime et maladie du sein*

On ne dispose que des données concernant les effectifs tels que reportés dans le tableau, mais il n'est pas nécessaire d'avoir les données brutes pour réaliser le test du χ^2. Celui-ci est obtenu à partir de la commande `chisq.test()` et inclut par défaut une correction de continuité (Yates). Pour construire le tableau de contingence, on peut utiliser la commande `matrix()` :

```
> regime <- matrix(c(26,38,21,44), nrow=2)
> dimnames(regime)<-list(c("amélioration","pas d'amélioration"),
                         c("régime","pas de régime"))
```

Par défaut, les effectifs sont saisis par colonne, sauf si l'on précise l'option `byrow=TRUE`. Dans le cas d'un tableau 2x2, il n'est pas nécessaire de spécifier le nombre de lignes lorsque l'on a indiqué le nombre de colonnes (ou vice-versa). Le test du χ^2 s'obtient à l'aide de `chisq.test()` :

```
> chisq.test(regime)
```

Si l'on ne souhaite pas appliquer de correction de continuité, il faut ajouter l'option `correct=FALSE`.

Les effectifs théoriques ne sont pas affichés avec le résultat du test, mais on peut les obtenir comme suit :

```
> chisq.test(regime)$expected
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Concernant le test de Fisher, on procède de la même manière en utilisant la commande `fisher.test()` à partir du tableau de contingence :

```
> fisher.test(regime)
```

Les proportions d'intérêt sont $26/64 = 0,41$ (régime) et $21/65 = 0,32$ (pas de régime), d'où le test de proportion qui donne l'intervalle de confiance à 95 % pour la différence recherchée :

```
> prop.test(c(26,21), c(64,65))
```

3) Une étude a porté sur 86 enfants suivis dans un centre pour apprendre à gérer leur maladie. Lors de leur arrivée, on a demandé aux enfants s'ils savaient gérer leur maladie dans les conditions optimales (non détaillées), c'est-à-dire s'ils savaient à quel moment il leur fallait recourir au traitement prescrit. La même question a été posée à ces enfants à la fin de leur suivi dans le centre. La variable mesurée est la réponse (affirmative ou non) à cette question à l'entrée et à la sortie de l'étude [PEA 05]. Les données, disponibles au format SPSS dans le fichier `health-camp.sav`, sont résumées dans le tableau 3.2.

			Gestion maladie (sortie)		
			Non	Oui	Total
Gestion maladie (entrée)	Non	Effectif	27	29	56
		Fréquence	31,4 %	33,7 %	65,1 %
	Oui	Effectif	6	24	30
		Fréquence	7,0 %	27,9 %	34,9 %
	Total	Effectif	33	53	86
		Fréquence	38,4 %	61,6 %	100,0 %

Tableau 3.2. *Gestion de la maladie*

On se demande si le fait d'avoir suivi le programme de formation proposé dans le centre a augmenté le nombre d'enfants ayant une bonne connaissance de leur maladie et de sa gestion quotidienne.

a) Reproduire le tableau d'effectifs et de fréquences relatives précédent à partir des données brutes.

b) Indiquer le résultat d'un test de McNemar.

c) Comparer le résultat du test réalisé en (b) mais sans continuité de correction avec le résultat obtenu à partir d'un test binomial.

Il n'est pas vraiment nécessaire d'importer le tableau de données brutes et l'on pourrait se contenter de travailler avec les effectifs présentés dans le tableau. Toutefois, pour charger le fichier SPSS, il est nécessaire d'utiliser une commande R spécifique et disponible dans le package `foreign` :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> library(foreign)
> hc <- read.spss("health-camp.sav", to.data.frame=TRUE)
```

L'option `to.data.frame=TRUE` est importante car c'est celle qui assure qu'à l'issue de l'importation les données seront bien stockées dans un tableau où les lignes figurent les observations et les colonnes les variables. Si l'on regarde comment les données sont représentées dans le tableau importé grâce à la commande `head()`, on constatera que l'information est un peu vague :

```
> head(hc)
```

Ceci s'explique par la perte des labels associés aux variables dans SPSS. On peut retrouver cette information en interrogeant la base de données R avec la commande `str()` :

```
> str(hc)
```

A partir de la ligne `attr(*, "variable.labels")`, on voit donc que les colonnes `BEFORE` et `AFTER` correspondent à la question portant sur la connaissance de la maladie, alors que les colonnes `BEFORE2` et `AFTER2` correspondent à la question portant sur le traitement.

Finalement, on peut reproduire le tableau initial (effectifs et fréquences relatives) ainsi :

```
> table(hc[,c("BEFORE","AFTER")])
> round(prop.table(table(hc[,c("BEFORE","AFTER")])), 2)
```

Pour obtenir les distributions marginales, on peut utiliser la commande `margin.table()` :

```
> margin.table(table(hc[,c("BEFORE","AFTER")]), 1)
> margin.table(table(hc[,c("BEFORE","AFTER")]), 2)
```

Pour réaliser le test de McNemar, on peut utiliser directement le tableau de contingence construit précédemment. Par souci de commodité, on stockera ce dernier dans une variable R appelé `hc.tab` :

```
> hc.tab <- table(hc[,c("BEFORE","AFTER")])
> mcnemar.test(hc.tab)
```

On peut comparer les résultats du test de McNemar sans appliquer la correction de continuité avec ceux d'un test binomial (test exact) :

```
> binom.test(6, 6+29)
> mcnemar.test(hc.tab, correct=FALSE)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 4

Analyse de variance et plans d'expérience

Dans ce chapitre, on aborde l'analyse de la variance et quelques notions élémentaires sur les plans d'expérience. Seuls les cas de l'ANOVA à un et deux facteurs de classification (effets fixes, modèle avec interaction) sont abordés. Pour l'ANOVA à un facteur, le test de tendance linéaire est approché par deux techniques : la régression linéaire et la méthode des contrastes. Le cas des comparaisons multiples entre traitements est également abordé, en se limitant à la méthode de Bonferroni pour protéger les tests de l'inflation du risque de première espèce. Il est important de procéder dans tous les cas à une description approfondie des données avant de réaliser le modèle d'ANOVA (techniques de résumé numérique, méthodes graphiques, etc.).

4.1. Représentation des données et statistiques descriptives

4.1.1. *Format de représentation des données*

Si l'on considère une variable numérique servant de variable réponse et une variable qualitative à k niveaux (modalités, ou traitements) utilisée comme facteur de classification, on peut imaginer au moins deux manières de représenter les données sous R : un tableau où les k séries d'observations sont représentées dans différentes colonnes (soit un tableau à k colonnes et n lignes dans le cas d'un plan équilibré où l'on a le même nombre d'observations par traitement) ou bien un tableau où toutes les mesures figurent dans une première colonne et les traitements associés dans une seconde colonne, comme discuté en section 3.2.2.

On prendra toujours bien garde au fait que le facteur de classification doit bien être traité comme un `factor` et pas une variable numérique.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

4.1.2. *Statistiques descriptives et structuration des données*

Dans l'exemple suivant, on génère une structure de données composée de vingt observations réparties de manière équilibrée sur les quatre niveaux (traitements) d'une variable qualitative. Les données sont disposées en quatre colonnes, chaque colonne correspondant à un traitement spécifique, et on a donc au final un tableau à cinq lignes et quatre colonnes. Les mesures sont générées artificiellement à partir de tirages indépendants (`replicate`) dans une loi normale de moyenne 12 et d'écart-type 1,2 :

```
> d <- data.frame(replicate(4, rnorm(5, mean=12, sd=1.2)))
> d
        X1        X2       X3       X4
1 12.67078  9.729374 12.47485 12.38534
2 12.52286 11.428080 10.16006 11.86140
3 12.77444 11.866122 12.42084 13.01769
4 10.20481 10.663182 10.90229 10.45179
5 13.55448 11.733637 10.49526 10.76963
> summary(d)
       X1              X2               X3              X4
 Min.   :10.20   Min.   : 9.729   Min.   :10.16   Min.   :10.45
 1st Qu.:12.52   1st Qu.:10.663   1st Qu.:10.50   1st Qu.:10.77
 Median :12.67   Median :11.428   Median :10.90   Median :11.86
 Mean   :12.35   Mean   :11.084   Mean   :11.29   Mean   :11.70
 3rd Qu.:12.77   3rd Qu.:11.734   3rd Qu.:12.42   3rd Qu.:12.39
 Max.   :13.55   Max.   :11.866   Max.   :12.47   Max.   :13.02
```

On souhaite calculer la moyenne des mesures pour chaque colonne. Dans le cas où le tableau de données est un data frame, le plus simple pour appliquer la même commande à chaque colonne du tableau consiste à utiliser la commande `sapply()` comme ceci :

```
> sapply(d, mean)
      X1       X2       X3       X4
12.34548 11.08408 11.29066 11.69717
```

Supposons à présent que ces mêmes données soient arrangées dans un tableau à deux colonnes, la première colonne indiquant le traitement et la seconde colonne la mesure recueillie. Par souci de simplicité, on ne fera pas apparaître le numéro d'observation dans une autre colonne. Le package `reshape2` (à charger au préalable) fournit la commande `melt()` que l'on a déjà utilisée et qui permet de transformer un tableau où les niveaux d'un facteur sont arrangés en colonnes en un tableau où ceux-là sont arrangés dans la même colonne :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> dm <- melt(d, value.name="y", variable.name="trt")
No id variables; using all as measure variables
> head(dm)
  trt         y
1  X1 12.670780
2  X1 12.522859
3  X1 12.774442
4  X1 10.204814
5  X1 13.554482
6  X2  9.729374
> summary(dm)
 trt          y
 X1:5   Min.   : 9.729
 X2:5   1st Qu.:10.621
 X3:5   Median :11.798
 X4:5   Mean   :11.604
        3rd Qu.:12.487
        Max.   :13.554
> with(dm, tapply(y, trt, mean))
      X1       X2       X3       X4
12.34548 11.08408 11.29066 11.69717
```

On voit qu'avec ce type de structure, on se retrouve dans les situations évoquées aux chapitres 2 et 3 et que l'on peut utiliser sans problème la commande `tapply()`. On cherchera en règle générale à se ramener à ce type d'organisation des données pour les modèles statistiques sous R.

Une commande similaire à `tapply()` mais qui permet d'utiliser une notation par formule est la commande `aggregate()` dont l'usage est relativement simple : on indique le type de relation à laquelle on s'intéresse entre deux (ou plusieurs) variables, le data frame où trouver les variables et la commande à appliquer, ici `mean()` :

```
> aggregate(y ~ trt, data=dm, mean)
  trt        y
1  X1 12.34548
2  X2 11.08408
3  X3 11.29066
4  X4 11.69717
```

Cette commande est strictement équivalente à l'instruction `tapply()` précédente, seul le format du résultat renvoyé est différent : la commande `aggregate()` renvoit ses résultats sous forme de data frame ce qui parfois peut être intéressant pour des analyses secondaires ou des représentations graphiques.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

4.2. ANOVA à un facteur

4.2.1. *Le modèle d'ANOVA à un facteur*

Prenons l'exemple des variations entre les poids des bébés (`bwt`) selon l'ethnicité de la mère (`race`) dans les données `birthwt`. La question que l'on peut se poser est la suivante : le poids moyen des bébés à la naissance diffère-t-il selon l'ethnicité de la mère ?

Le modèle d'ANOVA est construit à l'aide de la commande `aov()` sous R, en indiquant à l'aide d'une formule la relation reliant la variable réponse (à gauche du symbole ~) et la variable explicative (à droite du symbole ~) :

```
> aov(bwt ~ race, data=birthwt)
Call:
   aov(formula = bwt ~ race, data = birthwt)

Terms:
                    race Residuals
Sum of Squares   5015725  94953931
Deg. of Freedom        2       186

Residual standard error: 714.4963
Estimated effects may be unbalanced
```

On voit que le résultat renvoyé par R ne ressemble pas vraiment à un tableau d'analyse de variance et ne contient que des sommes de carrés (avec leurs degrés de liberté) ainsi qu'une estimation de la variance résiduelle (`sqrt(96179472/187)`). Pour afficher le tableau d'analyse de variance correspondant au modèle `bwt ~ race`, il faut utiliser la commande `summary()`. Il s'agit de la même commande que celle utilisée pour résumer un data frame ou une variable mais elle se comporte différemment dans le cas où l'objet R en question est un modèle d'ANOVA construit avec la commande `aov()` (plus précisément, la commande est en fait `summary.aov`) :

```
> summary(aov(bwt ~ race, data=birthwt))
             Df   Sum Sq Mean Sq F value  Pr(>F)
race          2  5015725 2507863   4.913 0.00834 **
Residuals   186 94953931  510505
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Notons qu'il sera souvent plus commode de stocker le résultat du modèle ANOVA dans une variable pour le réutiliser ensuite avec d'autres commandes. Les deux étapes précédentes peuvent donc être reformulées ainsi :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> res <- aov(bwt ~ race, data=birthwt)
> summary(res)
```

Une représentation graphique naturelle pour ce type de données (une variable numérique décrite par une variable qualitative) est un diagramme de type boîte à moustaches, pour avoir une idée des fluctuations à l'intérieur des groupes ou traitements, et un diagramme en barres ou en points pour avoir une idée de la variation des moyennes par traitement :

```
> bwplot(bwt ~ race, data=birthwt)
```

On remarquera que la notation par formule est identique à celle utilisée pour construire le modèle d'ANOVA. En ce qui concerne le diagramme en points, on doit introduire une petite variation :

```
> dotplot(race ~ bwt, data=birthwt, type="a")
```

L'option `type="a"` permet de calculer automatiquement les moyennes (*average*) de `bwt` par niveau de la variable `race`. Cette dernière figure ici à gauche du symbole ~ dans la formule R ce qui permet de la représenter sur l'axe des ordonnées plutôt que l'axe des abscisses. Le résultat produit par les commandes `bwplot()` et `dotplot()` sur ce jeu de données est illustré dans la figure 4.1. Dans le cas des boîtes à moustaches, les points à l'intérieur des boîtes représentent les médianes dans chaque groupe. Pour le diagramme en barres, les moyennes de groupe sont reliées par des segments afin de faciliter la lecture.

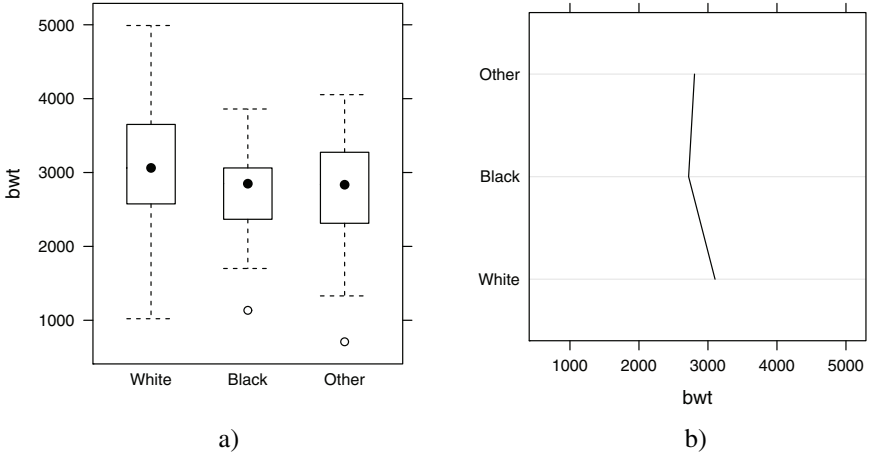


Figure 4.1. *a) Distribution des poids des bébés selon l'ethnicité de la mère à l'aide de boîtes à moustaches. b) Poids moyens des bébés en fonction de l'ethnicité*

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

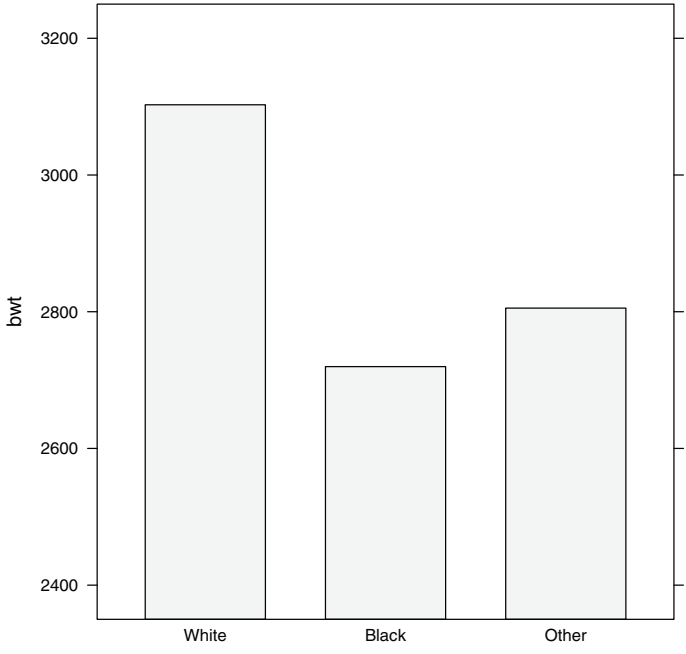


Figure 4.2. *Distribution des poids des bébés selon l'ethnicité de la mère à l'aide d'un diagramme en barres*

Une manière équivalente de résumer les données observées consiste à utiliser directement un tableau des moyennes de groupe, à l'aide de la commande `aggregate()` présentée dans la section précédente. Voici une illustration en remplaçant le diagramme en points (`dotplot()`) par un diagramme en barres à l'aide de `barchart()` (figure 4.2) :

```
> bwtmeans <- aggregate(bwt ~ race, data=birthwt, mean)
> bwtmeans
   race      bwt
1 White 3102.719
2 Black 2719.692
3 Other 2805.284
> barchart(bwt ~ race, data=bwtmeans, horizontal=FALSE,
           ylim=c(2350, 3250))
```

Il serait tout à fait possible d'ajouter les écarts-type ou intervalles de confiance associés à ces moyennes de groupe.

Pour vérifier les conditions d'application de l'ANOVA (normalité des résidus et homogénéité des variances), on peut utiliser des représentations graphiques ou des

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

tests formels. Le test de Shapiro-Wilk (normalité d'une distribution, cas des grands échantillons) est accessible *via* la commande `shapiro.test()`, mais on lui préfèrera généralement les méthodes graphiques (QQ plot ou boîtes à moustaches). Pour l'égalité des variances, on dispose des tests de Levene (commande `leveneTest()` dans le package `car`) ou de Bartlett (`bartlett.test()`). Ces commandes utilisent la même notation par formule :

```
> bartlett.test(bwt ~ race, data=birthwt)

Bartlett test of homogeneity of variances

data:  bwt by race
Bartlett's K-squared = 0.65952, df = 2, p-value = 0.7191
```

Enfin, les commandes `fitted()` (ou `predict`) et `resid()` permettent d'obtenir les valeurs prédites par le modèle et les résidus (écarts entre valeurs observées et valeurs prédites), respectivement. On vérifiera sans mal que dans ce cas les valeurs prédites correspondent bien aux moyennes des traitements :

```
> unique(predict(res))
[1] 2719.692 2805.284 3102.719
```

4.2.2. *Comparaisons par paires de traitement*

Un résultat significatif pour le test F de Fisher-Snedecor dans une ANOVA permet de conclure qu'au moins une paire de moyennes peut être considérée comme statistiquement différente au seuil d'erreur fixé à l'avance (5 % en règle générale). Plusieurs techniques dites de « comparaisons multiples » ont été proposées dans la littérature. On se contentera de présenter la méthode de Bonferroni pour la comparaison de l'ensemble des paires de moyennes à l'aide de tests t de Student.

Une approche assez conservatrice consiste à appliquer une correction de type Bonferroni pour chacun des tests réalisés : cela revient à multiplier chaque degré de signification par le nombre total de tests réalisés. La commande `pairwise.t.test()` permet de tester toutes les paires de moyennes à l'aide de tests de Student, et elle renvoie les degrés de signification corrigés pour chaque paire de traitement :

```
> with(birthwt, pairwise.t.test(bwt, race,
                                p.adjust.method="bonferroni"))

Pairwise comparisons using t tests with pooled SD

data:  bwt and race
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
      White Black
Black 0.049 -
Other 0.029 1.000

P value adjustment method: bonferroni
```

On peut vérifier que l'on obtient bien les mêmes résultats à l'aide de simples tests de Student, en utilisant `t.test()` et en excluant systématiquement l'un des niveaux de la variable `race`.

Voici un exemple avec les comparaisons `White/Black` et `White/Other` :

```
> t1 <- t.test(bwt ~ race, data=birthwt, subset=race != "Other")
> t2 <- t.test(bwt ~ race, data=birthwt, subset=race != "Black")
> c(t1$p.value, t2$p.value) * 3
[1] 0.03509656 0.03277981
> with(birthwt, pairwise.t.test(bwt, race, p.adj="bonf",
                                pool.sd=FALSE))

Pairwise comparisons using t tests with non-pooled SD

data:  bwt and race
      White Black
Black 0.035 -
Other 0.033 1.000

P value adjustment method: bonferroni
```

Pour que la comparaison entre les deux approches soit valide, il est nécessaire de demander à R de ne pas utiliser une estimation de la variance commune à l'ensemble des traitements (comme dans l'ANOVA) mais bien celle des seuls traitements pris en compte dans la comparaison, d'où l'option `pool.sd=FALSE`. Enfin, on retiendra que, lorsque les options ne prêtent pas à confusion, elles peuvent être abbrégées ; dans le cas ci-dessus, on peut très bien écrire `p.adj="bonf"` au lieu de `p.adjust.method = "bonferroni"`.

4.2.3. *Test de tendance linéaire*

Dans les cas où cela fait sens de considérer que les niveaux du facteur de classification sont ordonnés, il peut être intéressant de formuler l'ANOVA sous la forme d'un test de tendance linéaire. On supposera généralement que les niveaux du facteur peuvent être considérés comme équi-espacés.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Considérons par exemple la variable `ftv`, qui représente le nombre de visites chez le gynécologue durant le premier trimestre de grossesse. Cette variable prend des valeurs entre 0 et 6, les valeurs supérieures à 2 étant assez rarement observées (douze cas au total, `which(birthwt$ftv > 3)`) :

```
> table(birthwt$ftv)

  0   1   2   3   4   6
100  47  30   7   4   1
```

On peut décider de recoder cette variable de comptage en variable qualitative à trois niveaux, `ftv2`, après avoir regroupé les trois dernières valeurs (3, 4 et 6) avec la valeur 2 dans une modalité appelée 2+ :

```
> birthwt$ftv2 <- birthwt$ftv
> birthwt$ftv2[birthwt$ftv2 > 2] <- 2
> birthwt$ftv2 <- factor(birthwt$ftv2, labels=c("0","1","2+"))
> table(birthwt$ftv2)

  0   1  2+
100  47  42
```

On peut représenter ces données sous forme d'un diagramme de dispersion décrivant les variations de `bwt` selon les niveaux de `ftv2` à l'aide de la commande `xyplot()`. L'option `type=c("p","a")` permet de combiner la représentation des observations individuelles sous forme de points (`"p"`) avec les moyennes pour chaque niveau du facteur `ftv2` (`"a"`). L'option `jitter.x=TRUE` ajoute un léger décalage horizontal des points pour faciliter la lecture en cas de chevauchement des points (cela n'affecte en rien les valeurs lues sur l'axe des ordonnées puisque le décalage est horizontal) :

```
> xyplot(bwt ~ ftv2, data=birthwt, type=c("p","a"),jitter.x=TRUE)
```

Un modèle d'ANOVA classique donnerait les résultats suivants :

```
> summary(aov(bwt ~ ftv2, data=birthwt))
             Df   Sum Sq Mean Sq F value Pr(>F)
ftv2          2  1887925  943963    1.79   0.17
Residuals   186 98081730  527321
```

Pour tester un effet de tendance linéaire entre ces deux variables, on peut utiliser deux approches : l'approche basée sur la régression linéaire (la variable `ftv2` est alors considérée comme une variable numérique et non un facteur), qui sera vue plus en détail au chapitre suivant, ou celle reposant sur l'usage de contrastes associés aux niveaux de `ftv2`.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

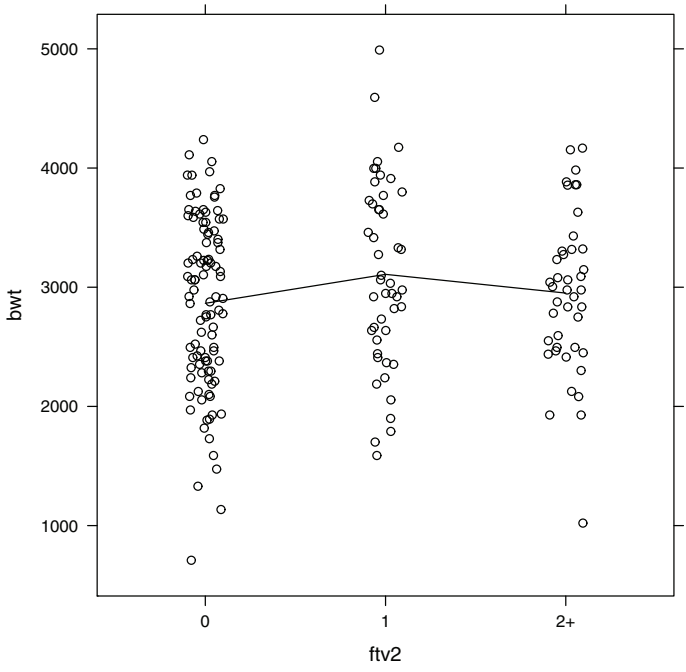


Figure 4.3. *Distribution du poids des bébés en fonction de la fréquence des visites gynécologiques durant le premier trimestre de grossesse*

Dans l'approche par régression linéaire, on se contente de considérer que la variable explicative est en fait une variable numérique et cela revient à effectuer une régression en considérant les variables `bwt` et `ftv2` sans convertir cette dernière en facteur comme on l'a fait plus haut. On peut tout de même indiquer à R de traiter la variable qualitative `ftv2` comme une variable numérique en utilisant la commande `as.numeric()`. Le résultat de la régression linéaire, réalisée à l'aide de la commande `lm()`, est résumé dans le tableau d'analyse de variance ci-après. Il indique que les données ne sont pas compatibles avec l'existence d'une relation linéaire entre les variables `bwt` et `ftv2` :

```
> anova(lm(bwt ~ as.numeric(ftv2), birthwt))
Analysis of Variance Table

Response: bwt
                  Df   Sum Sq Mean Sq F value Pr(>F)
as.numeric(ftv2)   1   542578  542578  1.0205 0.3137
Residuals        187 99427078  531696
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Notons que l'on peut aisément vérifier comment R a transformé les niveaux de `ftv2` en valeurs numériques :

```
> levels(birthwt$ftv2)
[1] "0"  "1"  "2+"
> sort(unique(as.numeric(birthwt$ftv2)))
[1] 1 2 3
```

Les différents niveaux du facteurs sont bien codés comme des nombres entiers, avec un décalage d'une unité entre chaque niveau.

Concernant la méthode des contrastes, R fournira automatiquement ce type de test de tendance lorsque la variable qualitative est spécifiée comme un `factor` à modalités ou niveaux ordonnés à l'aide de la commande `ordered()` ou `as.ordered()`. Dans le cas d'une variable à k modalités, R construira automatiquement $k-1$ contrastes orthogonaux permettant de tester l'existence d'une relation linéaire, quadratique, cubique, etc. Pour notre exemple, la variable `ftv2` possède trois niveaux et R fournira deux contrastes : le premier pour la tendance linéaire et le second pour la tendance quadratique (qui revient à considérer les valeurs au carré de `ftv2` dans un modèle de régression) :

```
> birthwt$ftv3 <- ordered(birthwt$ftv2)
> res <- aov(bwt ~ ftv3, data=birthwt)
> summary(res, split=list(ftv3=c(Linear=1, Quadratic=2)))
                   Df   Sum Sq Mean Sq F value Pr(>F)
ftv3                2  1887925  943963   1.790  0.170
  ftv3: Linear      1   542578  542578   1.029  0.312
  ftv3: Quadratic   1  1345348 1345348   2.551  0.112
Residuals         186 98081730  527321
```

A titre de remarque, un résultat similaire peut être obtenu en utilisant la commande `summary.lm()` (cette fois, on est obligé de spécifier explicitement le suffixe `.lm`) pour obtenir un résumé des coefficients du modèle linéaire :

```
> summary.lm(res)
v
Call:
aov(formula = bwt ~ ftv3, data = birthwt)

Residuals:
     Min       1Q   Median       3Q      Max
-2156.14  -484.88    26.12   578.86  1882.00
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  2974.67      56.81  52.360   <2e-16 ***
ftv3.L         60.63      94.42   0.642    0.522
ftv3.Q       -163.29     102.23  -1.597    0.112
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 726.2 on 186 degrees of freedom
Multiple R-squared:  0.01888,Adjusted R-squared:  0.008335
F-statistic:  1.79 on 2 and 186 DF,  p-value: 0.1698
```

4.3. Approche non paramétrique de l'ANOVA à un facteur

L'alternative non paramétrique à l'ANOVA à un facteur est l'ANOVA de Kruskal-Wallis qui consiste en une ANOVA réalisée sur les rangs des observations plutôt que les valeurs d'origine. La commande R permettant de réaliser ce type d'analyse est `kruskal.test()`, et celle-ci s'utilise de la même manière que `aov()` à l'aide d'une formule dans laquelle on indique la variable réponse et la variable explicative. Contrairement à `aov()`, il n'est pas nécessaire de combiner la commande `kruskal.test()` avec `summary()`.

Voici un exemple d'utilisation avec les données de poids à la naissance selon l'éthnicité de la mère :

```
> kruskal.test(bwt ~ race, data=birthwt)

Kruskal-Wallis rank sum test

data:  bwt by race
Kruskal-Wallis chi-squared = 8.5199, df = 2, p-value = 0.01412
```

Des comparaisons par paires de traitement peuvent être réalisées à l'aide de simples tests de Wilcoxon, selon le même principe que celui présenté en section 4.2.2.

4.4. ANOVA à deux facteurs

En présence de deux facteurs, la formulation du modèle d'ANOVA reste sensiblement équivalente à celle présentée plus haut. On exprime la relation entre les variables à l'aide d'une formule R, les facteurs de classification se trouvant à droite du symbole ~. Cette approche par formule est dérivée de la notation symbolique pour les plans d'expérience [WIL 73]. Par exemple, `y ~ x1 + x2` représente un

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

modèle à deux facteurs `x1` et `x2`. Pour représenter le terme d'interaction, on utilise le symbole `:` pour lier les deux variables. Avec les notations précédentes, on utiliserait `y ~ x1 + x2 + x1:x2` pour représenter le modèle d'ANOVA complet (deux effets principaux et un effet d'interaction). Cette dernière notation peut se simplifier sous la forme `y ~ x1 * x2`. Dans ce contexte, on appellera « traitement » le croisement de deux modalités de chacun des facteurs.

R utilise par défaut des sommes de carrés dites de type II. Dans le cas des plans d'expérience équilibrés, les différentes méthodes de calcul des sommes de carrés fournissent des résultats équivalents. Pour utiliser d'autres sommes de carrés, il est nécessaire d'utiliser la commande `drop1()` ou `Anova()` (package `car`).

4.4.1. *Construction du tableau d'ANOVA*

Si l'on s'intéresse à l'effet de l'ethnicité et des antécédents d'hypertension chez la mère sur le poids des nouveaux-nés, on peut formuler le modèle d'ANOVA, incluant l'interaction entre ces deux facteurs, de la manière suivante :

```
> m <- aov(bwt ~ race + ht + race:ht, data=birthwt)
```

Comme on l'a dit, l'instruction `aov(bwt ~ race * ht, data=birthwt)` est strictement équivalente. Les résultats de ce modèle sont indiqués ici :

```
> summary(m)
             Df   Sum Sq Mean Sq F value  Pr(>F)
race          2  5015725 2507863   4.973 0.00789 **
ht            1  1776713 1776713   3.523 0.06211 .
race:ht       2   889649  444825   0.882 0.41568
Residuals   183 92287568  504304
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Le tableau d'ANOVA indique pour chaque source de variabilité la somme de carrés associée, les degrés de liberté, le carré moyen et le test F. On voit que dans ce cas, seul le facteur `race` est significatif en considérant un risque d'erreur de 5 %. L'interaction n'est pas significative, ce qui indique que l'effet de ce facteur ne dépend pas du niveau de `ht`.

Le modèle sans interaction s'obtient sans difficulté en omettant le dernier terme du modèle, soit :

```
> summary(aov(bwt ~ race + ht, data=birthwt))
             Df   Sum Sq Mean Sq F value  Pr(>F)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
race          2  5015725 2507863   4.979 0.00783 **
ht            1  1776713 1776713   3.528 0.06193 .
Residuals   185 93177217  503661
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Outre un résumé numérique pour les moyennes des traitements, un graphique d'interaction permet souvent de mieux se représenter les résultats. On peut le construire à l'aide des commandes suivantes (figure 4.4) :

```
> xyplot(bwt ~ race, data=birthwt, groups=ht, type=c("a","g"),
         auto.key=list(corner=c(0,1), title="ht",
                       cex.title=1, points=FALSE, lines=TRUE))
```

Par souci de simplicité, nous ne considérons pas l'ajout des écarts-type ou intervalles de confiance. Les options de légende utilisées permettent de préciser la variable utilisée pour le groupement et la position de la légende (coin supérieur droit), dans laquelle on affiche des segments de ligne (`lines=TRUE`) plutôt que des points. L'option `type=c('a','g')` permet de n'afficher que les moyennes des traitements (croisement des niveaux des deux facteurs), ainsi qu'un quadrillage du graphique ce qui facilite le repérage des valeurs affichées.

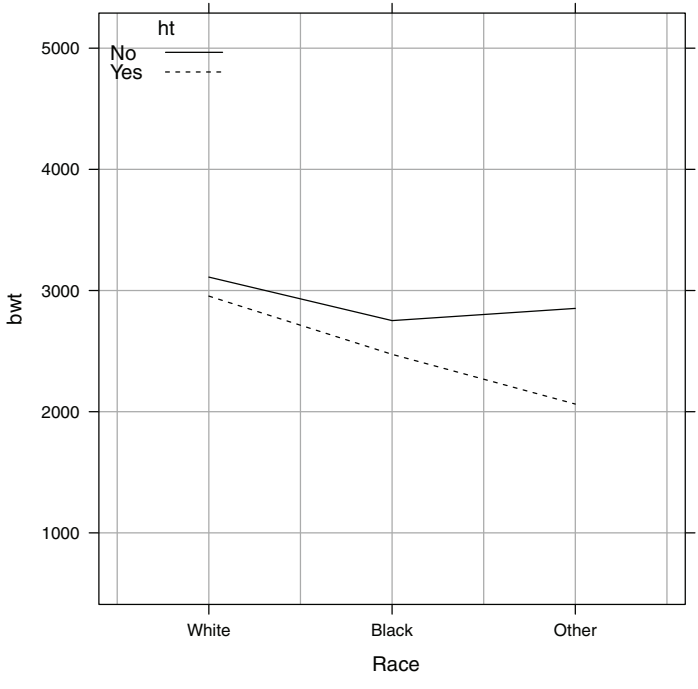


Figure 4.4. *Diagramme d'interaction pour le modèle d'ANOVA à deux facteurs*

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Pour obtenir les moyennes des traitements, on peut utiliser soit la commande `tapply()`, soit la commande `aggregate()`. Avec cette dernière, on écrirait ainsi, en utilisant exactement la même formule :

```
> aggregate(bwt ~ race + ht, data=birthwt, mean)
   race  ht      bwt
1 White  No 3110.890
2 Black  No 2751.826
3 Other  No 2852.413
4 White Yes 2954.000
5 Black Yes 2473.333
6 Other Yes 2063.000
```

Notons également que R dispose de la commande `model.tables()` qui calcule automatiquement les effets moyens (déviations entre les moyennes conditionnelles et la moyenne générale) ainsi que les erreurs type associées :

```
> model.tables(m)
Tables of effects

 race
    White  Black  Other
    158.1 -224.9 -139.3
rep  96.0   26.0   67.0

 ht
        No  Yes
     25.15 -371
rep 177.00   12

 race:ht
       ht
race    No     Yes
  White  -12.6  229.6
  rep     91.0    5.0
  Black  -13.9  106.7
  rep     23.0    3.0
  Other   23.3 -367.0
  rep     63.0    4.0
```

4.4.2. *Diagnostic du modèle*

Les hypothèses du modèle d'ANOVA à deux facteurs restent identiques à celles du modèle à un facteur (indépendance des observations, normalité et égalité des

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

variances), à ceci près qu'elles portent sur les résidus du modèle, et non les valeurs brutes observées. Pour tester l'hypothèse d'homoscédasticité (égalité des variances), par exemple, on prendra donc garde au fait que le test de Bartlett doit porter sur les six traitements formés à partir du terme d'interaction, et non sur les niveaux de chacun des facteurs séparément. Avec le modèle précédent, `bwt ~ race * ht`, on formulerait le test de la manière suivante :

```
> htrace <- with(birthwt, interaction(race, ht))
> bartlett.test(bwt ~ htrace, data=birthwt)

Bartlett test of homogeneity of variances

data:  bwt by htrace
Bartlett's K-squared = 5.1225, df = 5, p-value = 0.4011
```

Le même principe de décomposition selon le terme d'interaction peut être utilisé pour visualiser la distribution des réponses, des valeurs prédites ou des résidus en fonction des traitements.

4.5. Ce qu'il faut retenir

– Plutôt que de considérer des séries d'observations arrangées en colonne (cas d'un plan équilibré), on préférera travailler avec un tableau de données faisant explicitement apparaître une variable réponse et un ou plusieurs facteurs de classification.

– La commande `aggregate()` peut avantageusement remplacer la commande `tapply()` grâce à l'usage de formules R.

– La commande `aov()` permet de construire un modèle d'ANOVA, le tableau de décomposition de la variance étant directement fourni par la commande `summary()`.

4.6. Applications

1) Dans une étude sur le gène du récepteur à œstrogènes, des généticiens se sont intéressés à la relation entre le génotype et l'âge de diagnostic du cancer du sein. Le génotype était déterminé à partir des deux allèles d'un polymorphisme de restriction de séquence (1,6 et 0,7 kb), soit trois groupes de sujets : patients homozygotes pour l'allèle 0,7 kb (0,7/0,7), patients homozygotes pour l'allèle 1,6 kb (1,6/1,6), et patients hétérozygotes (1,6/0,7). Les données ont été recueillies sur 59 patientes atteintes d'un cancer du sein, et sont disponibles dans le fichier `polymorphism.dta` (fichier Stata) [DUP 09]. Les données moyennes sont indiquées dans le tableau 4.1.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

	Génotype			
	1,6/1,6	1,6/0,7	0,7/0,7	Total
Nombre de patients	14	29	16	59
Age lors du diagnostic				
Moyenne	64,64	64,38	50,38	60,64
Ecart-type	11,18	13,26	10,64	13,49
IC 95 %	(58,1-71,1)	(59,9-68,9)	(44,3-56,5)	

Tableau 4.1. *Polymorphisme du gène du récepteur à œstrogènes*

a) Tester l'hypothèse nulle selon laquelle l'âge de diagnostic ne varie pas selon le génotype à l'aide d'une ANOVA. Représenter sous forme graphique la distribution des âges pour chaque génotype.

b) Les intervalles de confiance présentés dans le tableau 4.1 ont été estimés en supposant l'homogénéité des variances, c'est-à-dire en utilisant l'estimé de la variance commune ; donner la valeur de ces intervalles de confiance sans supposer l'homoscédasticité.

c) Estimer les différences de moyenne correspondant à l'ensemble des combinaisons possibles des trois génotypes, avec une estimation de l'intervalle de confiance à 95 % associé et un test paramétrique permettant d'évaluer le degré de significativité de la différence observée.

d) Représenter graphiquement les moyennes de groupe avec des intervalles de confiance à 95 %.

Pour charger les données, il est nécessaire d'importer la librairie `foreign` qui permet de lire les fichiers enregistrés par Stata :

```
> library(foreign)
> polymsm <- read.dta("polymorphism.dta")
> head(polymsm)
```

Notons que `polymsm` est un data frame, ce qui sera utile pour utiliser les commandes graphiques ou réaliser l'ANOVA. La première colonne contient une série d'identifiants uniques pour les individus ; elle ne sera pas utile dans le cas présent. Pour calculer les moyennes et écarts-type de chaque groupe, on peut procéder comme suit :

```
> with(polymsm, tapply(age, genotype, mean))
> with(polymsm, tapply(age, genotype, sd))
```

La distribution des âges selon le génotype est indiquée dans le diagramme en boîtes à moustaches suivant :

```
> bwplot(age ~ genotype, data=polymsm)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Il est également possible d'utiliser des histogrammes, en utilisant `histogram()` au lieu de `bwplot()` :

```
> histogram(~ age | genotype, data=polymsm)
```

Le modèle d'ANOVA est réalisé à l'aide de la commande `aov()` :

```
> aov.res <- aov(age ~ genotype, data=polymsm)
> summary(aov.res)
```

La statistique de test F est reportée dans la colonne `F value`, avec le degré de signification associé dans la colonne suivante (`Pr(>F)`). L'ANOVA indique qu'au moins une paire de moyennes est significativement différente, en considérant un rique d'erreur de 5 %.

On peut vérifier l'exactitude des intervalles de confiance reportés dans le tableau présenté plus haut. Pour cela, il nous faut une estimation de l'erreur résiduelle, qui est simplement la racine carrée du carré moyen associé au terme d'erreur dans le tableau d'ANOVA précédent :

```
> mse <- unlist(summary(aov.res))["Mean Sq2"]
> se <- sqrt(mse)
```

Soit, une estimation de la racine de la variance commune de `round(se,2)`. A partir de là, on peut construire les intervalles de confiance à 95 % pour chaque moyenne de groupe comme suit :

```
> ni <- table(polymsm$genotype)
> n <- sum(ni)
> m <- with(polymsm, tapply(age, genotype, mean))
> lci <- m - qt(0.975, n-3) * se / sqrt(ni)
> uci <- m + qt(0.975, n-3) * se / sqrt(ni)
> rbind(lci, uci)
```

Dans le cas plus général, pour l'estimation de l'intervalle de confiance d'une moyenne (ou d'une proportion) on pourra se référer à la commande `epi.conf()` du package `epiR`, par exemple :

```
> library(epiR)
> epi.conf(polymsm$age[polymsm$genotype=="1.6/1.6"],
           ctype = "mean.single")
```

Pour représenter graphiquement les moyennes de groupe, on peut utiliser deux approches. La première consiste à constuire le graphique manuellement, en affichant les moyennes sous forme de points et les intervalles de confiance sous forme de segments :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> mm <- as.data.frame(cbind(m, lci, uci))
> mm$g <- levels(polymsm$genotype)
> rownames(mm) <- NULL
> dotplot(m ~ g, data=mm, ylim=c(40,75),
          panel=function(x, y, ...) {
            panel.dotplot(x, y, ...)
            panel.segments(x, mm$lci, x, mm$uci)
          })
```

On a donc regroupé les moyennes de groupe, ainsi que les bornes inférieures et supérieures des intervalles de confiance et les niveaux du facteur de classification dans un même data frame. La commande graphique consiste à enchaîner un appel à `panel.dotplot()` pour afficher les moyennes, puis `panel.segments()` pour tracer les intervalles de confiance associés.

L'autre solution, plus simple, consiste à utiliser la commande `xYplot()` (à ne pas confondre avec `xyplot()`) du package `Hmisc` décrit en annexe, ou `segplot()` du package `latticeExtra`. Dans le premier cas, la seule subtilité est que la variable explicative (reportée sur l'axe des abscisses) doit être traitée en tant que variable numérique. Il faut adapter légèrement la syntaxe pour parvenir à l'effet désiré, en particulier afficher correctement les étiquettes correspondant aux différents génotypes :

```
> xYplot(Cbind(m,lci,uci) ~ 1:3, data=mm,
         scales=list(x=list(at=1:3, labels=mm$g)),
         xlab="", ylim=c(40,75))
```

2) Un service d'obstétrique s'intéresse au poids de nouveaux-nés nés à terme et âgés d'un mois [PEA 05]. Pour cet échantillon de 550 bébés, on dispose également d'une information concernant la parité (nombre de frères et sœurs), mais on sait qu'il n'y aucune relation de gemellité parmi les enfants ayant des frères et sœurs. L'objet de l'étude est de déterminer si la parité (quatre classes) influence le poids des nouveaux-nés à un mois. Les données sont résumées dans le tableau 4.2, et elles sont disponibles dans un fichier SPSS nommé `weights.sav`.

	Nombre de frères et sœurs				
	0	1	2	≥ 3	Total
Echantillon					
Effectif	180	192	116	62	550
Fréquence	32,7	34,9	21,1	11,3	100,0
Poids (kg)					
Moyenne	4,26	4,39	21,1	11,3	
Ecart-type	0,62	0,59	0,61	0,54	
(Min-Max)	(2,92-5,75)	(3,17-6,33)	(3,09-6,49)	(3,20-5,48)	

Tableau 4.2. *Poids des nouveaux-nés*

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

a) Vérifier les données reportées dans le tableau précédent.

b) Procéder à une analyse de variance à un facteur. Conclure sur la significativité globale et indiquer la part de variance expliquée par le modèle.

c) Afficher la distribution des poids selon la parité. Procéder à un test d'homogénéité des variances (rechercher dans l'aide en ligne le test de Levenne).

d) On décide de regrouper les deux dernières catégories (2 et $\geq$ 3). Refaire l'analyse et comparer aux résultats obtenus en (b).

e) Réaliser un test de tendance linéaire (par ANOVA) sur les données recodées en trois niveaux pour la parité.

Pour charger les données, il est nécessaire d'importer la librairie `foreign` qui permet de lire les fichiers enregistrés par SPSS :

```
> library(foreign)
> weights <- read.spss("weights.sav", to.data.frame=TRUE)
> str(weights)
```

Comme on l'a vu dans un des exercices précédents, l'option `to.data.frame=TRUE` est importante car c'est elle qui permet de stocker les données lues sous forme de data frame (variables en colonnes, individus en lignes).

Dans un premier temps, procédons au résumé numérique uni et bivarié des données d'intérêt (variables `WEIGHT` et `PARITY`). Pour la variable qualitative, les tableaux d'effectifs et de fréquences relatives sont obtenus ainsi :

```
> table(weights$PARITY)
> round(prop.table(table(weights$PARITY))*100, 1)
```

Pour la variable quantitative, les moyennes et écarts-types par type de parité sont obtenus comme suit :

```
> round(with(weights, tapply(WEIGHT, PARITY, mean)), 2)
> round(with(weights, tapply(WEIGHT, PARITY, sd)), 2)
```

Pour l'analyse de variance, on utilise le même principe qu'à l'exercice précédent :

```
> aov.res <- aov(WEIGHT ~ PARITY, data=weights)
> summary(aov.res)
```

Le test F est significatif, donc on peut rejeter l'hypothèse nulle d'égalité des quatre moyennes. La part de variance expliquée est simplement le rapport entre la somme des carrés (`Sum Sq`) associée au facteur d'étude (`PARITY`), soit 3,48, et la somme des carrés totaux, soit $3,48/(3,48 + 195,36)$: on obtient 0.018, soit environ 2 %.

Pour afficher la distribution des poids, on peut bien sûr utiliser des boîtes à moustaches, comme dans les exercices précédents. Si l'on souhaite visualiser directement

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

les données individuelles, un diagramme de dispersion conditionné sur les groupes est également intéressant :

```
> stripplot(WEIGHT ~ PARITY, data=weights, ylab="Poids (kg)",
            jitter.data=TRUE)
```

L'autre alternative consiste à utiliser des histogrammes pour chaque groupe :

```
> histogram(~ WEIGHT | PARITY, data=weights)
```

On notera que l'on a ajouté un léger décalage aléatoire des données (sur l'axe horizontal uniquement) en utilisant l'option `jitter.data=TRUE`, ce qui permet d'éviter le chevauchement total des points.

Le test de Levene n'est pas disponible dans les commandes de base de R, mais on peut installer le package `car` qui fournit la commande `leveneTest()` et que l'on peut utiliser comme n'importe quelle procédure de test sous R :

```
> library(car)
> leveneTest(WEIGHT ~ PARITY, data=weights)
```

Une alternative consisterait à utiliser un test de Bartlett (`bartlett.test()`) pour l'homogénéité des variances. Cette commande s'utilise exactement de la même manière que `leveneTest()` (variable réponse décrite par le facteur d'étude, nom du data frame où chercher les variables).

Pour regrouper les deux dernières catégories, on peut créer une nouvelle variable et générer manuellement les nouvelles modalités associées, ou plus simplement recoder la variable qualitative `PARITY` en une nouvelle variable :

```
> PARITY2 <- weights$PARITY
> levels(PARITY2)[3:4] <- "2 siblings or more"
```

Le modèle d'analyse de variance est construit de la même manière que précédemment :

```
> aov.res2 <- aov(WEIGHT ~ PARITY2, data=weights)
> summary(aov.res2)
```

Enfin, pour réaliser un test de tendance centrale, il y a deux solutions : soit par la méthode des contrastes, soit par une approche de régression linéaire. Ces deux approches fournissent des résultats identiques, et suppose généralement que les niveaux du facteur de classification sont équi-espacés (variation du même nombre d'unités entre chaque niveau du facteur). Dans le cas présent, on utlisera la commande `lm()` pour réaliser une régression linéaire simple (voir le chapitre sur la régression linéaire, pour plus de détails) :

```
> lm.res <- lm(WEIGHT ~ as.numeric(PARITY2), data=weights)
> summary(lm.res)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Le test de la tendance linéaire pour l'ANOVA correspond au test de la pente de la droite de régression, qui ici est significatif suggérant une augmentation du poids moyen avec la taille de la fratrie. De manière équivalente, on peut utiliser l'approche consistant à recoder le facteur de classification en facteur à modalités ordonnées (ou niveaux) à l'aide de la commande `as.ordered()`. Le test pour la tendance linéaire correspondant au contraste nommé `as.ordered(PARITY2).L` dans la sortie suivante :

```
> summary(lm(WEIGHT ~ as.ordered(PARITY2), data=weights))
```

3) Le jeu de données `ToothGrowth` disponible dans R contient les données d'une étude sur la longueur des odontoblastes (variable `len`) chez dix cochons d'inde après administration de vitamine C à différentes doses (0,5, 1 ou 2 mg, variable `dose`) sous forme d'acide ascorbique ou de jus d'orange (variable `supp`) [BLI 52].

a) Vérifier la distribution d'effectifs selon les différents traitements (croisement des modalités des deux facteurs, `supp` et `dose`) de ce plan d'expérience.

b) Calculer la moyenne et l'écart-type de chaque traitement.

c) Construire un tableau d'analyse de variance pour le modèle complet avec interaction entre les deux facteurs.

d) Réaliser un graphique d'interaction représentant les valeurs moyennes de la variable réponse en fonction des niveaux des deux variables explicatives.

e) Vérifier que l'hypothèse d'homogénéité des variances est acceptable pour ces données.

Pour charger les données, on utilisera la commande `data()` puisque les données sont fournies avec R :

```
> data(ToothGrowth)
> str(ToothGrowth)
```

L'examen du codage des variables à l'aide de `str()` indique que les variables `supp` et `dose` sont actuellement traitées comme des variables numériques. Pour la variable `supp`, cela ne pose pas vraiment de problèmes car elle ne possède que deux niveaux distincts. Par contre, la variable `dose` doit être recodée en facteur si l'on souhaite utiliser des modèles d'ANOVA :

```
> ToothGrowth$dose <- factor(ToothGrowth$dose)
```

Ici, on considère les modalités de `dose` comme non ordonnées.

Pour vérifier la distribution des effectifs selon les différentes modalités de ce plan factoriel, il est tout à fait possible d'utiliser `table()` ou `xtabs()`, par exemple :

```
> xtabs(~ dose + supp, data=ToothGrowth)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Cependant, R fournit également une commande permettant de résumer les effectifs de chaque facteur et de l'interaction entre ces deux facteurs (ce qui correspond aux effectifs associés à chacun des traitements lorsque le plan d'expérience est équilibré). Dans un premier temps, on peut indiquer à l'aide d'une formule R le type de relation qui nous intéresse. Ici, cela correspond au modèle d'ANOVA où l'on cherche à expliquer les variations de la variable réponse, `len`, en fonction des niveaux des variables `supp` et `dose`, ces deux facteurs étant considérés en interaction :

```
> fm <- len ~ supp * dose
> replications(fm, data=ToothGrowth)
```

L'intérêt d'utiliser une formule est que celle-ci pourra être réutilisée pour calculer les moyennes par traitement et le modèle d'ANOVA. Le symbole `*` dans la formule permet de formuler de manière concise que les deux facteurs sont en interaction dans le modèle.

Voici comment on peut calculer les moyennes et écarts-type, après avoir défini une petite fonction R permettant de calculer ces deux statistiques à partir d'une série d'observations :

```
> f <- function(x) c(mean = mean(x), sd = sd(x))
> aggregate(fm, ToothGrowth, f)
```

Le modèle d'ANOVA ne pose pas de difficulté particulière et le tableau d'analyse de variance s'obtiendra en combinant `aov()` et `summary()` :

```
> aov.fit <- aov(fm, data = ToothGrowth)
> summary(aov.fit)
```

Il est possible de construire un tableau des effets simples dans ce modèle à l'aide de la commande `model.tables()`. Dans ce cas, les valeurs reportées par R correspondent aux différences entre les moyennes pour chaque niveau des facteurs et la « grande moyenne », ou moyenne générale :

```
> model.tables(aov.fit, type = "means", se = TRUE,
               cterms = "supp:dose")
```

Quant au graphique d'interaction, on utilisera un simple diagramme de dispersion légèrement adapté :

```
> xyplot(len ~ dose, data = ToothGrowth, groups = supp,
         type = c("a", "g"), xlab = "Dose (mg)",
         ylab = "Tooth length", lwd = 2,
         auto.key = list(space = "top", columns = 2,
                         points = FALSE, lines = TRUE))
```

L'option `type="a"` permet de calculer automatiquement les moyennes conditionnellement aux niveaux des facteurs entrant dans la composition du graphique. R

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

dispose également d'une commande `interaction.plot()` parmi les graphiques de base.

La distribution des observations pour chaque traitement permet d'inspecter visuellement si l'hypothèse d'homogénéité des variances paraît acceptable ou non. Cependant, ce qui compte ici ce sont bien les traitements (puisque le modèle d'ANOVA inclut le terme d'interaction entre les deux facteurs) et par conséquent l'affichage sous forme de boîtes à moustaches de la distribution de la variable réponse suppose que l'on travaille bien avec les traitements. Pour cela, il est possible de recourir à la commande `interaction()` qui permet de combiner l'ensemble des niveaux de chaque facteur :

```
> bwplot(len ~ interaction(supp, dose), data = ToothGrowth,
         do.out = FALSE, pch = "|")
```

Un test formel d'égalité des variances est disponible *via* la commande `bartlett.test()` :

```
> bartlett.test(len ~ interaction(supp, dose), data=ToothGrowth)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 5

Corrélation et régression linéaire

Dans ce chapitre, on s'intéressera aux mesures d'association linéaire ou monotone entre deux variables numériques (estimation ponctuelle ou par intervalle, test pour le coefficient de corrélation) et aux représentations graphiques exploratoires associées (diagramme de dispersion et courbe loess). Le modèle de régression linéaire est ensuite développé dans le cas d'une variable explicative (régression linéaire simple), en particulier l'estimation des coefficients de la droite de régression et la construction des tests de significativité associés, le tableau d'analyse de variance associé au modèle de régression, la prédiction ponctuelle et par intervalle, ainsi que la vérification des conditions d'application et le diagnostic du modèle par l'analyse des résidus.

5.1. Corrélation

5.1.1. *Statistiques descriptives*

Les résumés descriptifs pour deux variables numériques s'obtiennent aisément avec `summary()` ou une commande opérant par variable (`sapply()`, vue au chapitre précédent, ou `apply()`), dans le cas où les variables sont arrangées en colonnes. Si l'on s'intéresse aux mesures de poids des mères (`lwt`) et des nouveaux-nés (`bwt`), les commandes suivantes fournissent essentiellement les mêmes informations :

```
> summary(birthwt[,c("bwt","lwt")])
> summary(birthwt[,c(10,3)])
> summary(subset(birthwt, select=c(bwt, lwt)))
> sapply(birthwt[,c("bwt","lwt")], summary)
> apply(birthwt[,c(10,3)], 2, summary)
         bwt   lwt
Min.     709  80.0
1st Qu. 2414 110.0
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
Median  2977 121.0
Mean    2945 129.8
3rd Qu. 3487 140.0
Max.    4990 250.0
```

Quelques remarques : il est toujours plus facile de relire des scripts R lorsque les variables sont explicitement nommées (`c("bwt","lwt")`) plutôt qu'adressées par leur position dans le tableau (`c(10,3)`); la commande `apply()` présente l'avantage de fonctionner également avec les objets R de type `matrix` (des tableaux rectangulaires composés de variables du même type); le nom des variables dans la commande `subset()` ne doit pas être entouré de quotes anglo-saxonnes.

Il est possible de visualiser directement les histogrammes de ces deux variables d'intérêt à l'aide de la commande `histogram()` (figure 5.1). La seule subtilité est que généralement les commandes du package `lattice` cherchent à unifier les unités de mesure pour les axes des ordonnées et des abscisses. Dans le cas présent, cela signifie que `histogram()` construira un axe des abscisses représentant les plus grandes variations (poids des bébés exprimés en grammes, avec une étendue 700 à 5 000) ce qui rendra illisible l'histogramme du poids des mères. Pour remédier à ce problème, il est nécessaire d'imposer à R de construire deux systèmes de coordonnées différents, *via* l'option `scales=`, de la manière suivante :

```
> histogram(~ bwt + lwt, data=birthwt, breaks=NULL, xlab="",
            scales=list(x=list(relation="free")))
```

En l'absence de variable de classification (facteur) permettant de séparer deux séries de mesures, il suffit d'indiquer la liste des variables dont on souhaite afficher la distribution sous forme d'histogramme séparées par un +.

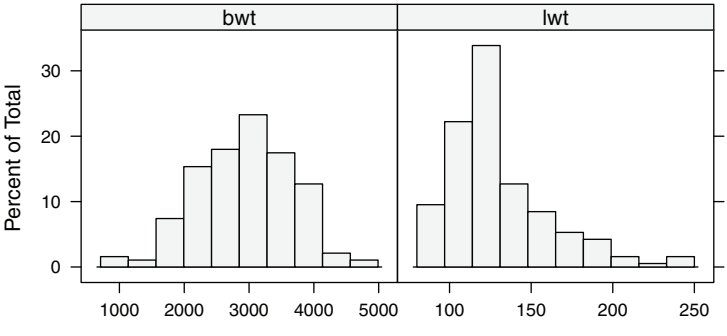


Figure 5.1. *Histogrammes des poids des mères (lwt, en kg) et des nouveaux-nés (bwt, en g). Les données sont présentées sous la forme de pourcentages (Percent of Total)*

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

5.1.2. *Diagramme de dispersion et courbe loess*

Avant de calculer n'importe quelle statistique sur une série bivariée d'observations, il est utile de visualiser graphiquement la relation entre les deux variables, le plus souvent sous forme d'un diagramme de dispersion. La commande `xyplot()` dispose d'un certain nombre d'options *via* l'option `type=` qui facilite l'identification d'une association plus ou moins linéaire entre les deux variables, en particulier l'option `type="smooth"`. Celle-ci permet de superposer sur le diagramme de dispersion une courbe de type « loess » [CLE 79] qui est obtenue à partir d'une sorte de régression locale dont on peut contrôler le degré de lissage, voir `help(loess)`.

Convertissons dans un premier temps les poids des mères en kilogrammes, puis affichons le diagramme de dispersion et la courbe loess :

```
> birthwt$lwt <- birthwt$lwt/2.2
> xyplot(bwt ~ lwt, data=birthwt, type=c("p","g","smooth"))
```

Le degré de lissage de la courbe loess peut se contrôler avec l'option `span=` ; par exemple, `span=1` fournira une courbe moins sensible aux variations locales et donc d'apparence beaucoup plus lisse (figure 5.2).

5.1.3. *Mesures d'association paramétrique et non paramétrique*

Le coefficient de corrélation de Pearson est obtenu à partir de la commande `cor()`, en indiquant les deux variables d'intérêt :

```
> with(birthwt, cor(bwt, lwt))
[1] 0.1857333
```

Lorsqu'il y a des données manquantes, il est nécessaire d'indiquer à R comment calculer le coefficient de corrélation dans ce cas. Une approche classique (dans le cas bivarié, ou pour construire des matrices de corrélation sur un plus grand nombre de variables) consiste à ne considérer que les paires d'observations complètes. Si l'on a deux variables `x` et `y`, R excluera les unités statistiques pour lesquelles `is.na(x)` ou `is.na(y)` vaut `TRUE` (ceci inclut évidemment la conjonction des deux conditions). L'option à utiliser dans la commande `cor` est `use="pairwise"`.

Pour calculer le coefficient de corrélation de Spearman (basé sur les rangs des observations), on ajoutera l'option `method="spearman"` :

```
> with(birthwt, cor(bwt, lwt, method="spearman"))
[1] 0.2488882
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

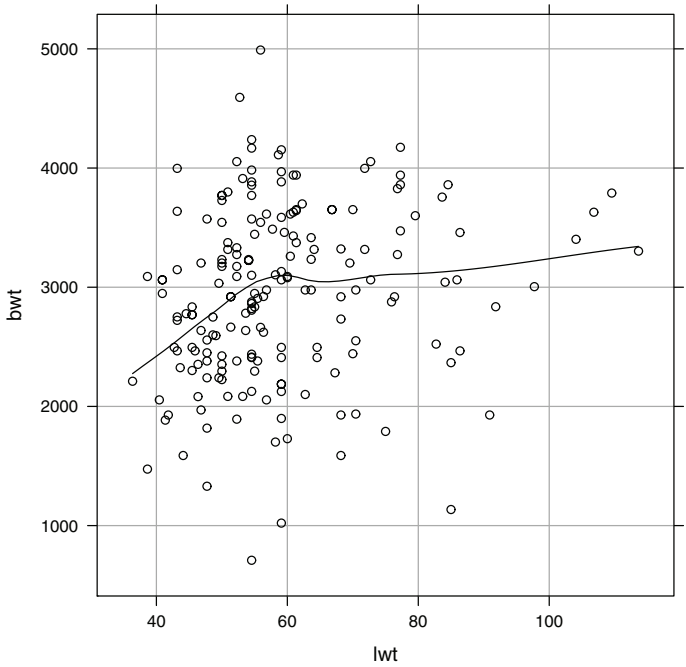


Figure 5.2. *Diagramme de dispersion représentant les variations du poids des nouveaux-nés (bwt) en fonction du poids des mères (lwt)*

Ces deux commandes fonctionnent également avec plus de deux variables : il suffit dans ce cas de fournir à `cor()` un tableau avec les variables d'intérêt, par exemple :

```
> cor(birthwt[,c("age","lwt","bwt")])
           age       lwt        bwt
age 1.00000000 0.1800732 0.09031781
lwt 0.18007315 1.0000000 0.18573328
bwt 0.09031781 0.1857333 1.00000000
```

5.1.4. *Estimation par intervalles et test d'inférence*

La commande `cor.test()` fournit à la fois un test de l'hypothèse $H_0 : \rho = 0$ (nullité du paramètre, c'est-à-dire corrélation théorique égale à zéro) *via* un test de Student, ainsi qu'un intervalle de confiance calculé par transformation inverse de Fisher. Dans le cas non paramétrique, R propose de calculer un degré de signication exact pour les échantillons de taille $n < 50$ (sans *ex-æquo*) ; le cas échéant, une approximation par la loi normale est utilisée :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> with(birthwt, cor.test(bwt, lwt))

Pearson's product-moment correlation

data:  bwt and lwt
t = 2.5848, df = 187, p-value = 0.0105
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 0.04417405 0.31998094
sample estimates:
      cor
0.1857333
```

Notons que l'on peut utiliser une notation par formule avec cette commande, de manière similaire à celle utilisée pour `xtabs()` lorsque les deux variables jouent un rôle symétrique. Il est également possible de restreindre les observations utilisées pour l'estimation du paramètre et le test de sa nullité à l'aide d'une option `subset=`. On utilisera, par exemple :

```
> cor.test(~ bwt + lwt, data=birthwt, subset=bwt > 2500)

Pearson's product-moment correlation

data:  bwt and lwt
t = 1.6616, df = 128, p-value = 0.09905
alternative hypothesis: true correlation is not equal to 0
95 percent confidence interval:
 -0.02757158  0.30974090
sample estimates:
      cor
0.1453043
```

pour restreindre le calcul aux seules observations pour lesquelles le poids des nouveaux-nés est supérieur à 2,5 kg.

On ajoutera l'option `method="spearman"` pour réaliser un test d'inférence équivalent portant sur le coefficient de corrélation basé sur les rangs des observations.

5.2. Régression linéaire simple

5.2.1. *Droite de régression*

Pour réaliser une régression linéaire, et plus généralement n'importe quel modèle linéaire (régression linéaire multiple, analyse de variance, analyse de covariance), R

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

offre la commande `lm()`. Son usage est globalement identique à celui de `aov()` vu au chapitre précédent : on décrit à l'aide d'une formule R la relation entre les variables (variable réponse et variable explicative), le data frame dans lequel se trouvent les variables, et éventuellement le sous-ensemble d'observations sur lequel on souhaite travailler avec l'option `subset=`.

Pour modéliser la relation entre le poids des nouveaux-nés (variable réponse) et le poids des mères (variable explicative ou prédicteur), on utilisera la formulation suivante :

```
> lm(bwt ~ lwt, data=birthwt)

Call:
lm(formula = bwt ~ lwt, data = birthwt)

Coefficients:
(Intercept)          lwt
   2369.624        9.744
```

R renvoie l'ordonnée à l'origine (« Intercept »), c'est-à-dire la valeur de `bwt` lorsque `lwt`=0, et la pente de la droite de régression, celle-ci reflétant les variations de `bwt` lorsque `lwt` varie d'une unité.

Comme on l'a vu dans le cas de l'ANOVA avec `aov()` (section 4.2.1), l'information retournée par la commande `lm` est minimale. Pour obtenir le tableau des coefficients de régression, leur erreur standard et leur degré de signification obtenu à l'aide de tests de Student, on associera à nouveau la commande `lm()` à `summary()` :

```
> r <- lm(bwt ~ lwt, data=birthwt)
> summary(r)

Call:
lm(formula = bwt ~ lwt, data = birthwt)

Residuals:
     Min       1Q   Median       3Q      Max
-2192.12  -497.97    -3.84   508.32  2075.60

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 2369.624    228.493  10.371   <2e-16 ***
lwt            9.744      3.770   2.585   0.0105 *
---
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 718.4 on 187 degrees of freedom
Multiple R-squared:  0.0345,Adjusted R-squared:  0.02933
F-statistic: 6.681 on 1 and 187 DF,  p-value: 0.0105
```

En plus des tests de signification sur les coefficients de régression, R inclut des informations supplémentaires comme la variance résiduelle et le coefficient de détermination R^2 qui reflète la part de variance expliquée par la régression (voir section 5.2.2).

Il est possible de faire figurer la droite de régression sur le diagramme de dispersion, comme on l'a fait avec la courbe loess, à l'aide de l'option `type="r"` (au lieu de `type="smooth"`), bien que les deux options puissent être combinées (figure 5.3) :

```
> xyplot(bwt ~ lwt, data=birthwt, type=c("p","g","r"))
```

Le type de symbole utilisé par défaut (cercle) convient dans le cas des observations en nombre élevé, mais on peut soit en régler la taille (`cex=0.8`, par exemple), soit utiliser des symboles transparents (`pch=19, alpha=0.5`).

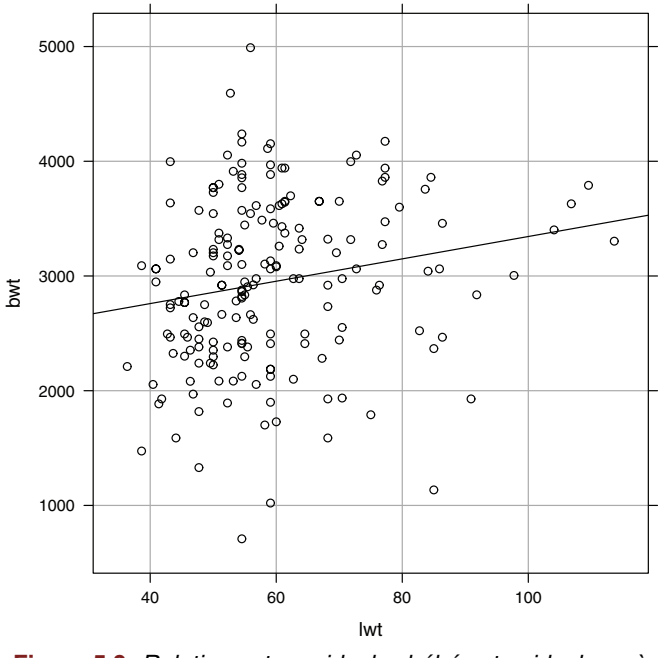


Figure 5.3. *Relation entre poids des bébés et poids des mères et droite de régression associée*

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

5.2.2. *Estimation par intervalles et tableau d'analyse de variance*

La commande `summary()` appliquée à un modèle de régression ne fournit pas d'intervalles de confiance pour les coefficients de régression. Pour cela, on utilisera la commande `confint()`, en précisant éventuellement la largeur de l'intervalle de confiance *via* l'option `level=` (par défaut, elle vaut 0,95 pour un intervalle à 95 %) :

```
> confint(r)
                  2.5 %     97.5 %
(Intercept) 1918.867879 2820.37916
lwt            2.307459   17.18061
```

On pourrait associer ces intervalles aux coefficients estimés à l'étape précédente en construisant un tableau à trois colonnes :

```
> res <- cbind(coef(r), confint(r))
> colnames(res)[1] <- "Coef"
> round(res, 3)
                Coef    2.5 %   97.5 %
(Intercept) 2369.624 1918.868 2820.379
lwt            9.744    2.307   17.181
```

On utilise ici le fait que la commande `coef()` renvoie les coefficients de régression estimés dans le modèle que l'on a appelé `r`, tandis que `cbind()` permet de concaténer des listes de nombres en colonnes (sous réserve que chaque liste possède le même nombre d'éléments).

Le tableau d'ANOVA associé au modèle de régression permet de décomposer la variabilité totale en une part de variance expliquée par le modèle et la résiduelle. Il s'obtient à l'aide de la commande `anova()` :

```
> anova(r)
Analysis of Variance Table

Response: bwt
           Df   Sum Sq Mean Sq F value Pr(>F)
lwt         1  3448639 3448639  6.6814 0.0105 *
Residuals 187 96521017  516155
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

On retrouve essentiellement les mêmes résultats que ceux retournés par la commande `summary()` appliquée au résultat de la commande `lm()`.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

5.2.3. *Prédictions à partir du modèle de régression*

Comme dans le cas de la commande `aov()` pour l'ANOVA, les deux commandes `fitted()` et `predict()` sont utilisées pour obtenir les valeurs ajustées et construire de nouvelles prédictions, respectivement. La commande `fitted(r)` renverra les valeurs prédites pour les poids des bébés observés. Dans ce cas, on parle généralement de « valeurs ajustées » :

```
> head(fitted(r))
      85       86       87       88       89       91
3175.721 3056.135 2834.680 2847.967 2843.538 2918.833
```

On peut facilement comparer les valeurs observées aux valeurs prédites par le modèle (situées sur la droite de régression) en juxtaposant les deux séries de valeurs de la manière suivante :

```
> head(cbind(birthwt$bwt, fitted(r)))
   [,1]     [,2]
85 2523 3175.721
86 2551 3056.135
87 2557 2834.680
88 2594 2847.967
89 2600 2843.538
91 2622 2918.833
```

Il est également possible de rajouter au diagramme de dispersion précédent les valeurs prédites en modifiant légèrement la formule R utilisée (figure 5.4) : dans ce cas, on fera apparaître deux variables à gauche de l'opérateur de liaison ~ pour indiquer à R qu'il y a deux séries de mesure à représenter en fonction de la même variable reportée sur l'axe des abscisses (ici, `lwt`) :

```
> xyplot(bwt + fitted(r) ~ lwt, data=birthwt)
```

La commande `predict()` s'utilise pour obtenir les valeurs prédites sur un ensemble d'observations autre que celui observé dans les données de travail. Il est alors nécessaire d'indiquer à R les valeurs prises par la variable explicative, ici `lwt`, et de les stocker dans un data frame *via* l'option `newdata=`. Voici un exemple où l'on estime les valeurs prédites par le modèle de régression pour des poids maternels allant de 35 à 120 kg, par pas de 5 kg :

```
> dp <- data.frame(lwt=seq(35, 120, by=5))
> bwtpred <- predict(r, newdata=dp)
> dp$bwt <- bwtpred
> head(dp)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
  lwt      bwt
1  35 2710.665
2  40 2759.385
3  45 2808.105
4  50 2856.825
5  55 2905.546
6  60 2954.266
```

Il est possible d'ajouter l'option `se=TRUE` pour obtenir l'erreur standard associée à chaque valeur prédite, et `interval="prediction"` pour un intervalle de confiance à 95 % pour la prédiction. En ce qui concerne les valeurs utilisées pour construire le modèle de régression, on peut se contenter de `interval="confidence"` pour construire des intervalles de confiance à 95 % pour les valeurs prédites (ajustées) sur les données actuelles.

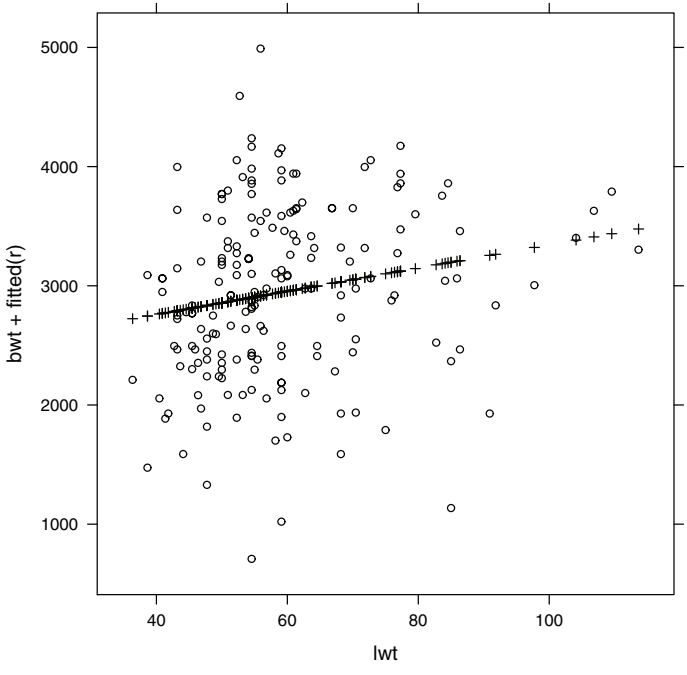


Figure 5.4. *Données observées (o) et prédites (+) par le modèle de régression linéaire*

5.2.4. *Diagnostic du modèle et analyse des résidus*

Le coefficient de détermination est un bon indicateur de la qualité du modèle, mais il ne renseigne pas sur les éventuelles observations qui pourraient avoir

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

influé sur les paramètres du modèle (coefficients de régression). Pour cela, R offre toute une batterie de mesures d'influence des observations sur le modèle *via* la commande `influence.measures()`. Ceci inclut les valeurs DFBETAS pour les deux coefficients de régression, les valeurs DFFIT, ou les distances de Cook [FOX 10, DRA 98]. Il est également possible de calculer les résidus standardisés ou studentisés d'un modèle de régression à l'aide des commandes `rstandard()` (ou `stdres()` dans le package `MASS`) et `rstudent()` (ou `studres()` dans le package `MASS`).

A l'image de `fitted()`, la commande `resid()` permet d'obtenir la valeur des résidus (écarts entre les valeurs observées et les prédictions) du modèle de régression :

```
> head(resid(r))
       85        86        87        88        89        91
-652.7211 -505.1352 -277.6798 -253.9671 -243.5380 -296.8329
> head(birthwt$bwt - fitted(r))
       85        86        87        88        89        91
-652.7211 -505.1352 -277.6798 -253.9671 -243.5380 -296.8329
```

Une représentation graphique utile pour vérifier la constance de la variance consiste à afficher la valeurs des résidus en fonction des valeurs observées ou prédites par le modèle (figure 5.5). Par exemple :

```
> xyplot(resid(r) ~ fitted(r), type=c("p","g"))
```

Pour tracer une droite horizontale coupant l'axe vertical en 0, il est possible d'ajouter l'option `abline=list(h=0, lty=2)`, par exemple. Le package `lattice` fournit la commande `rfs()` qui produit essentiellement le même résultat.

Les autres conditions d'application du modèle de régression linéaire (linéarité et normalité des erreurs) peuvent se vérifier à l'aide d'un simple diagramme de dispersion (avec courbe loess, par exemple) et d'un histogramme des résidus.

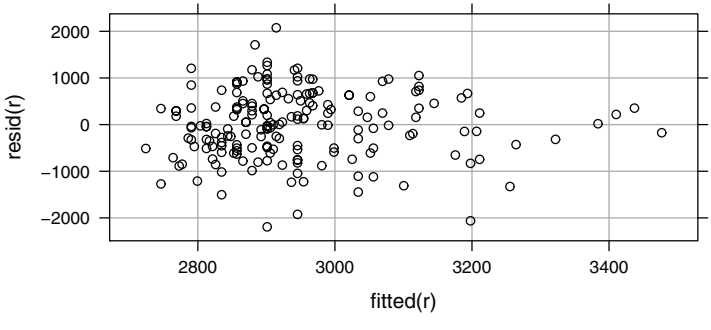


Figure 5.5. *Diagramme de fluctuation des résidus en fonction des valeurs ajustées*

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

5.2.5. *Lien avec l'ANOVA*

La commande `aov()` repose essentiellement sur `lm()` mais permet en plus de construire des termes d'erreur spécifiques pour les tests inférentiels, en particulier dans le cas où les observations ne sont pas indépendantes (cas des protocoles dits « à mesures répétées », par exemple). En revanche, on peut tout à fait étudier la relation entre une variable réponse numérique et un prédicteur catégoriel par une approche de régression : les résultats s'interpréteront sensiblement de la même manière. Sans entrer dans les détails du codage des contrastes et de leur manipulation sous R, on peut regarder les résultats d'une régression linéaire considérant les variables `bwt` et `race`, qui ont été étudiées au chapitre précédent (section 4.2.1) :

```
> lm(bwt ~ race, data=birthwt)

Call:
lm(formula = bwt ~ race, data = birthwt)

Coefficients:
(Intercept)    raceBlack    raceOther
     3102.7       -383.0       -297.4
```

La commande précédente renvoie trois coefficients : un terme pour l'ordonnée à l'origine, et deux coefficients de régression correspondant aux deux dernières modalités du facteur `race`. R utilisant par défaut des contrastes de type traitement, ces résultats sont à interpréter de la manière suivante : l'ordonnée à l'origine représente le poids moyen des bébés pour les mères dont l'ethnicité est `White`, tandis que les deux autres termes représentent les différences entre les poids moyens pour les groupes `Black` et `Other` et le poids moyen des bébés du groupe `White`. Il est assez simple de le vérifier numériquement :

```
> bwtmeans <- with(birthwt, tapply(bwt, race, mean))
> bwtmeans
   White    Black    Other
3102.719 2719.692 2805.284
> bwtmeans[2:3] - bwtmeans[1]
    Black     Other
-383.0264 -297.4352
```

On notera que l'on pourrait très bien supprimer le terme d'ordonnée à l'origine pour retrouver les trois moyennes de groupe. Pour cela, il est nécessaire de modifier légèrement la formule R :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> lm(bwt ~ race - 1, data=birthwt)

Call:
lm(formula = bwt ~ race - 1, data = birthwt)

Coefficients:
raceWhite  raceBlack  raceOther
     3103       2720       2805
```

La notation `bwt ~ race + 0` est également valable.

Indépendemment de l'interprétation des coefficients de régression, on peut également vérifier que le tableau d'analyse de variance pour la régression fournit bien les mêmes informations que celui de l'ANOVA à un facteur :

```
> anova(lm(bwt ~ race, data=birthwt))
Analysis of Variance Table

Response: bwt
           Df   Sum Sq Mean Sq F value   Pr(>F)
race        2  5015725 2507863  4.9125 0.008336 **
Residuals 186 94953931  510505
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

5.3. Régression linéaire multiple

En ce qui concerne la régression multiple, on se contentera d'indiquer que l'usage de `lm()` reste le même : on fait figurer à gauche de l'opérateur ~ la variable réponse, et à sa droite les variables explicatives que l'on souhaite inclure dans le modèle. Pour un modèle additif, on aura par exemple une formulation du type `lm(y ~ x1 + x2)` pour signifier à R de calculer les coefficients de régression (partiels) associés à `x1` et `x2`. La notation `x1 :x2` pour désigner une interaction reste valable, et dans le cas où l'une des deux variables est catégorielle on retrouve le modèle d'analyse de covariance dont on trouvera un exemple détaillé dans [VEN 02].

Il est parfois utile de centrer la variable numérique, en particulier dans le cas où elle entre dans un terme d'interaction avec une variable qualitative ; ceci peut être réalisé à l'aide de la commande `scale()`. Par exemple, la commande :

```
> lm(bwt ~ scale(lwt, scale=FALSE) + ftv, data=birthwt)
```

permet d'estimer l'effet du poids de la mère (`lwt`, centré sur sa moyenne) et de la fréquence des visites chez le gynécologue durant le premier trimestre de grossesse.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Comme l'on ne souhaite pas standardiser les observations par l'écart-type, il est nécessaire de préciser l'option `scale=FALSE` lorsque l'on utilise `scale()`.

Enfin, on peut tout à fait transformer les variables (réponse ou explicatives) lorsque l'on construit la formule R liant les variables. Par exemple, `y ~ log10(x1) + x2` permettrait de modéliser le logarithme (base 10) de `x1` plutôt que `x1` directement. On prendra garde au fait que l'inclusion de termes quadratiques, cubiques ou autres doit se faire de la manière suivante : `y ~ x1 + I(x1^2) + x2`, afin d'indiquer à R que la transformation `^2` s'applique aux valeurs de `x1` et ne doit pas être interprétée comme un terme de formule (en l'occurrence, la notation `^2` permet de construire automatique des termes d'interaction de second ordre).

5.4. Ce qu'il faut retenir

– Avant d'estimer un coefficient de corrélation ou de régression supposant une relation linéaire entre deux variables, on représentera graphiquement la distribution conjointe des deux variables à l'aide d'un histogramme (`histogram()` avec une option `type="smooth"`).

– Les commandes `cor()` et `cor.test()` permettent de calculer le coefficient de corrélation de Bravais-Pearson ou de Spearman (`method="spearman"`), et de tester l'hypothèse de nullité du paramètre.

– La commande `lm()` permet de construire un modèle de régression linéaire ; elle s'utilise avec `summary()` pour obtenir le tableau des coefficients de régression et `anova()` pour le tableau de variance associé.

5.5. Pour aller plus loin

Les packages `car` [FOX 10] et `rms` [HAR 01] fournissent un ensemble de commandes spécifiques à la régression linéaire et qui permettent entre autres de vérifier si les conditions d'application du modèle de régression linéaire simple ou multiple sont vérifiées. Les ouvrages de Julian Faraway et John Fox [FAR 14, FOX 10] offrent une discussion approfondie et de nombreuses illustrations de la modélisation des données à l'aide de modèles linéaires (et non linéaires).

5.6. Applications

1) Une étude a porté sur une mesure de malnutrition chez 25 patients âgés de 7 à 23 ans et souffrant de fibrose kystique. On disposait pour ces patients de différentes informations relatives aux caractéristiques antropométriques (taille, poids, etc.) et à la fonction pulmonaire [EVE 01]. Les données sont disponibles dans le fichier `cystic.dat`.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

a) Calculer le coefficient de corrélation linéaire entre les variables `PEmax` et `Weight`, ainsi que son intervalle de confiance à 95 %.

b) Afficher l'ensemble des données numériques sous forme de diagrammes de dispersion, soit 45 graphiques arrangés sous forme d'une « matrice de dispersion ».

c) Calculer l'ensemble des corrélations de Pearson et de Spearman entre les variables numériques.

d) Calculer la corrélation entre `PEmax` et `Weight`, en contrôlant l'âge (`Age`) (corrélation partielle). Représenter graphiquement la covariation entre `PEmax` et `Weight` en mettant en évidence les deux terciles les plus extrêmes pour la variable `Age`.

Les données étant disponibles dans un format texte où les valeurs (« champs ») sont séparées par des tabulations, on utilisera la commande `read.table()` pour les importer dans R. Comme la première ligne dans le fichier permet d'identifier les variables, on ajoutera l'option `header=TRUE` :

```
> cystic <- read.table("cystic.dat", header=TRUE)
> str(cystic)
> summary(cystic)
```

On voit d'emblée que le sexe des patients n'est pas codé sous la forme d'une variable qualitative mais d'un nombre (0/1). Bien que cela ne soit pas fondamentalement nécessaire dans cet exercice, il est toujours préférable de convertir les variables dans le bon format :

```
> cystic$Sex <- factor(cystic$Sex, labels=c("M","F"))
> table(cystic$Sex)
```

La corrélation linéaire entre `PEmax` et `Weight` est obtenue à l'aide de la commande `cor()`, qui par défaut calcule un coefficient de corrélation de Pearson :

```
> with(cystic, cor(PEmax, Weight))
```

La construction un peu étrange `with(cystic, ...)` permet de ne pas avoir à répéter le nom du data frame, `cystic`, pour désigner les variables d'intérêt. Autrement, on aurait écrit : `cor(cystic$PEmax, cystic$Weight)`. La commande `cor()` ne permet que l'estimation ponctuelle du paramètre. Pour obtenir l'intervalle de confiance associé, on utilisera directement la commande `cor.test()` qui répond en même temps au test d'hypothèse sur la nullité du coefficient dans la population :

```
> with(cystic, cor.test(PEmax, Weight))
```

Pour afficher l'ensemble des diagrammes de dispersion, on peut utiliser la commande de base `pairs()` ou bien la solution suivante :

```
> splom(cystic[,-c(1,3)], varname.cex=0.7, cex=.8)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Notons que nous avons exclus l'identifiant des sujets ainsi que la variable `Sex` qui figure dans la troisième colonne du tableau de données. Les deux autres options (`varname.cex` et `cex`) permettent de contrôler la taille des polices et des points.

Pour calculer les corrélations de toutes les paires de variables, on peut toujours utiliser la commande `cor()`. Par contre, il ne sera pas possible de tester ces corrélations avec la commande `cor.test()` ; il faudrait pour cela utiliser par exemple la commande `corr.test()` du package `psych` :

```
> round(cor(cystic[,-3]), 3)
```

Pour les corrélations de Spearman, il suffit d'ajouter l'option `method= "spearman"` comme ci-après :

```
> round(cor(cystic[,-3], method="spearman"), 3)
```

Enfin, pour estimer la corrélation partielle entre `PEmax`, `Weight` et `Age`, on utilisera la commande `pcor.test()` du package `ppcor` :

```
> library(ppcor)
> with(cystic, pcor.test(PEmax, Weight, Age))
```

La variable de contrôle est indiquée en troisième position. On pourra vérifier que la corrélation entre `PEmax` et `Weight` diminue fortement lorsque l'on tient compte de la corrélation entre ces deux variables et l'âge à l'aide du graphique suivant :

```
> cystic$Age.ter <- cut(cystic$Age,
                        breaks=quantile(cystic$Age,
                                        c(0,0.33,0.66,1)),
                        include.lowest=TRUE)
> cystic2 <- subset(cystic, as.numeric(Age.ter) %in% c(1,3))
> cystic2$Age.ter <- factor(cystic2$Age.ter)
> xyplot(PEmax ~ Weight, data=cystic2, groups=Age.ter,
         auto.key=list(corner=c(0,1)))
```

2) Dans l'étude Framingham, on dispose de donnée sur la pression artérielle systolique (`sbp`) et l'indice de masse corporelle (`bmi`) de 2 047 hommes et 2 643 femmes [DUP 09]. On s'intéresse à la relation entre ces deux variables (après transformation logarithmique) chez les hommes et chez les femmes séparément. Les données sont disponibles dans le fichier `Framingham.csv`.

a) Représenter graphiquement les variations entre pression artérielle et IMC (`bmi`) chez les hommes et chez les femmes.

b) Les coefficients de corrélation linéaire estimés chez les hommes et chez les femmes sont-ils significativement différents à 5 % ?

c) Estimer les paramètres du modèle de régression linéaire considérant la pression artérielle comme variable réponse et l'IMC comme variable explicative, pour

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

ces deux sous-échantillons. Donner un intervalle de confiance à 95 % pour l'estimé des pentes respectives.

Cette fois, les données ont été générées à partir d'un tableur (Excel ou autre) mais le séparateur de champ est la « , », d'où l'usage de `read.csv()` au lieu de `read.csv2()` puisque les options par défaut correspondent au cas à traiter :

```
> fram <- read.csv("data/Framingham.csv")
> head(fram)
> str(fram)
```

La variable `sex` est traitée comme une variable quantitative (1/2) et pour faciliter l'interprétation, nous la recodons d'emblée en variable qualitative :

```
> table(fram$sex)
> fram$sex <- factor(fram$sex, labels=c("M","F"))
```

Avant de répondre à la question concernant les variations entre pression artérielle et IMC, on peut vouloir vérifier la présence de valeurs manquantes. Cela ne gêne en rien l'estimation des moyennes et écarts-types, des paramètres de la droite de régression ou la représentation graphique des données, mais cela permet de connaître le nombre de « cas complets » sur les données d'intérêt :

```
> apply(fram, 2, function(x) sum(is.na(x)))
```

On utilise une commande `apply()` pour répéter une même opération pour chaque variable, cette opération consistant à compter (`sum`) le nombre de valeurs manquantes (`is.na(x)`). On constate que pour l'une des variables de notre modèle, l'IMC (`bmi`), neuf observations sont manquantes. D'où la distribution par sexe suivante :

```
> with(fram, table(sex[!is.na(bmi)]))
```

Pour représenter les données dans un diagramme de dispersion, on prendra garde au fait qu'en raison des « grands effectifs » par sous-groupe (plus de 2 000 observations), il risque d'y avoir beaucoup de points qui se chevauchent. Une possibilité est d'utiliser la semi-transparence et de réduire la taille des points :

```
> xyplot(sbp ~ bmi | sex, data=fram, type=c("p","g"), alpha=0.5,
         cex=0.7, pch=19)
```

Les corrélations entre les variables `sbp` et `bmi` chez les hommes et chez les femmes peuvent être obtenues à partir de la commande `cor()`, en restreignant bien sûr l'analyse à chacun des deux sous-échantillons :

```
> with(subset(fram, sex=="M"), cor(sbp, bmi, use="pair"))
> with(subset(fram, sex=="F"), cor(sbp, bmi, use="pair"))
```

On notera que l'on a rajouté l'option `use="pair"` (l'option complète se lit `"pairwise.complete.obs"` mais il est possible d'abréger les options lorsque cela

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

ne pose pas de problème d'ambiguïté) pour calculer les corrélations sur l'ensemble des données observées. Pour tester si ces deux coefficients estimés à partir des données peuvent être considérés comme significativement différents au seuil 5 %, il faut utiliser la commande `r.test()` du package `psych` :

```
> library(psych)
> r.test(n=2047, r12=0.23644, n2=2643, r34=0.37362)
```

Pour vérifier l'effet de la transformation logarithmique sur la distribution des variables `sbp` et `bmi`, on peut procéder de deux manières : soit créer un data frame contenant les valeurs de densité de chaque histogramme, associées au type de variable (`bmi`, `log(bmi)`, `sbp` et `log(sbp)`), soit construire séparément les quatre histogrammes et les combiner en une seule figure à l'aide de la commande `grid.arrange()` du package `gridExtra`. Voici ce qui est obtenu avec cette deuxième solution :

```
> library(gridExtra)
> p1 <- histogram(~ bmi, data=fram)
> p2 <- histogram(~ log(bmi), data=fram)
> p3 <- histogram(~ sbp, data=fram)
> p4 <- histogram(~ log(sbp), data=fram)
> grid.arrange(p1, p2, p3, p4)
```

Le modèle de régression dans chaque sous-groupe et les intervalles de confiance à 95 % associés aux pentes de régression peuvent être calculés comme suit :

```
> reg.resM <- lm(log(sbp) ~ log(bmi), data=fram, subset=sex=="M")
> reg.resF <- lm(log(sbp) ~ log(bmi), data=fram, subset=sex=="F")
> summary(reg.resM)   # Hommes
> confint(reg.resM)
> summary(reg.resF)   # Femmes
> confint(reg.resF)
```

Pour les intervalles de confiance, on utilise la commande `confint()`. On peut regrouper tous les résultats qui nous intéressent dans un même tableau. Par exemple :

```
> res <- data.frame(pente=c(coef(reg.resM)[2],
                            coef(reg.resF)[2]),
                    rbind(confint(reg.resM)[2,],
                          confint(reg.resF)[2,]))
> rownames(res) <- c("M","F")
> colnames(res)[2:3] <- c("2.5 %", "97.5 %")
> round(res, 3)
```

3) A partir des données sur les poids à la naissance [HOS 89], on cherche à étudier la relation entre le poids des bébés (traité en tant que variable numérique, `bwt`) et deux caractéristiques de la mère : son poids (`lwt`) et son origine ethnique (`race`).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

a) Représenter graphiquement la relation entre poids des bébés et poids des mères, en fonction de l'ethnicité des mères.

b) Estimer les paramètres de la régression linéaire en considérant les poids des bébés comme variable réponse et les poids des mères centrés sur leur moyenne comme variable explicative. La pente estimée est-elle significative au seuil usuel de 5 % ?

c) Estimer les paramètres de la régression linéaire où cette fois la variable explicative est l'ethnicité des mères, la variable réponse restant le poids des bébés. Comparer la significativité du modèle dans son ensemble avec les résultats obtenus à partir d'une ANOVA à un facteur (ethnicité).

d) Quelle est le poids prédit pour un bébé dont la mère pèse 60 kg ? Donner un intervalle de confiance à 95 % pour une prédiction ponctuelle en moyenne.

e) Refaire l'analyse de régression décrite en (c) après avoir modifié la manière dont R génère les contrastes pour les variables qualitatives en tapant ceci à l'invite de commande R :

```
> options(contrasts=c("contr.sum", "contr.poly"))
```

Comparer avec les résultats précédents.

Les données peuvent être importées et recodées exactement comme dans le chapitre 1 (section 1.5.2).

On peut représenter la relation entre poids des bébés et poids des mères à l'aide d'un simple diagramme de dispersion. Ici, on a choisi de stratifier sur l'ethnicité des mères et d'afficher trois graphiques séparés :

```
> xyplot(bwt ~ lwt | race, data=birthwt, layout=c(3,1),
         type=c("p","g"), aspect=0.8)
```

L'option `aspect=0.8` permet de modifier le rapport largeur/hauteur des graphiques. L'option `layout=c(3,1)` permet d'afficher les trois graphiques côte à côte (une ligne, trois colonnes). En modifiant légèrement la commande précédente, il serait tout à fait possible d'afficher toutes les observations dans le même graphique mais en utilisant des symboles ou couleurs différents selon l'origine ethnique :

```
> xyplot(bwt ~ lwt, data=birthwt, groups=race)
```

Les paramètres du modèle de régression linéaire sont estimés à partir de la commande `lm()`, sachant que la commande `scale()` permet de centrer et/ou réduire une série de mesures. Ici, on souhaite uniquement centrer les poids des mères sur leur moyenne, pas les standardiser par unités d'écart-type :

```
> reg.res <- lm(bwt ~ scale(lwt, scale=FALSE), data=birthwt)
> summary(reg.res)
```

Le test t évaluant la nullité de l'hypothèse nulle pour la pente est significatif si l'on considère un risque de première espèce de 5 % ($p = 0,011$).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Si l'on considère l'ethnicité comme variable explicative, le modèle de régression est évalué de la même manière :

```
> reg.res2 <- lm(bwt ~ race, data=birthwt)
> summary(reg.res2)
```

On suppose que la variable explicative est effectivement recodée sous forme de facteur comme dans le chapitre 1. L'ANOVA nous donnerait quant à elle :

```
> aov.res <- aov(bwt ~ race, data=birthwt)
> summary(aov.res)
```

En fait, il est tout à fait possible d'obtenir le tableau de régression précédent en utilisant `summary.lm()` au lieu de `summary()`, comme on peut le vérifier dans la sortie suivante :

```
> summary.lm(aov.res)
```

Réciproquement, on peut tout à fait afficher un tableau d'ANOVA correspondant au modèle de régression précédent :

```
> anova(reg.res2)
```

On peut vérifier que la statistique F est identique dans les deux cas (elle vaut 4,913, pour 2 et 186 degrés de liberté).

Les contrastes utilisés dans le modèle de régression (`reg.res2`) sont appelés contrastes de traitement et ils permettent de tester la différence entre les scores moyens (ici, l'âge) de deux catégories d'une variable qualitative (ici, l'ethnicité), l'une des deux catégories servant de catégorie dite « de référence ». Avec R, la catégorie de référence est toujours le premier niveau du facteur, en suivant l'ordre lexicographique, soit dans le cas présent le niveau `White`. Les coefficients de régression représentent alors la différence de poids moyen entre les bébés des mères de la catégorie `Black` *versus* `White` (-383.03), et `Other` *versus* `White` (-297.44). On peut le vérifier en calculant manuellement ces différences de moyennes :

```
> grp.means <- with(birthwt, tapply(bwt, race, mean))
> grp.means[2:3] - grp.means[1] # modèle de régression reg.res2
```

Pour prédire le poids d'un individu dont la mère pèse 60 kg, on utilisera la commande `predict()`, sachant que cette même commande permet également de faire une estimation par intervalle (en moyenne ou pour une observation future) grâce à l'option `interval=`.

```
> m <- lm(bwt ~ lwt, data=birthwt)
> d <- data.frame(lwt=60)
> predict(m, newdata=d, interval="confidence")
```

Si l'on ne précise aucune valeur pour l'option `newdata=`, R produira une estimation de la valeur prédite pour chaque unité statistique avec les valeurs observées pour le ou les cofacteurs présents dans le modèle.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 6

Mesures d'association en épidémiologie et régression logistique

Dans ce chapitre on revisite les mesures d'association classiquement retrouvées dans les études d'épidémiologie ou de recherche clinique et en partie discutées au chapitre 2, en particulier les mesures de risque telles que l'odds-ratio dans les études prospectives ou les mesures de sensibilité/spécificité d'un test diagnostique. Le modèle de régression logistique simple où l'on considère une seule variable explicative, numérique ou qualitative, est présenté en détail : estimation des paramètres du modèle, dans le cas des données individuelles ou groupées, prédiction ponctuelle et par intervalle. Enfin, la construction d'une courbe ROC complète ce chapitre portant sur le cas où la variable réponse est une variable binaire.

6.1. Tableaux de contingence et mesures d'association

6.1.1. *Tableaux de contingence*

Comme on l'a déjà vu au chapitre 3, les commandes `table()` et `xtabs()` permettent de construire des tableaux à deux dimensions, voire plus (c'est le cas de `xtabs()`, mais également `ftable()`). On rappelle le principe de la construction d'un tableau à partir d'une matrice de décompte des effectifs, ici avec les variables `low` et `race` des données `birthtw` dont les effectifs sont saisis manuellement :

```
> tab <- matrix(c(73, 23, 15, 11, 42, 25), nrow=2)
> rownames(tab) <- c(">= 2.5 kg", "< 2.5 kg")
> colnames(tab) <- c("White", "Black", "Other")
> tab
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
          White Black Other
>= 2.5 kg    73    15    42
< 2.5 kg     23    11    25
```

Lorsque l'on saisit les effectifs par colonnes, comme dans l'exemple précédent, il suffit d'indiquer à R comment arranger le tableau en termes de nombre de lignes ou de colonnes. Si les effectifs sont saisis par ligne (73, 15, 42 puis 23, 11, 25), il faut ajouter l'option `by.row=TRUE`. Bien évidemment, on a les mêmes résultats en utilisant directement le jeu de données :

```
> with(birthwt, table(low, race))
     race
low   White Black Other
  No     73    15    42
  Yes    23    11    25
```

ce qui est équivalent à `xtabs(~ low + race, data=birthwt)`.

6.1.2. *Mesures et tests d'association*

Les mesures et tests d'association pour deux variables qualitatives ont été présentés en sections 3.3 et 3.4. Pour l'odds-ratio et le risque relatif, on prendra garde à l'orientation du tableau (lignes et colonnes) et à l'ordre des modalités des variables binaires sur chaque dimension (généralement, les événements « positifs » figureront en première ligne et première colonne).

6.1.3. *Odds-ratio et stratification*

Lorsque l'on s'intéresse au croisement de deux variables binaires selon k niveaux ou modalités d'une troisième variable qualitative, considérée comme facteur de stratification, il est possible de calculer k odds-ratio, et s'ils s'avèrent qu'ils sont homogènes on peut combiner les estimations par strates en un odds-ratio commun. Le test permettant de vérifier l'homogénéité entre strates est le test de Mantel-Haenszel et s'obtient avec la commande `mantelhaen.test()` sous R. Les données doivent être arrangées sous la forme d'un tableau à trois dimensions, la dernière dimension représentant les niveaux du facteur de stratification. Il y a deux approches pour construire un tel tableau : à partir de matrices à deux dimensions (`matrix()`) assemblées ensemble pour former un `array` (une généralisation du type `matrix`), ou à partir de la commande `xtabs()`, plus simple d'utilisation en règle générale.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Voici comment il est possible de procéder avec une série de matrices résumant le tableau d'effectifs obtenu en croisant les modalités des variables `ui` (douleurs intra-utérines) et `low` (statut pondéral à la naissance), pour chaque niveau de la variable `race` (ethnicité) :

```
> t1 <- with(subset(birthwt, race == "White"), table(ui, low))
> t2 <- with(subset(birthwt, race == "Black"), table(ui, low))
> t3 <- with(subset(birthwt, race == "Other"), table(ui, low))
> tab <- array(c(t1, t2, t3), dim=c(2,2,3),
+              dimnames=list(c("ui=0","ui=1"), c(">= 2.5 kg",
                                                 "< 2.5 kg"),
+                            levels(birthwt$race)))
> tab
, , White

     >= 2.5 kg < 2.5 kg
ui=0        65       18
ui=1         8        5

, , Black

     >= 2.5 kg < 2.5 kg
ui=0        14        9
ui=1         1        2

, , Other

     >= 2.5 kg < 2.5 kg
ui=0        37       18
ui=1         5        7
```

Notons que plutôt que d'utiliser l'option `dimnames=` dans la commande `array()`, il est également possible de nommer les lignes et colonnes du tableaux *a posteriori* à l'aide de `dimnames()`, comme illustré ci-après :

```
> dimnames(tab) <- list(c("ui=0","ui=1"),
                        c(">= 2.5 kg", "< 2.5 kg"),
                        levels(birthwt$race))
```

Voici une autre possibilité, reposant sur la commande `xtabs()`, pour laquelle on indique grâce à une formule la liste des variables à considérer, avec la variable de stratification en dernière position. On pourra vérifier que la commande suivante fournit bien le même tableau que l'illustration précédente :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> xtabs(~ ui + low + race, data=birthwt)
, , race = White

     low
ui    No Yes
  No  65  18
  Yes  8   5

, , race = Black

     low
ui    No Yes
  No  14   9
  Yes  1   2

, , race = Other

     low
ui    No Yes
  No  37  18
  Yes  5   7
```

Quel que soit le tableau que l'on utilise, le résultat du test de Mantel-Haenszel indique que l'association entre le statut pondéral des bébés et la présence de douleurs intra-utérines n'est pas indépendante de l'ethnicité des mères, suggérant qu'il n'est pas possible de dériver une estimation de l'odds-ratio commun :

```
> mantelhaen.test(tab)

Mantel-Haenszel chi-squared test with continuity correction

data:  tab
Mantel-Haenszel X-squared = 4.2339, df = 1, p-value = 0.03962
alternative hypothesis: true common odds ratio is not equal to 1
95 percent confidence interval:
 1.133437 6.015297
sample estimates:
common odds ratio
         2.611122
```

R propose également le test de Woolf pour tester l'hétérogénité entre strates grâce à la commande `woolf_test()` du package `vcd` :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> library(vcd)
> woolf_test(tab)

Woolf-test on Homogeneity of Odds Ratios (no 3-Way
assoc.)

data:  tab
X-squared = 0.093904, df = 2, p-value = 0.9541
```

6.2. Etudes diagnostiques

Dans les situations où l'on s'intéresse à la capacité prédictive ou discriminante d'un test de screening ou de dépistage (malade/pas malade), connaissant le diagnostic des patients (*via* un jugement d'experts ou un test déjà existant servant de « gold standard »), la plupart des mesures d'intérêt (sensibilité, spécificité, valeurs prédictives positive et négative, odds-ratio pré ou post-test, etc.) peuvent se calculer à partir d'un tableau de contingence.

6.2.1. *Sensibilité et spécificité d'un test diagnostique*

Comme cela a été discuté en section 3.4, afin de calculer et d'interpréter correctement la plupart des mesures d'association sur un tableau 2×2, il convient de prendre garde à l'orientation du tableau (lignes et colonnes) et aux modalités reportées sur les lignes et les colonnes. Considérons les données suivantes :

```
> dat <- as.table(matrix(c(670,202,74,640), nrow=2, byrow=TRUE))
> colnames(dat) <- c("Dis+","Dis-")
> rownames(dat) <- c("Test+","Test-")
> dat
      Dis+ Dis-
Test+  670  202
Test-   74  640
```

Il s'agit des résultats d'une étude de validation d'un nouveau test diagnostique réalisée chez 1 586 patients [SCO 08], tirés de l'aide en ligne du package `epiR`. Parmi les 744 patients malades, 670 ont été identifiés comme tels par ce nouveau test. La commande `as.table()` n'est pas indispensable pour les calculs qui suivent. On notera simplement que les lignes et colonnes du tableau sont nommées ce qui permet d'adresser ses différentes cellules à l'aide des modalités des deux variables, plutôt que par les numéros de ligne ou de colonnes.

La sensibilité et la spécificité peuvent se calculer par simple indexation du tableau, de la manière suivante :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> sens <- dat["Test+","Dis+"]/sum(dat[,"Dis+"])
> spec <- dat["Test-","Dis-"]/sum(dat[,"Dis-"])
> round(c(sens=sens, spec=spec), 3)
 sens  spec
0.901 0.760
```

On peut calculer des intervalles de confiance approximatifs à partir de la loi normale, en utilisant la formule $p \pm 1,96\sqrt{p(1-p)/n}$, où p est la proportion étudiée. Les bornes de l'intervalle de confiance seraient ainsi estimées comme suit :

```
> sens + c(-1, 1) * 1.96 * sqrt(sens * (1-sens)/sum(dat))
[1] 0.8858082 0.9152670
```

6.2.2. *Valeurs prédictives positive et négative*

Le calcul de la valeur prédictive positive (VPP) et négative (VPN) peut s'effectuer sur le même principe :

```
> vpp <- dat["Test+","Dis+"]/sum(dat["Test+",])
> vpn <- dat["Test-","Dis-"]/sum(dat["Test-",])
> round(c(VPP=vpp, VPN=vpn), 3)
  VPP   VPN
0.768 0.896
```

Comme il s'agit de simples proportions, on peut tout à fait tester l'hypothèse nulle $H_0 : p = \frac{1}{2}$ à l'aide d'un test binomial. Pour la VPP, par exemple, on a les résultats suivants :

```
> binom.test(dat["Test+","Dis+"], sum(dat["Test+",]))

Exact binomial test

data:  dat["Test+", "Dis+"] and sum(dat["Test+", ])
number of successes=670, number of trials=872, p-value<2.2e-16
alt hypothesis: true probability of success is not equal to 0.5
95 percent confidence interval:
 0.7388926 0.7959784
sample estimates:
probability of success
             0.7683486
```

On peut également utiliser ce type de distribution binomiale pour calculer des intervalles de confiance, comme illustré dans les paragraphes suivants.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

6.2.3. *Tableau de synthèse des propriétés diagnostiques d'un test*

La commande `epi.tests()` du package `epiR` fournit l'ensemble des indicateurs précédents dans un seul et même tableau, avec des intervalles de confiance reposant sur la distribution binomiale :

```
> epi.tests(dat, conf.level = 0.95)
          Disease +    Disease -      Total
Test +          670          202        872
Test -           74          640        714
Total           744          842       1586

Point estimates and 95 % CIs:
--------------------------------------------------------
Apparent prevalence                    0.55 (0.52, 0.57)
True prevalence                        0.47 (0.44, 0.49)
Sensitivity                            0.90 (0.88, 0.92)
Specificity                            0.76 (0.73, 0.79)
Positive predictive value              0.77 (0.74, 0.80)
Negative predictive value              0.90 (0.87, 0.92)
Positive likelihood ratio              3.75 (3.32, 4.24)
Negative likelihood ratio              0.13 (0.11, 0.16)
--------------------------------------------------------
```

En fait, cette commande calcule d'autres quantités d'intérêt qui ne sont pas affichées mais que l'on peut obtenir assez simplement lorsque l'on stocke le résultat de l'appel à la commande `epi.tests()` dans une variable auxiliaire et que l'on ajoute l'option `verbose=TRUE`. Pour cela, il est nécessaire d'assigner le résultat produit par l'instruction précédente à une variable appelée, par exemple, `diagstats` :

```
> diagstats <- epi.tests(dat, conf.level = 0.95)
```

A partir de maintenant, on peut accéder aux autres statistiques calculées sur les données `dat` en suffixant notre variable auxiliaire par le nom des statistiques décrites dans l'aide en ligne (`help(epi.tests)`). Les résultats exposés ci-après correspondent à l'OR diagnostique et au nombre de patients à tester pour obtenir un résultat positif correct :

```
> diagstats$rval$diag.or
       est    lower    upper
1 28.68611 21.51819 38.24174
> diagstats$rval$nnd
       est  lower    upper
1 1.513701 1.4091 1.648743
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

6.3. Régression logistique

6.3.1. *Estimation des paramètres du modèle*

Dans le chapitre précédent, on s'est intéressé à la relation entre le poids des bébés à la naissance, traité comme une variable numérique, et le poids des mères. On peut s'intéresser au même type de relation, mais en considérant cette fois le statut pondéral à la naissance traité comme variable binaire (`low`). En fait, la variable `low` dans les données `birthwt` est simplement une variable indicatrice construite à partir de la variable `bwt` (`low` = 1 si `bwt` < 2 500, 0 sinon).

Pour construire un modèle de régression logistique, on utilisera la commande `glm()` qui fonctionne exactement de la même manière que la commande `lm()` pour la régression linéaire, à ceci près qu'il faut indiquer à R quel type de relation lie la variable réponse binaire à la combinaison linéaire du ou des prédicteurs d'intérêt. Pour la régression logistique, on considérera une transformation logit plutôt que la fonction identité utilisée en régression linéaire :

```
> glm(low ~ lwt, data=birthwt, family=binomial("logit"))

Coefficients:
(Intercept)          lwt
    0.99831     -0.03093

Degrees of Freedom: 188 Total (i.e. Null);  187 Residual
Null Deviance:     234.7
Residual Deviance: 228.7  AIC: 232.7
```

La fonction de lien par défaut étant le logit, on peut simplifier l'option `family=` en indiquant simplement `binomial`. Comme dans le cas de l'ANOVA ou de la régression, la commande `glm()` en elle-même ne fournit qu'une partie des informations et il est nécessaire de la coupler avec `summary()` pour obtenir le tableau des coefficients de régression :

```
> m <- glm(low ~ lwt, data=birthwt, family=binomial("logit"))
> summary(m)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-1.0951  -0.9022  -0.8018   1.3609   1.9821

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
(Intercept)  0.99831    0.78529   1.271   0.2036
lwt         -0.03093    0.01357  -2.279   0.0227 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 234.67  on 188  degrees of freedom
Residual deviance: 228.69  on 187  degrees of freedom
AIC: 232.69

Number of Fisher Scoring iterations: 4
```

L'interprétation du coefficient de régression est relativement simple : -0,03 reflète la variation du log-odds pour la variable réponse binaire (`low`) lorsque la variable `lwt` varie d'une unité.

On procèdera de la même manière lorsque la variable explicative est une variable qualitative. Par exemple, si l'on souhaite modéliser la relation entre statut pondéral à la naissance et les antécédents de douleurs intra-utérines, on aura le modèle `low ~ ui` :

```
> table(birthwt$ui)

 No Yes
161  28
> levels(birthwt$ui)
[1] "No"  "Yes"
```

On prendra garde au fait que le premier niveau de la variable `ui` est la réponse `No` (pas de douleurs), mais si ce n'était pas le cas on pourrait interchanger les deux niveaux pour que l'interprétation des coefficients de régression se fasse par rapport à cette modalité. Pour cela, il suffirait de modifier l'ordre des niveaux à l'aide de la commande `relevel()`, qui permet de changer le niveau de référence d'une variable qualitative :

```
> birthwt$ui <- relevel(birthwt$ui, ref="No")
```

Les résultats de ce second modèle sont indiqués ci-après :

```
> m2 <- glm(low ~ ui, data=birthwt, family=binomial)
> summary(m2)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
Call:
glm(formula = low ~ ui, family = binomial, data = birthwt)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-1.1774  -0.8097  -0.8097   1.1774   1.5967

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  -0.9469     0.1756  -5.392 6.97e-08 ***
uiYes         0.9469     0.4168   2.272   0.0231 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 234.67  on 188  degrees of freedom
Residual deviance: 229.60  on 187  degrees of freedom
AIC: 233.6

Number of Fisher Scoring iterations: 4
```

L'odds-ratio s'obtient simplement en prenant l'exponentielle du coefficient de régression (pente), mais par souci de simplicité on peut appliquer cette transformation à l'ensemble des coefficients du modèle (on prendra garde que dans le cas de l'ordonnée à l'origine (« Intercept »), on aura alors un odds simple) :

```
> exp(coef(m2))
(Intercept)       uiYes
   0.387931    2.577778
```

Pour afficher l'odds-ratio pour la variable `ui`, il suffit de remplacer l'expression précédente par `exp(coef(m2)["uiYes"])`. Avec la commande `oddsratio()` du package `vcd`, on peut vérifier que l'on retrouve bien la même valeur pour l'odds-ratio :

```
> oddsratio(xtabs(~ ui + low, data=birthwt), log=FALSE)
[1] 2.577778
```

On peut toujours utiliser la commande `confint()` sur les coefficients du modèle pour obtenir leurs intervalles de confiance à 95 %, par exemple :

```
> confint(m2)
Waiting for profiling to be done...
                 2.5 %     97.5 %
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
(Intercept) -1.3007911 -0.6105528
uiYes        0.1250472  1.7717715
```

A l'image du tableau de variance dérivé à partir d'une analyse de régression linéaire, il est possible de fournir une table de déviance pour le modèle de régression logistique. Cela se fait toujours avec la commande `anova()`, mais en spécifiant le type de test qui doit être réalisé à l'aide de l'option `test=`, dans ce cas un test du χ^2 plutôt qu'un test F :

```
> anova(m2, test="Chisq")
Analysis of Deviance Table

Model: binomial, link: logit

Response: low

Terms added sequentially (first to last)

     Df Deviance Resid. Df Resid. Dev Pr(>Chi)
NULL                   188     234.67
ui    1   5.0761       187     229.60  0.02426 *
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

6.3.2. *Prédiction par intervalles*

On utilisera toujours la commande `predict()` pour estimer les valeurs prédites (ou ajustées) par le modèle pour les observations actuelles ou pour des observations futures. Dans ce dernier cas, il ne faut pas oublier de fournir un data frame avec les valeurs de la variable explicative pour lesquelles on souhaite calculer les prédictions.

Toutefois, à la différence de la régression linéaire, il est nécessaire d'indiquer quel type de prédictions on souhaite calculer : des valeurs exprimées sur l'échelle du log-odds (`type="link"`) ou en termes de probabilités de réponse (`type="response"`) :

```
> head(predict(m2, type="link"), n=4)
           85            86            87            88
 2.220446e-16 -9.469277e-01 -9.469277e-01  2.220446e-16
> head(predict(m2, type="response"), n=4)
       85        86        87        88
0.5000000 0.2795031 0.2795031 0.5000000
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

On peut également utiliser la commande `fitted()` pour obtenir les valeurs ajustées calculées sur l'échantillon. Dans le cas du premier modèle avec le poids des mères, on utiliserait ainsi :

```
> xyplot(fitted(m) ~ lwt, data=birthwt)
```

Dans le graphique produit par cette commande (figure 6.1), chaque point représente une prédiction individuelle.

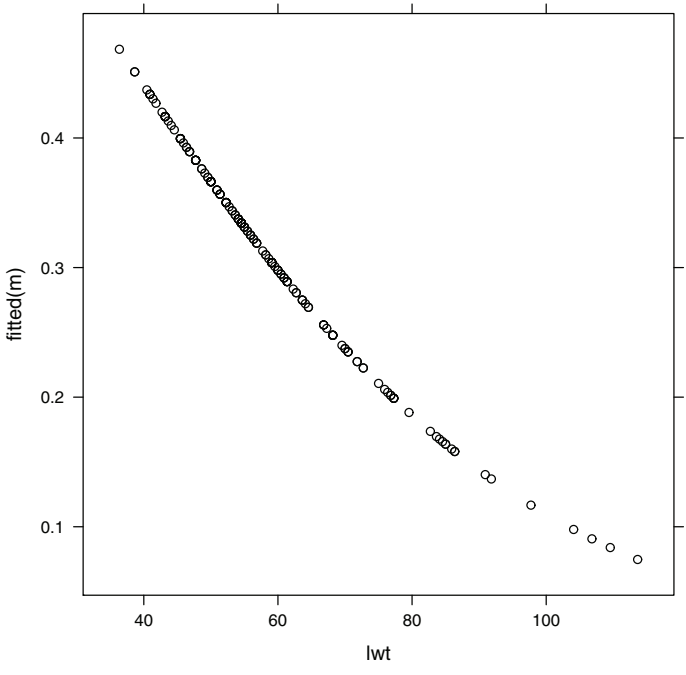


Figure 6.1. *Valeurs ajustées pour le modèle de régression logistique*

Pour la prédiction par intervalles, on utilisera `predict()`. Cependant, à la différence du cas de la régression linéaire, il n'y a pas d'option `interval=` mais on peut calculer des intervalles asymptotiques très facilement à partir du moment où l'on dispose de l'erreur-type d'estimation, obtenue grâce à l'option `se.fit=`. Soit :

```
> pp <- predict(m2, type="link", se.fit=TRUE)
> lwr <- pp$fit - qnorm(0.975) * pp$se.fit
> upr <- pp$fit + qnorm(0.975) * pp$se.fit
> ppd <- data.frame(y=pp$fit, lwr=lwr, upr=upr)
> head(ppd)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
               y        lwr        upr
85  2.220446e-16 -0.7407968  0.7407968
86 -9.469277e-01 -1.2911393 -0.6027161
87 -9.469277e-01 -1.2911393 -0.6027161
88  2.220446e-16 -0.7407968  0.7407968
89  2.220446e-16 -0.7407968  0.7407968
91 -9.469277e-01 -1.2911393 -0.6027161
```

Les intervalles de confiance précédents portent sur les prédictions exprimées en unités logit. Pour repasser aux probabilités, on peut utiliser la commande `plogis()` et l'appliquer à chacune des colonnes du tableau nommé `ppd` :

```
> head(sapply(ppd, plogis))
             y       lwr       upr
[1,] 0.5000000 0.3228299 0.6771701
[2,] 0.2795031 0.2156600 0.3537225
[3,] 0.2795031 0.2156600 0.3537225
[4,] 0.5000000 0.3228299 0.6771701
[5,] 0.5000000 0.3228299 0.6771701
[6,] 0.2795031 0.2156600 0.3537225
```

On pourra vérifier que l'on obtient bien les mêmes résultats en faisant le calcul manuellement, comme ci-après dans le cas de la borne supérieure de l'IC à 95 % :

```
> head(exp(pp$fit+1.96*pp$se.fit)/(1+exp(pp$fit+1.96*pp$se.fit)))
      85       86       87       88       89       91
0.677173 0.353724 0.353724 0.677173 0.677173 0.353724
```

6.3.3. *Cas des données groupées*

Dans l'exemple précédent, on disposait des données individuelles, c'est-à-dire des valeurs de `low` et `lwt` pour chaque unité statistique. C'est souvent le cas lorsque l'une des variables explicatives est de type numérique. Mais dans le cas de variables purement qualitatives, il arrive souvent que les données disponibles soient présentées sous la forme d'un tableau de contingence à deux dimensions, dans le cas le plus simple un tableau à deux dimensions croisant les deux modalités de la variable réponse (malade/pas malade) et celles de la variable explicative, typiquement un tableau produit par `xtabs()`, comme par exemple :

```
> tab <- xtabs(~ ui + low, data=birthwt)
> tab
     low
ui     No Yes
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
  No  116  45
  Yes  14  14
```

Une représentation alternative pour ces données consisterait à indiquer pour chaque modalité de la variable explicative le nombre d'événements positifs (ici, `low=1`) et le nombre total d'événements (ou d'observations), soit :

```
> tab2 <- data.frame(ui=c("No","Yes"), n=c(45,14),
                     tot=c(45+116, 14+14))
> tab2
   ui  n tot
1  No 45 161
2 Yes 14  28
```

Les deux formulations suivantes sont alors équivalentes :

```
> glm(n/tot ~ ui, data=tab2, family=binomial, weights=tot)
> glm(cbind(n, tot-n) ~ ui, data=tab2, family=binomial)
```

6.4. Courbe ROC

Il existe de nombreux packages R permettant de construire une courbe ROC. Voici une illustration avec le package `ROCR` qui a l'avantage de ne pas avoir beaucoup de contraintes de formatage des données : à partir d'un modèle de régression, dans l'exemple suivant le statut pondéral en fonction du poids et de l'éthnicité des mères, il suffit d'utiliser les deux commandes `prediction()` et `performance()` pour formater les données et calculer les statistiques nécessaires pour l'analyse ROC. Une commande `plot()` se charge d'afficher les résultats de manière graphique, en fonction des statistiques choisies.

Dans le cas le plus classique, on s'intéresse au compromis entre la sensibilité (vrais positifs, ou `tpr`) et le complémentaire de la spécificité (faux positifs, ou `fpr`) ce qui se traduit par les instructions suivantes :

```
> m3 <- glm(low ~ lwt + race, data=birthwt, family=binomial)
> pred <- prediction(predict(m3, type="response"), birthwt$low)
> perf <- performance(pred, "tpr","fpr")
> plot(perf, ylab="Sensibilité", xlab="1-Spécificité")
```

Il est nécessaire de fournir à la fois les prédictions du modèle (de type `"response"`) et le diagnostique réel (ici, `low`). La dernière commande permet de construire la courbe ROC (figure 6.2). Une courbe s'écartant de la première bissectrice dans le quadrant supérieur gauche indique que les prédictions observées s'écartent de prédictions réalisées au hasard.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

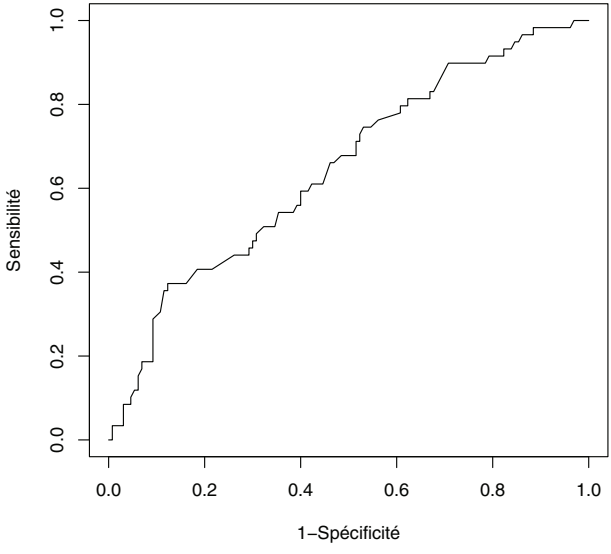


Figure 6.2. *Courbe ROC pour le modèle de régression logistique*

La commande `roc.from.table()` du package `epicalc` permet de travailler directement à partir de données tabulées (cas du tableau croisant les résultats d'un test diagnostique avec un « gold standard », par exemple). Dans le cas d'une régression logistique, il suffit de donner la variable contenant le résultat du modèle, dans notre cas `m3`. Pour un tableau composé de données individuelles, c'est la commande `lroc()` qu'il faudra utiliser. Le package `pROC` (http://web.expasy.org/pROC/) fournit essentiellement le même genre de fonctionnalités, avec la possibilité d'afficher des courbes ROC lissées et de comparer différentes courbes ROC.

6.5. Ce qu'il faut retenir

– En plus de la commande `oddsratio()`, les commandes `mantelhaen.test()` et `woolf_test()` sont utiles pour évaluer l'indépendance conditionnelle des associations entre deux variables et l'homégénéité des odds-ratio selon les niveaux définis par une troisième variable qualitative.

– La commande `epi.tests()` du package `epiR` fournit l'essentiel des mesures permettant d'évaluer la qualité d'un test diagnostique.

– La commande `glm()` permet de construire un modèle de régression logistique ; elle s'utilise avec `summary()` pour obtenir le tableau des coefficients de régression et `anova()` pour le tableau de variance associé.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

6.6. Applications

1) Le tableau 6.1 fournit les résultats d'une étude de cohorte visant à déterminer, entre autres, l'intérêt d'utiliser comme outil de screening une mesure de test d'effort physique (EST), pour lequel un résultat de type réussite/échec peut être dérivé, lors du diagnostic d'une maladie coronarienne (CAD) [PEP 80].

		CAD		
		Non-malade	Malade	Total
EST	Négatif	327	208	535
	Positif	115	815	930
	Total	442	1 023	1 465

Tableau 6.1. *Test de screening pour l'effort physique*

On fera l'hypothèse qu'il n'y a pas de biais de vérification.

a) A partir de cette matrice de confusion, indiquer les valeurs suivantes (avec intervalles de confiance à 95 %) : sensibilité et spécificité, valeur prédictive positive et négative.

b) Quelle est la valeur de l'aire sous la courbe pour les données reportées ?

La création du tableau de données peut se faire comme indiqué ci-après, à partir d'un tableau créé à l'aide de la commande `matrix()` :

```
> tab <- as.table(matrix(c(815,115,208,327), nrow=2, byrow=TRUE,
                         dimnames=list(EST=c("+","-"),
                                       CAD=c("+","-"))))
> tab
```

On notera que le tableau a été légèrement réorganisé de manière à correspondre aux notations habituelles, avec les événements « positifs » sur la première ligne (typiquement, l'exposition) et la première colonne (typiquement, la maladie) du tableau.

Il serait tout à fait possible de calculer en très peu d'opérations arithmétiques avec R toutes les quantités demandées. Cependant, le package `epiR` contient toutes les commandes nécessaires pour répondre aux questions épidémiologiques sur les tableaux 2x2. Ainsi, la commande `epi.tests()` fournit les valeurs de prévalence, sensibilité/spécificité, et valeurs prédictives positive et négative :

```
> library(epiR)
> epi.tests(tab)
```

Concernant l'aire sous la courbe, on peut également utiliser une commande externe, `roc.from.table()`, dans le package `epicalc` :

```
> library(epicalc)
> roc.from.table(tab, graph=FALSE)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

2) Le tableau 6.2 résume la proportion d'infarctus du myocarde observée chez des hommes âgés de 40 à 59 ans et pour lesquels on a relevé le niveau de tension artérielle et le taux de cholestérol, considérés sous forme de classes ordonnées [EVE 01].

	Cholestérol (mg/100 ml)						
TA	< 200	$200 - 209$	$210 - 219$	$220 - 244$	$245 - 259$	$260 - 284$	> 284
< 117	2/53	0/21	0/15	0/20	0/14	1/22	0/11
$117 - 126$	0/66	2/27	1/25	8/69	0/24	5/22	1/19
$127 - 136$	2/59	0/34	2/21	2/83	0/33	2/26	4/28
$137 - 146$	1/65	0/19	0/26	6/81	3/23	2/34	4/23
$147 - 156$	2/37	0/16	0/6	3/29	2/19	4/16	1/16
$157 - 166$	1/13	0/10	0/11	1/15	0/11	2/13	4/12
$167 - 186$	3/21	0/5	0/11	2/27	2/5	6/16	3/14
> 186	1/5	0/1	3/6	1/10	1/7	1/7	1/7

Tableau 6.2. *Infarctus et tension artérielle*

Les données sont disponibles dans le fichier `hdis.dat` sous forme d'un tableau comprenant quatre colonnes indiquant, respectivement, la pression artérielle (huit catégories, notées 1 à 8), le taux de cholestérol (sept catégories, notées 1 à 7), le nombre d'infarctus et le nombre total d'individus. On s'intéresse à l'association entre la pression artérielle et la probabilité d'avoir un infarctus du myocarde.

a) Calculer les proportions d'infarctus pour chaque niveau de pression artérielle et les représenter dans un tableau et sous forme graphique.

b) Exprimer les proportions calculées en (a) sous forme de logit.

c) A partir d'un modèle de régression logistique, déterminer s'il existe une association significative au seuil $\alpha = 0,05$ entre la pression artérielle, traitée en tant que variable quantitative en considérant les centres de classe, et la probabilité d'avoir un infarctus.

d) Exprimer en unités logit les probabilités d'infarctus prédites par le modèle pour chacun des niveaux de pression artérielle.

e) Afficher sur un même graphique les proportions empiriques et la courbe de régression logistique en fonction des valeurs de pression artérielle (centres de classe).

L'importation des données `hdis.dat` se fait comme suit :

```
> bp <- read.table("hdis.dat", header=TRUE)
> str(bp)
```

Comme on pourra le vérifier, aucun label n'est associé aux modalités de variables d'intérêt (`bpress` pour la pression artérielle, `chol` pour le taux de cholestérol). Pour générer et associer les labels, on peut utiliser les commandes suivantes :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> blab <- c("<117","117-126","127-136","137-146",
            "147-156","157-166","167-186",">186")
> clab <- c("<200","200-209","210-219","220-244",
            "245-259","260-284",">284")
> bp$bpress <- factor(bp$bpress, labels=blab)
> bp$chol <- factor(bp$chol, labels=clab)
```

La dernière commande convertit les variables d'origine en variables qualitatives et associe à leurs modalités les labels définis par `blab` et `clab`. Pour vérifier que la base de données est bien sous la forme désirée, les commandes suivantes sont toujours utiles :

```
> str(bp)
> summary(bp)
```

On peut maintenant reproduire le tableau de fréquences relatives fourni dans l'énoncé :

```
> round(xtabs(hdis/total ~ bpress + chol, data=bp), 2)
```

Puisque l'on ne va s'intéresser qu'à la relation entre pression artérielle et infarctus, il est nécessaire d'aggréger les données sur le taux de cholestérol. En d'autres termes, il est nécessaire de cumuler les effectifs pour chaque niveau de pression artérielle, tous niveaux de cholestérol confondus. On en profitera également pour renommer les niveaux de la variable `bpress` en utilisant les centres des intervalles de classe :

```
> blab2 <- c(111.5,121.5,131.5,141.5,151.5,161.5,176.5,191.5)
> bp$bpress <- rep(blab2, each=7)
> dfrm <- aggregate(bp[,c("hdis","total")],
                    list(bpress=bp[,"bpress"]), sum)
```

C'est la dernière commande, `aggregate()`, qui permet d'aggréger les données : on additionne tous les effectifs (`sum`) de la variable `chol` qui ne figure pas dans la liste des variables que l'on souhaite conserver pour l'analyse. On en profite pour stocker les résultats dans une nouvelle base de données nommée `dfrm`. Notons que dans ce cas nous n'utilisons pas de notation par formule mais la syntaxe alternative d'`aggregate()` (variable réponse suivie d'une liste composée d'une ou plusieurs variables explicatives). Un aperçu des données aggrégées peut être obtenu à l'aide de `head()` :

```
> head(dfrm, 5)
```

Lorsque l'on dispose d'une proportion, p, sa valeur sur une échelle dont les unités sont des logits est donnée par la relation $\log(p/(1-p))$. D'où les commandes suivantes pour convertir les proportions, calculées comme `hdis/total` en unités logit :

```
> logit <- function(x) log(x/(1-x))
> dfrm$prop <- dfrm$hdis/dfrm$total
> dfrm$logit <- logit(dfrm$hdis/dfrm$total)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

On remarquera que l'on a défini une petite fonction permettant de convertir des valeurs `x`, qui ici sont supposées être des proportions, en leur équivalent $\log(x/(1-x))$. On aurait pu écrire de manière équivalente :

```
> log((dfrm$hdis/dfrm$total)/(1-dfrm$hdis/dfrm$total))
```

Les valeurs calculées ont été ajoutées directement au data frame sous le nom `prop` et `logit`.

Le modèle de régression logistique s'écrit de la manière suivante :

```
> glm(cbind(hdis,total-hdis) ~ bpress,data=dfrm,family=binomial)
```

La formulation utilisée, `cbind(hdis, total-hdis) ~ bpress`, tient compte du fait que nous disposons de données groupées, et non des réponses individuelles. La commande `glm()` avec l'option `family=binomial` correspond à une régression logistique, qui, sans vouloir entrer trop dans les détails, utilise par défaut la fonction `logit` comme lien canonique.

Les coefficients de régression peuvent être affichés à l'aide de `summary()` :

```
> summary(glm(cbind(hdis, total-hdis) ~ bpress, data=dfrm,
              family=binomial))
```

Le résultat précédent comprend les informations essentielles pour répondre à la question de la significativité statistique de l'association entre pression artérielle et probabilité d'un infarctus : le coefficient de régression (sur l'échelle log-odds) vaut 0,024 et est significatif au seuil usuel de 5 % (voir colonne `Pr(>|z|)`).

La probabilité d'avoir un infarctus selon les différents niveaux de pression artérielle considérés est obtenue comme suit :

```
> glm.res <- glm(cbind(hdis, total-hdis) ~ bpress, data=dfrm,
                 family=binomial)
> predict(glm.res)
```

Notons que l'on a stocké les résultats intermédiaires générés par R sous le nom `glm.res` avant d'utiliser la commande `predict()`. Les prédictions générées par R sont exprimées sous forme de logit, et l'on peut comparer les logits observés et prédits :

```
> cbind(dfrm, logit.predit=predict(glm.res))
```

Pour représenter graphiquement les proportions d'infarctus observées et prédites en fonction du niveau de pression artérielle, on a pratiquement tous les éléments. Il nous manque les prédictions du modèle exprimées sous forme de proportions, et non sur l'échelle log-odds. Ensuite, on peut vouloir tracer la courbe de prédiction, c'est-à-dire la probabilité d'avoir un infarctus en fonction de la pression artérielle, sans limiter cette dernière aux huit valeurs observées pour la variable `bpress`. Voici une solution possible :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> dfrm$prop.predit <- predict(glm.res, type="response")
> f <- function(x)
    1/(1+exp(-(coef(glm.res)[1]+coef(glm.res)[2]*x)))
> xyplot(hdis/total ~ bpress, data=dfrm, aspect=1.2, cex=.8,
         xlab="Pression artérielle",ylab="Probabilité infarctus",
         panel=function(x, y, ...) {
           panel.xyplot(x, y, col="gray30", pch=19, ...)
           panel.curve(f, lty=3, col="gray70")
           panel.points(x, dfrm$prop.predit, col="gray70", ...)
         })
```

3) Une enquête cas-témoin a porté sur la relation entre la consommation d'alcool et de tabac et le cancer de l'œsophage chez l'homme (étude « Ille et Villaine »). Le groupe des cas était composé de 200 patients atteints d'un cancer de l'œsophage et diagnostiqué entre janvier 1972 et avril 1974. Au total, 775 témoins de sexe masculin ont été sélectionnés à partir des listes électorales. Le tableau 6.3 indique la répartition de l'ensemble des sujets selon leur consommation journalière d'alcool, en considérant qu'une consommation supérieure à 80 g est considérée comme un facteur de risque [BRE 80].

	Consommation d'alcool (g/jour)		
	≥ 80	< 80	Total
Cas	96	104	200
Témoins	109	666	775
Total	205	770	975

Tableau 6.3. *Consommation d'alcool et cancer de l'œsophage*

a) Quelle est la valeur de l'odds-ratio et son intervalle de confiance à 95 % (méthode de Woolf) ? Est-ce une bonne estimation du risque relatif ?

b) Est-ce que la proportion de consommateurs à risque est la même chez les cas et chez les témoins (considérer $\alpha = 0,05$) ?

c) Construire le modèle de régression logistique permettant de tester l'association entre la consommation d'alcool et le statut des sujets. Le coefficient de régression est-il significatif ?

d) Retrouvez la valeur de l'odds-ratio observé, calculé en (b), et son intervalle de confiance à partir des résultats de l'analyse de régression.

Comme les données brutes (individuelles) ne sont pas disponibles, il faut travailler directement avec le tableau d'effectifs fourni dans l'énoncé :

```
> alcool <- matrix(c(666,104,109,96), nr=2,
                   dimnames=list(c("Témoin","Cas"),
                                 c("<80",">=80")))
> alcool
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Concernant l'odds-ratio, on utilisera la commande `oddsratio()` du package `vcd` :

```
> library(vcd)
> oddsratio(alcool, log=FALSE)
```

L'option `log=FALSE` nous assure que le résultat renvoyé correspond bien à un odds-ratio et non au log odds-ratio. Pour obtenir un intervalle de confiance asymptotique, on utilisera :

```
> confint(oddsratio(alcool, log=FALSE))
```

De même, on pourrait utiliser `summary(oddsratio(alcool))` pour effectuer un test d'hypothèse sur le log odds-ratio ($H_0 : \log(\hat{\theta}) = 0$).

Pour tester l'hypothèse que la proportion de personnes avec une consommation journalière est ≥ 80 g est identique chez les cas et les témoins, on peut utiliser la commande `prop.test()` en indiquant les effectifs observés à partir du tableau croisé donné dans l'énoncé :

```
> prop.test(c(96,109), c(200,775), correct=FALSE)
```

Ce test est exactement équivalent à un test Z pour tester la différence entre deux proportions estimées à partir des données (si l'on n'utilise pas de correction de continuité).

Pour le modèle de régression logistique, on a besoin de transformer le tableau de contingence en un tableau de données où l'on fait clairement apparaître les deux variables qualitatives (maladie et exposition), c'est-à-dire un data frame. Cela peut être réalisé en utilisant la commande la commande `melt()` du package `reshape2` qui permet de transformer les données sous forme tabulaire en format « long ». On en profitera pour recoder les niveaux de la variable maladie en 0 (témoins) et 1 (cas), même si ce n'est pas vraiment nécessaire, et considérer le niveau 0 comme la catégorie de référence (ce qui facilite l'interprétation des résultats) :

```
> library(reshape2)
> alcool.df <- melt(alcool)
> names(alcool.df) <- c("maladie", "exposition", "n")
> levels(alcool.df$maladie) <- c(1,0)
> alcool.df$maladie <- relevel(alcool.df$maladie, "0")
```

Le modèle de régression logistique s'écrit alors :

```
> glm.res <- glm(maladie ~ exposition, data=alcool.df,
                 family=binomial, weights=n)
> summary(glm.res)
```

Le résultat qui nous intéresse est la ligne associée à `exposition>=80`, puisqu'elle nous renseigne sur la valeur du coefficient de régression associé à l'exposition et estimé par R, avec son erreur standard, ainsi que la valeur de la statistique de test.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Ici, le coefficient de régression s'interprète comme le log de l'odds-ratio. Notons que l'on obtiendrait exactement les mêmes résultats en intervertissant le rôle des variables dans la formulation précédente, `exposition ~ maladie`.

On peut retrouver l'odds-ratio calculé plus haut à partir du coefficient de régression associé au facteur d'intérêt (`exposition`), ainsi que son intervalle de confiance à 95 %, de la manière suivante :

```
> exp(coef(glm.res)[2])
> exp(confint(glm.res)[2,])
```

Une deuxième solution consiste à considérer le nombre de cas et le nombre total d'individus :

```
> alcool2 <- data.frame(expos=c("<80",">=80"), cas=c(104,96),
                        total=c(104+666, 96+109))
> summary(glm(cbind(cas, total-cas) ~ expos, data=alcool2,
              family=binomial))
```

4) On dispose de données issues d'une étude cherchant à établir la validité pronostique de la concentration en créatine kinase dans l'organisme sur la prévention de la survenue d'un infarctus du myocarde [RAB 04].

Les données sont disponibles dans le fichier `sck.dat` : la première colonne correspond à la variable créatine kinase (`ck`), la deuxième à la variable présence de la maladie (`pres`) et la dernière à la variable absence de maladie (`abs`).

a) Quel est le nombre total de sujets ?

b) Calculer les fréquences relatives malades/non-malades, et représenter leur évolution en fonction des valeurs de créatine kinase à l'aide d'un diagramme de dispersion (points + segments reliant les points).

c) A partir d'un modèle de régression logistique dans lequel on cherche à prédire la probabilité d'être malade, calculer la valeur de `ck` à partir de laquelle ce modèle prédit que les personnes présentent la maladie en considérant une valeur seuil de 0,5 (si $P(\text{malade}) \geq 0,5$ alors `malade=1`).

d) Représenter graphiquement les probabilités d'être malade prédites par ce modèle ainsi que les proportions empiriques en fonction des valeurs `ck`.

e) Etablir la courbe ROC correspondante, et reporter la valeur de l'aire sous la courbe.

Comme le nom des variables ne figure pas dans le fichier de données, il faudra les attribuer aussitôt après avoir importé les données :

```
> sck <- read.table("sck.dat", header=FALSE)
> names(sck) <- c("ck", "pres", "abs")
> summary(sck)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Le nombre total de sujets correspond à la somme des effectifs pour les deux variables `pres` et `abs`, soit :

```
> sum(sck[,c("pres","abs")])
```

ou de manière équivalente : `sum(sck$pres) + sum(sck$abs)` (mais pas `sck$pres + sck$abs`).

Pour calculer les fréquences relatives de ces deux variables, il est nécessaire de connaître les effectifs totaux par variable. Ceux-ci peuvent être obtenus en utilisant la commande `apply` et en opérant par colonnes :

```
> ni <- apply(sck[,c("pres","abs")], 2, sum)
```

A partir de là, il suffit de diviser chaque valeur des variables `pres` et `abs` par les nombres calculés précédemment. On stockera les valeurs obtenues dans deux nouvelles variables, dans le même tableau de données :

```
> sck$pres.prop <- sck$pres/ni[1]
> sck$abs.prop <- sck$abs/ni[2]
```

On peut vérifier que les calculs sont corrects : la somme des valeurs pour chaque variable doit maintenant valoir 1 :

```
> apply(sck[,c("pres.prop","abs.prop")], 2, sum)
```

Il suffit ensuite de représenter les proportions obtenues dans un même graphique, en considérant les valeurs de la variable `ck` comme abscisses :

```
> xyplot(pres.prop + abs.prop ~ ck, data=sck, type=c("b", "g"),
         auto.key=TRUE, ylab="Fréquence")
```

L'instruction `type=c("b", "g")` signifie que l'on souhaite afficher des points reliés par des lignes (`"b"="o"+"l"`) ainsi qu'un quadrillage (`"g"`).

Le modèle de régression pour données groupées s'écrit :

```
> glm.res <- glm(cbind(pres,abs) ~ ck, data=sck, family=binomial)
> summary(glm.res)
```

Les prédictions, exprimées sous forme de probabilités et non sur l'échelle log-odds, sont obtenues à l'aide de la commande `predict` en précisant l'option `type="response"` comme ci-après :

```
> glm.pred <- predict(glm.res, type="response")
> names(glm.pred) <- sck$ck
```

En considérant que des probabilités $\geq 0,5$ signifie que les individus sont malades, on obtient donc la répartition suivante :

```
> glm.pred[glm.pred >= 0.5]
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

On en conclut que les personnes seront considérées malades, selon ce modèle, pour des valeurs ck de 80 ou plus. Cela se vérifie aisément sur un graphique dans lequel on reporte les probabilités prédites en fonction des valeurs de la variable ck :

```
> sck$malade <- sck$pres/(sck$pres+sck$abs)
> xyplot(glm.pred ~ sck$ck, type="l",
         ylab="Probabilité", xlab="ck",
         panel=function(...) {
           panel.xyplot(...)
           panel.xyplot(sck$ck, sck$malade, pch=19, col="grey")
         })
```

Dans un premier temps, il faut « décompacter » les données groupées et créer un tableau avec deux colonnes : la première représentant la variable ck et la seconde représentant la présence ou absence de maladie. On utilisera les effectifs par sous-groupe calculés précédemment et disponible dans la variable ni :

```
> sck.expand <- data.frame(ck=c(rep(sck$ck, sck$pres),
                                rep(sck$ck, sck$abs)),
                           malade=c(rep(1,ni[1]), rep(0,ni[2])))
> table(sck.expand$malade)
> with(sck.expand, tapply(malade, ck, sum))\vspace*{6pt}
```

Les deux dernières commandes visent à s'assurer que l'on retombe bien sur les mêmes effectifs et que la distribution des personnes malades par valeur de ck est correcte. On peut également vérifier que l'on obtient bien les mêmes résultats concernant la régression logistique, et en profiter pour ajouter les valeurs prédites au tableau de données précédent (on pourrait procéder comme précédemment et répliquer les prédictions, mais cela est plus simple ainsi) :

```
> glm.res2 <- glm(malade ~ ck, data=sck.expand, family=binomial)
> sck.expand$prediction <- ifelse(predict(glm.res2,
                                          type="response")>=0.5,
                                          1, 0)
> with(sck.expand, table(malade, prediction))\vspace*{6pt}
```

La dernière commande permet d'afficher une matrice de confusion dans laquelle on croise les diagnostiques réels et ceux prédits par le modèle de régression. On peut ainsi comparer les taux de classification correcte lorsque l'on varie le seuil de référence :

```
> classif.tab <- with(sck.expand, table(malade, prediction))
> sum(diag(classif.tab))/sum(classif.tab)
```

Pour afficher la courbe ROC, on utilisera le package ROCR :

```
> library(ROCR)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> pred <- prediction(predict(glm.res2, type="response"),
                     sck.expand$malade)
> perf <- performance(pred, "tpr","fpr")
> plot(perf, ylab="Sensibilité", xlab="1-Spécificité")
> grid()
> abline(0, 1, lty=2)
```

La valeur de l'aire sous la courbe est obtenue comme suit :

```
> performance(pred, "auc")@"y.values"
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Chapitre 7

Analyse des données de survie

Ce dernier chapitre constitue une introduction à la modélisation des données de survie avec R. Une attention particulière sera consacrée à la représentation des données censurées sous R et aux principales commandes permettant de manipuler et résumer ce type de données. L'estimation de la fonction de survie par la méthode de Kaplan-Meier et le modèle de régression de Cox sont ensuite discutés plus en détail.

Dans ce chapitre, les illustrations sont réalisées à partir d'un jeu de données différent de celui utilisé jusqu'à présent. Les données sont disponibles dans le package `survival` et peuvent être chargées de la manière suivante :

```
> library(survival)
> data(lung)
```

Il s'agit de données sur la survie de patients atteints d'un cancer du poumon [LOP 94]. Les variables d'intérêt sont les suivantes : `time` (temps de survie en jours), `status` (état à la date de point ; 1 = donnée censurée, 2 = patient décédé), `age` (âge du patient) et `sex` (sexe du patient ; 1 = homme, 2 = femme). Au total, il y a 228 patients.

Le package `lattice` ne sera pas utilisé dans ce chapitre ; au contraire, les fonctions graphiques de base disponibles dans le package `survival` seront utilisées. Cela dit, il n'est pas très difficile d'adapter les graphiques proposés par le package `survival` à l'aide de `lattice`.

7.1. Représentation des données et statistiques descriptives

Les données censurées représentent un type à part, de même que les dates et les séries chronologiques. Pour l'essentiel, les commandes de gestion des données

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

censurées sont fournies dans le package `survival`, qui figure parmi les packages de base de R.

Si l'on dispose d'une série d'événements codés en 0/1 et de données temporelles associées (durées, dates, etc.), la commande à utiliser est `Surv()` : elle permet de spécifier les mesures temporelles (`time=`) et l'événement (`event=`). Il est également possible de spécifier un intervalle de temps lorsque les données s'y prêtent (voir l'option `time2=`). Dans tous les cas, le statut de l'observation au moment de la date de point, ou de fin d'étude, doit permettre de distinguer les censures des événements survenus durant la date de suivi. Habituellement, les données sont codées en 0/1 ou 1/2, où 0 (respectivement, 1) indique une donnée censurée. Le statut peut également être codé en `TRUE/FALSE`, où `event=TRUE` signifie la survenue de l'événement (décès ou autre) et `event=FALSE` indique une donnée censurée (à droite). Voici comment procéder avec les données `lung` chargées précédemment :

```
> st <- with(lung, Surv(time=time, event=status))
> head(st)
[1]  306   455  1010+  210   883  1022+
```

Les données censurées sont signalées par la présence du suffixe +. C'est le cas de la troisième observation dans l'exemple précédent : cet individu n'était pas décédé au bout de 1 010 jours de suivi, mais on n'a plus d'information ensuite. On peut le vérifier à partir des données brutes en affichant les six premières valeurs pour le statut vital des patients :

```
> head(subset(lung, select=c(time, status)))
  time status
1  306      2
2  455      2
3 1010      1
4  210      2
5  883      2
6 1022      1
```

7.2. Fonction de survie et courbe de Kaplan-Meier

7.2.1. *Table de mortalité/survie*

Il est possible de construire un tableau de survie et d'obtenir les probabilités de survie, éventuellement après stratification selon les niveaux d'un facteur de groupe, à l'aide de la commande `survfit()`. Par défaut, `survfit()` affiche la médiane de survie et son intervalle de confiance à 95 % :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> s <- survfit(st ~ 1, data=lung)
> s
Call: survfit(formula = st ~ 1, data = lung)

records   n.max n.start  events  median 0.95LCL 0.95UCL
    228     228     228     165     310     285     363
```

L'option `conf.type=` permet éventuellement de modifier le type d'intervalles de confiance calculés pour l'estimateur de Kaplan-Meier. Par défaut, une transformation logarithmique est utilisée. Dans tous les cas, les intervalles de confiance sont estimés pour chaque point de la fonction de survie. Des informations complémentaires sont disponibles dans l'aide en ligne, voir `help(survfit.formula)`.

La commande `summary()` permet de générer le tableau de survie et l'estimation non paramétrique de la fonction de survie :

```
> summary(s)
```

Par défaut, R affiche les valeurs de la fonction de survie $S(t)$ pour l'ensemble des durées observées. On peut toutefois restreindre le calcul et l'affichage des résultats à une certaine fenêtre temporelle en utilisant l'option `times=`, par exemple :

```
> summary(s, times=seq(1, 200, by=20))
Call: survfit(formula = st ~ 1, data = lung)

 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    1    228       0    1.000  0.0000        1.000        1.000
   21    220       8    0.965  0.0122        0.941        0.989
   41    217       3    0.952  0.0142        0.924        0.980
   61    211       7    0.921  0.0179        0.887        0.957
   81    205       7    0.890  0.0207        0.851        0.932
  101    196       6    0.864  0.0227        0.821        0.910
  121    189       6    0.837  0.0245        0.791        0.887
  141    184       5    0.815  0.0257        0.766        0.867
  161    176       8    0.780  0.0275        0.728        0.836
  181    159      15    0.713  0.0301        0.656        0.774
```

ou plus simplement, pour un temps donné :

```
> summary(s, times=300)$surv
[1] 0.5306081
```

Comme discuté dans le chapitre 3 pour le test du χ^2 de Pearson, on exploite ici le fait que R produit un ensemble de résultats *via* la commande `summary()`

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

mais ne les affiche pas tous. Une façon d'accéder à l'ensemble des informations produites par cette commande est donc d'utiliser `str(summary(s, times=300))` et de sélectionner la variable contenant le résultat recherché.

Si l'on souhaite comparer deux groupes, il suffit d'ajouter la variable de groupement (de type `factor`) à droite de la formule, comme ceci :

```
> lung$sex <- factor(lung$sex, labels=c("Male", "Female"))
> s2 <- survfit(st ~ sex, data=lung)
> s2
Call: survfit(formula = st ~ sex, data = lung)

           records n.max n.start events median 0.95LCL 0.95UCL
sex=Male       138   138     138    112    270     212     310
sex=Female      90    90      90     53    426     348     550
```

La commande `summary()` fonctionne ensuite sur le même principe et fournit une estimation de $S(t)$ pour chacun des groupes définis par les niveaux de la variable `sex` :

```
> summary(s2, times=seq(5,20,by=5))
Call: survfit(formula = st ~ sex, data = lung)

                sex=Male
 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    5    138       0    1.000  0.0000        1.000        1.000
   10    138       0    1.000  0.0000        1.000        1.000
   15    132       7    0.949  0.0187        0.913        0.987
   20    131       0    0.949  0.0187        0.913        0.987

                sex=Female
 time n.risk n.event survival std.err lower 95% CI upper 95% CI
    5     90       1    0.989   0.011        0.967            1
   10     89       0    0.989   0.011        0.967            1
   15     89       0    0.989   0.011        0.967            1
   20     89       0    0.989   0.011        0.967            1
```

7.2.2. *Courbe de Kaplan-Meier*

Pour représenter graphiquement la fonction de survie sous la forme d'une courbe de Kaplan-Meier, il est nécessaire d'appliquer la commande `plot()` au résultat produit par `survfit()` (figure 7.1). Il est souvent plus pratique de sauvegarder dans

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

une variable auxiliaire le résultat de la commande `survfit()`, comme cela a été fait au paragraphe précédent (variables `s` et `s2`) :

```
> plot(s, xlab="Temps", ylab="Probabilité de survie")
```

Les censures apparaissent sous la forme de petits segments le long de la courbe de survie.

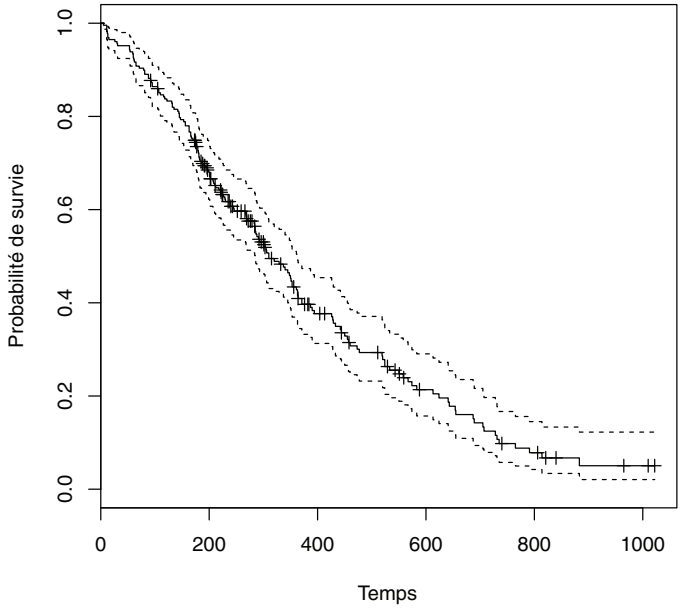


Figure 7.1. *Fonction de survie calculée à partir de l'estimateur de Kaplan-Meier, avec intervalle de confiance à 95 %*

Par défaut, R affiche des intervalles de confiance à 95 % pour la courbe de survie. Pour les supprimer, il faut ajouter l'option `conf.int="none"`. A l'inverse, lorsque l'on souhaite afficher plusieurs courbes de survie, R ne présente pas les intervalles de confiance, mais on peut les ajouter en ajoutant l'option `conf.int="both"`. Il est également possible de faire figurer une légende grâce à la commande `legend()` (figure 7.2) :

```
> plot(s2, conf.int="none", col=c("black", "gray"), xlab="Temps",
       ylab="Prob. survie")
> legend("bottomleft", legend=levels(lung$sex), lty=1,
         col=c("black", "gray"), bty="n")
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

L'option `bty="n"` permet de supprimer l'affichage d'un cadre autour de la légende. On remarquera que même si le rendu des lignes représentant les courbes de survie ne varie pas (trait plein, pointillés, etc.), il est tout de même nécessaire d'indiquer une option `lty=1` dans la commande `legend()` pour afficher des lignes dans l'encadré de légende.

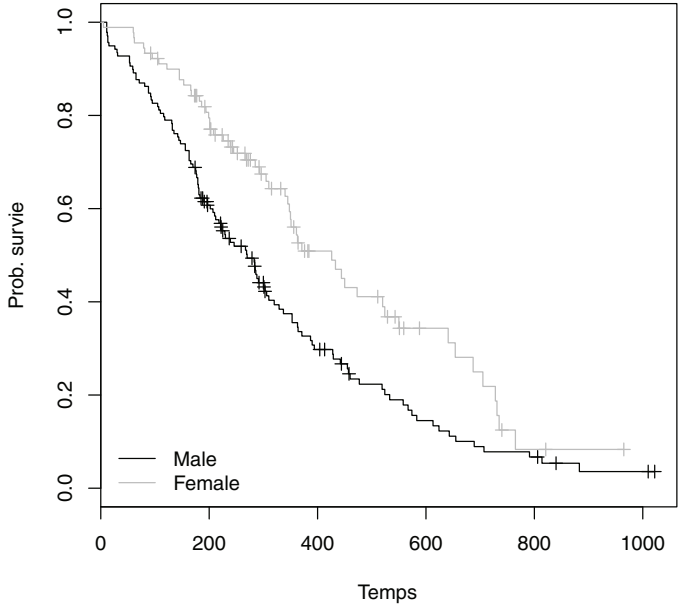


Figure 7.2. *Fonction de survie pour chacun des niveaux de la variable sex*

Les commandes graphiques utilisées dans ce cas appartiennent aux commandes des graphiques de base de R. Cela dit, on peut toujours retrouver les informations qui nous intéressent, ici le temps et la probabilité de survie estimée par la méthode de Kaplan-Meier, et travailler directement avec la commande `xyplot()` du package `lattice`.

7.2.3. *Fonction de risque cumulé*

Il est également possible de travailler avec la fonction de risque $H(t)$ (« hazard rate ») qui est reliée à la fonction de survie par la relation $S(t) = \exp(-H(t))$. Une façon d'estimer cette courbe consiste à utiliser une transformation de type $\hat{H}(t) = -\log \hat{S}(t)$, ce que propose la commande `plot()` pour les objets créés avec `survfit()` en ajoutant l'option `fun="cumhaz"` (figure 7.3) :

```
> plot(s, fun="cumhaz")
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

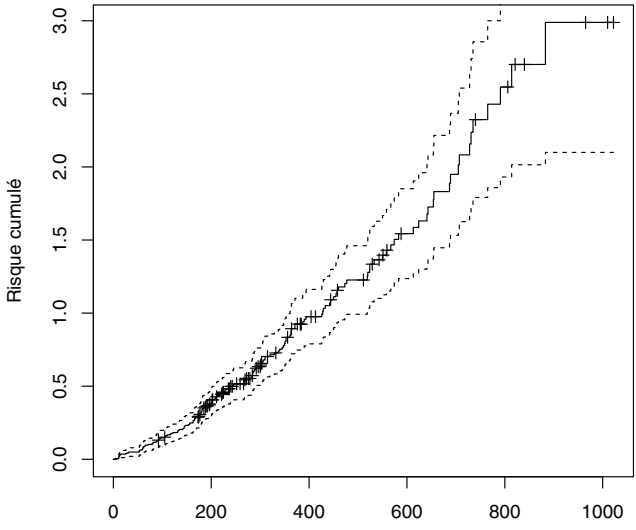


Figure 7.3. *Fonction de risque cumulé*

7.2.4. *Test d'égalité des fonctions de survie*

Pour tester l'hypothèse nulle d'égalité de deux ou plusieurs fonctions de survie, on peut utiliser le test du log-rank qui est disponible dans la commande `survdiff()` :

```
> survdiff(st ~ sex, data=lung)
Call:
survdiff(formula = st ~ sex, data = lung)

             N Observed Expected (O-E)^2/E (O-E)^2/V
sex=Male   138      112     91.6      4.55      10.3
sex=Female  90       53     73.4      5.68      10.3

 Chisq= 10.3  on 1 degrees of freedom, p= 0.00131
```

Si l'on souhaite réaliser un test de Gehan-Wilcoxon à la place, il est nécessaire de préciser l'option `rho=1` :

```
> survdiff(st ~ sex, data=lung, rho=1)
Call:
survdiff(formula = st ~ sex, data = lung, rho = 1)

             N Observed Expected (O-E)^2/E (O-E)^2/V
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
sex=Male   138     70.4     55.6      3.95      12.7
sex=Female  90     28.7     43.5      5.04      12.7

 Chisq= 12.7  on 1 degrees of freedom, p= 0.000363
```

Lorsque l'on souhaite donner plus de poids à la première partie de la fonction de survie, on donne généralement des poids (`rho=`) supérieurs à 0, alors qu'au contraire si l'on souhaite donner plus d'importance aux événements plus éloignés dans le temps on utilisera des valeurs négatives pour `rho=`.

7.3. Régression de Cox

La commande permettant de réaliser une régression de Cox est `coxph()`. Comme dans le cas de la régression linéaire (chapitre 5) ou logistique (chapitre 6), on associera cette commande qui permet de construire le modèle de régression à la commande `summary()` qui fournit les coefficients de régression et les tests de significativité associés :

```
> m <- coxph(st ~ sex, data=lung)
> m
Call:
coxph(formula = st ~ sex, data = lung)


            coef exp(coef) se(coef)     z      p
sexFemale -0.531     0.588    0.167 -3.18 0.0015

Likelihood ratio test=10.6  on 1 df, p=0.00111  n= 228
```

La variable `st` est bien la représentation adéquate pour les données de survie et correspond à `Surv(time, status)`, où `time` dénote les durées et `status` code pour l'événement (0=donnée censurée).

Il est possible de considérer différentes méthodes pour gérer le cas des ex-æquos grâce à l'option `method=`. Par défaut, R utilise la méthode de Efron, mais on peut ajouter `method="breslow"` pour utiliser la méthode de Breslow. On peut également faire intervenir des facteurs de stratification en utilisant l'instruction `strata()` directement dans le membre droit de la formule R spécifiant le modèle. Par exemple, le modèle suivant est un modèle stratifé sur le sexe et considérant l'âge comme principale variable explicative :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> coxph(Surv(time, status) ~ age + strata(sex), data=lung)
Call:
coxph(formula=Surv(time, status) ~ age + strata(sex), data=lung)


      coef exp(coef) se(coef)    z     p
age 0.0162      1.02  0.00919 1.76 0.078

Likelihood ratio test=3.18  on 1 df, p=0.0744  n= 228
```

Le tableau des coefficients de régression est donné ci-après :

```
> summary(m)
Call:
coxph(formula = st ~ sex, data = lung)

  n= 228, number of events= 165

             coef exp(coef) se(coef)      z Pr(>|z|)
sexFemale -0.5310    0.5880   0.1672 -3.176  0.00149 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

          exp(coef) exp(-coef) lower .95 upper .95
sexFemale     0.588      1.701    0.4237     0.816

Concordance= 0.579  (se = 0.022 )
Rsquare= 0.046   (max possible= 0.999 )
Likelihood ratio test= 10.63  on 1 df,   p=0.001111
Wald test            = 10.09  on 1 df,   p=0.001491
Score (logrank) test = 10.33  on 1 df,   p=0.001312
```

Comme dans le cas des autres modèles de régression discutés aux chapitres précédents, il est possible d'obtenir directement les coefficients en utilisant la commande `coef()`.

On peut retrouver l'estimateur de Nelson-Aalen à partir du modèle de Cox de la manière suivante :

```
> sna <- survfit(coxph(st ~ 1), type="aalen")
> summary(sna)
```

et comparer avec les probabilités de survie estimées par la méthode de Kaplan-Meier (section 7.2.3).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
h <- -log(sna$surv)
hest <- data.frame(time=sna$time, d=sna$n.event, n=sna$n.risk,
                   aalen=h, s=sna$surv)
kmest <- data.frame(time=s$time, s=s$surv)
psurv <- merge(hest, kmest, by="time")
```

Les commandes précédentes permettent de construire un tableau de survie avec pour chaque valeur de temps, le nombre d'événements observés (`d`), le nombre de personnes à risque (`n`), l'estimateur de Nelson-Aalen (`aalen`) calculé comme l'anti-log de la probabilité de survie estimée *via* le modèle de Cox. La commande `merge()` est utilisée pour assembler les deux tableaux de données, le premier contenant les données calculées à partir du modèle de Cox (`hest`) et le second les données calculées par la méthode actuariale en début de chapitre (`kmest`). L'opération de jointure des deux tableaux se fait à partir de la variable `time`. Après fusion des deux tableaux, la variable `s.x` désigne l'estimateur de Nelson-Aalen et `s.y` l'estimateur de Kaplan-Meier pour la fonction de survie. On peut les renommer pour plus de clarté et afficher les deux courbes de survie comme dans la figure 7.4 :

```
> xyplot(s.x + s.y ~ time, data=psurv, type=c("l", "g"),
         ylab="Prob. survival",
         auto.key=list(corner=c(0,0), text=c("Nelson-Aalen",
                                             "Kaplan-Meier"),
                       lines=TRUE, points=FALSE))
```

Enfin, la commande `cox.zph()` permet de vérifier l'hypothèse de risques proportionnels ; deux méthodes sont fournies : par défaut, cette commande affiche le résultat d'un test du χ^2 pour chaque variable du modèle ainsi qu'un test global (ces tests restent sensibles à la tendance linéaire au niveau du hasard), mais elle peut être couplée avec une commande `plot()` pour visualiser graphiquement l'évolution des résidus de Schoenfeld en fonction du temps (après transformation). Il est ensuite possible de rajouter un terme d'interaction entre un cofacteur et le temps afin de vérifier la présence d'une interaction entre ces deux variables, ou de construire un facteur de stratification (voir, par exemple, *Cox Proportional-Hazards Regression for Survival Data in R* de John Fox et Sanford Weisberg, http://socserv.mcmaster.ca/jfox/Books/Companion/appendix/Appendix-Cox-Regression.pdf).

7.4. Ce qu'il faut retenir

– Les données de survie sont représentées sous R à l'aide de la commande `Surv()` dans laquelle on indique les informations concernant le temps et les événements (décès ou autre, et censure).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

– L'estimateur de Kaplan-Meier s'obtient avec la commande `survfit()` et on peut comparer différentes fonctions de survie grâce à la commande `survdiff()` (test du log-rank ou de Wilcoxon).

– Le modèle de Cox fonctionne sur le même principe que les autres modèles de régression, et on utilise une notation par formule pour représenter la relation entre une variable réponse de type survie et des variables explicatives. La commande `summary()` permet d'afficher les coefficients du modèle.

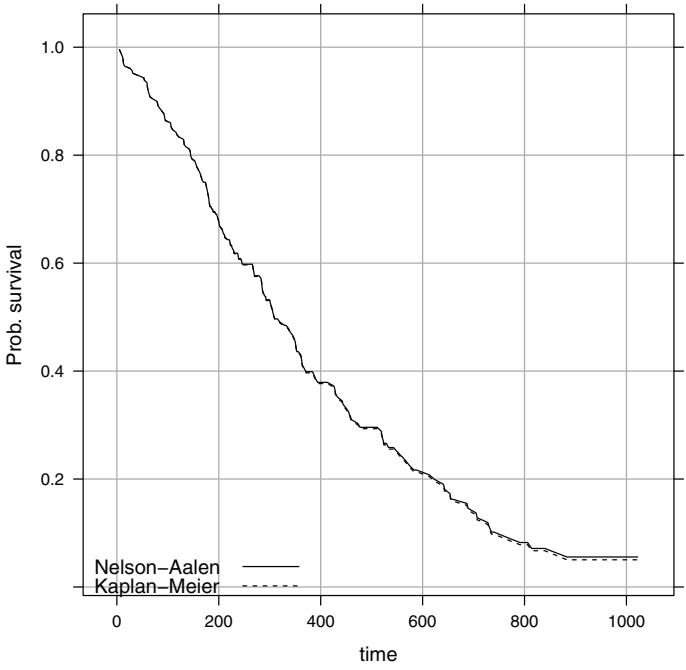


Figure 7.4. *Comparaison des fonctions de survie en utilisant l'estimateur de Kaplan-Meier ou de Nelson-Aalen*

7.5. Pour aller plus loin

Bien que datant de 1999, le manuel *A Package for Survival Analysis in S* [THE 99] rédigé par l'auteur du package `survival` fournit de nombreuses informations complémentaires. De même, le lecteur intéressé trouvera un développement plus approfondi de la modélisation des données de survie avec R dans l'ouvrage de Frank Harrell [HAR 01].

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

7.6. Applications

1) Dans un essai contre placebo sur la cirrhose biliaire, la D-penicillamine (DPCA) a été introduite dans le bras actif sur une cohorte de 312 patients. Au total, 154 patients ont été randomisés dans le bras actif (variable traitement, `rx`, 1=Placebo, 2=DPCA). Un ensemble de données telles que l'âge, des données biologiques et signes cliniques variés incluant le niveau de bilirubine sérique (`bilirub`) sont disponibles dans le fichier `pbc.txt` [VIT 05]. Le statut du patient est enregistré dans la variable `status` (0=vivant, 1=décédé) et la durée de suivi (`years`) représente le temps écoulé en années depuis la date de diagnostic.

a) Combien dénombre-t-on d'individus décédés ? Quelle proportion de ces décès retrouve-t-on dans le bras actif ?

b) Afficher la distribution des durées de suivi des 312 patients, en faisant apparaître distinctement les individus décédés. Calculer le temps médian (en années) de suivi pour chacun des deux groupes de traitement. Combien y a-t-il d'événements positifs au-delà de 10,5 années et quel est le sexe de ces patients ?

c) Les 19 patients dont le numéro (`number`) figure parmi la liste suivante ont subi une transplantation durant la période de suivi :

```
5 105 111 120 125 158 183 241 246 247 254 263 264 265 274 288 291
295 297 345 361 362 375 380 383
```

Indiquer leur âge moyen, la distribution selon le sexe et la durée médiane de suivi en jours jusqu'à la transplantation.

d) Afficher un tableau résumant la distribution des événements à risque en fonction du temps, avec la valeur de survie associée.

e) Afficher la courbe de Kaplan-Meier avec un intervalle de confiance à 95 %, sans considérer le type de traitement.

f) Calculer la médiane de survie et son intervalle de confiance à 95 % pour chaque groupe de sujets et afficher les courbes de survie correspondantes.

g) Effectuer un test du log-rank en considérant comme prédicteur le facteur `rx`. Comparer avec un test de Wilcoxon.

h) Effectuer un test du log-rank sur le facteur d'intérêt (`rx`) en stratifiant sur l'âge. On considèrera trois groupe d'âge : 40 ans ou moins, entre 40 et 55 ans inclus, plus de 55 ans.

i) Retrouver les résultats de l'exercice 1.g avec une régression de Cox.

Le chargement des données ne pose pas de difficultés particulières :

```
> pb <- read.table("data/pbc.txt", header=TRUE)
> names(pb)[1:20]
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Pour faciliter la lecture des résultats renvoyés par R, recodons d'emblée les variables traitement (`rx`) et sexe (`sex`) comme variables qualitatives :

```
> pb$rx <- factor(pb$rx, labels=c("Placebo", "DPCA"))
> table(pb$rx)
> pb$sex <- factor(pb$sex, labels=c("M","F"))
```

Le nombre ou la proportion de patients décédés peut être obtenu directement à partir d'un tableau d'effectif sur la variable `status` :

```
> prop.table(table(pb$status))
```

La proportion de décès par groupe de traitement s'obtient en croisant les deux variables `status` et `rx` et en calculant les fréquences relatives par ligne :

```
> prop.table(with(pb, table(status, rx)), 1)
```

Pour afficher la distribution des temps de suivi, on peut utiliser un simple diagramme de dispersion dans lequel les coordonnées des points à afficher sont définis par le numéro d'observation et la durée de suivi depuis le diagnostique :

```
> xyplot(number ~ years, data=pb, pch=pb$status, cex=.8)
```

Le temps médian de suivi par groupe de traitement est obtenu à partir d'une commande de type `tapply()` en considérant la variable `rx` comme variable de classification :

```
> with(pb, tapply(years, rx, median))
```

Le nombre de patients décédés au-delà de 10,5 ans est obtenu comme suit :

```
> with(pb, table(status[years > 10.5]))
> with(pb, table(sex[years > 10.5 & status == 1]))
```

soit quatre patients, dont deux patients de sexe féminin. Notons que la dernière commande pourrait également s'écrire :

```
> subset(pb, years > 10.5 & status == 1, sex)
```

ce qui a l'avantage de fournir les numéros d'observation.

Les patients transplantés figurent normalement parmi les individus vivants à la date de point. On peut le vérifier aisément à l'aide d'un simple tri à plat du `status` de ces patients :

```
> idx <- c(5,105,111,120,125,158,183,241,246,247,254,263,264,265,
           274,288,291,295,297,345,361,362,375,380,383)
> table(pb$status[pb$number %in% idx])
```

Pour calculer les quantités demandées (âge moyen, sexe, durée de suivi), on peut dans un premier temps réduire le tableau de données initial à ces seuls patients :

```
> pb.transp <- subset(pb, number %in% idx, c(age, sex, years))
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Ensuite, il s'agit simplement d'appliquer les commandes `mean()`, `table` et `median()` aux variables sélectionnées pour ces 19 patients :

```
> mean(pb.transp$age)
> table(pb.transp$sex)
> median(pb.transp$years * 365)
```

Pour obtenir un tableau des événements avec la survie associée, il est nécessaire d'utiliser la commande `Surv()` du package `survival`, dont voici un exemple d'utilisation :

```
> library(survival)
> head(with(pb, Surv(time=years, event=status)))
```

Les données censurées apparaissent suffixées avec un `+`. Un tableau des événements en fonction du temps peut être construit à l'aide de la commande `survfit()`. On notera que lorsque l'on ne considère aucun facteur de groupe ou de stratification, il faut utiliser la syntaxe un peu particulière `Surv(time, event) ~ 1` :

```
> s <- survfit(Surv(years, status) ~ 1, data=pb)
> summary(s)
```

Une fois que le tableau de survie a été construit, il est très simple d'afficher l'estimateur de la survie sous la forme d'une courbe de Kaplan-Meier à l'aide de la commande `plot()` :

```
> plot(s)
```

La médiane de survie pour chaque bras de traitement peut être obtenue à l'aide de la commande `survdiff()`. Redéfinissons dans un premier temps le modèle de survie pour inclure le prédicteur d'intérêt :

```
> s <- survfit(Surv(years, status) ~ rx, data=pb)
```

Pour obtenir la médiane de survie, il suffit d'afficher le résultat précédent en tapant simplement le nom de la variable dans laquelle on a stocké le résultat renvoyé par la commande `survfit`, ici `s`. On utilisera la commande `plot(s)` pour obtenir les courbes de survie correspondantes.

Le test du log-rank se réalise à partir de la commande `survdiff()`, avec les mêmes notations que dans le cas du calcul de la médiane de survie :

```
> survdiff(Surv(years, status) ~ rx, data=pb)
```

L'option `rho=` permet de contrôler le type de test réalisé. En ajoutant `rho=1` on réalise un test de Wilcoxon au lieu du log-rank classique :

```
> survdiff(Surv(years, status) ~ rx, data=pb, rho=1)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Pour réaliser une analyse stratifiée sur l'âge, il est nécessaire de recoder cette variable numérique en variable qualitative, ce que l'on peut réaliser à l'aide de la commande `cut()` :

```
> agec <- cut(pb$age, c(26, 40, 55, 79), include.lowest=TRUE)
```

On peut effectuer de nouveau le test du log-rank en incluant ce facteur de stratification additionnel en adaptatnt légèrement le modèle précédent :

```
> survdiff(Surv(years, status) ~ rx + strata(agec), data=pb)
```

Enfin, pour le modèle de Cox, c'est la commande `coxph()` qu'il faut utiliser.

```
> summary(coxph(Surv(years, status) ~ rx + strata(agec), data=pb))
```

2) Dans le tableau 7.1 figurent les durées de rémission en semaines dans un essai comparant 6-MP à un placebo [FRE 63]. Les observations censurées (sujets sans rechute aux dernières nouvelles) sont signalées par une astérisque.

Placebo	1 1 2 2 3 4 4 5 5 8 8 8 8 11 11 12 12 15 17 22 23
6-MP	6 6 6 6* 7 9* 10 10* 11* 13 16 17* 19* 20* 22 23 25* 32* 32* 34* 35*

Tableau 7.1. *Les données 6-MP*

a) Construire un tableau indiquant le nombre d'exposés et le nombre de rechutes en fonction du temps pour le groupe 6-MP.

b) Afficher la courbe de survie dans le groupe 6-MP, ainsi que son intervalle de confiance à 95 % (estimateur de Kaplan-Meier).

c) Afficher la fonction de risque cumulée (estimateur de Nelson) pour le même groupe.

d) Calculer la médiane de survie du groupe 6-MP.

e) Calculer la survie espérée pour un patient du groupe 6-MP à quinze semaines.

Dans un premier temps, il est nécessaire de construire un tableau des données brutes, dans lequel on reportera le traitement, la durée de survie et la censure (0 = vivant/1 = décédé) :

```
> placebo.time <- c(1,1,2,2,3,4,4,5,5,8,8,8,8,11,11,12,12,15,17,
                    22,23)
> placebo.status <- rep(1, length(placebo.time))
> mp.time <- c(6,6,6,6,7,9,10,10,11,13,16,17,19,20,22,23,25,32,
               32,34,35)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> mp.status <- c(1,1,1,0,1,0,1,0,0,1,1,0,0,0,1,1,0,0,0,0,0)
> mp <- data.frame(tx=rep(c("Placebo","6-MP"), c(21,21)),
                   time=c(placebo.time, mp.time),
                   status=c(placebo.status, mp.status))
> summary(mp)
```

On peut vérifier que les événements ont été correctement enregistrés dans ce nouveau tableau de données en vérifiant le format des données de survie sous R :

```
> with(mp, Surv(time, status))
```

Le tableau des événements dans le groupe 6-MP peut être construit à l'aide de la commande `survfit()`. On pourrait également limiter les calculs aux seules observations du groupe 6-MP à l'aide de l'option `subset()` :

```
> s <- survfit(Surv(time, status) ~ tx, data=mp)
> summary(s)
```

On utilisera le même résultat, contenu dans la variable `s` pour afficher la courbe de survie correspondante. Par défaut, lorsqu'il n'y a qu'un groupe R affiche automatiquement l'intervalle de confiance à 95 % :

```
> plot(survfit(Surv(time, status) ~ tx, data=mp,
               subset=tx=="6-MP"))
```

Pour afficher la fonction de risque cumulée, on ajoutera l'option `fun="cumhaz"` à la commande graphique précédente, soit en considérant les deux groupes :

```
> plot(s, fun="cumhaz")
```

3) Dans un essai randomisé, on a cherché à comparer deux traitements pour le cancer de la prostate. Les patients prenaient chaque jour par voie orale soit 1 mg de diethylstilbestrol (DES, bras actif) soit un placebo, et le temps de survie est mesuré en mois [COL 94]. La question d'intérêt est de savoir si la survie diffère entre les deux groupes de patients, et on négligera les autres variables présentes dans le fichier de données `prostate.dat`.

a) Calculer la médiane de survie pour l'ensemble des patients, et par groupe de traitement.

b) Quelle est la différence entre les proportions de survie dans les deux groupes à 50 mois ?

c) Afficher les courbes de survie pour les deux groupes de patients.

d) Effectuer un test du log-rank pour tester l'hypothèse selon laquelle le traitement par DES a un effet positif sur la survie des patients.

Pour importer les données, on utilisera la commande `read.table()`. On profitera de l'inspection préalable des données pour recoder la variable `Treatment` en variable qualitative :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> prostate <- read.table("prostate.dat", header=TRUE)
> str(prostate)
> head(prostate)
> prostate$Treatment <- factor(prostate$Treatment)
> table(prostate$Status)
```

Il n'est pas vraiment nécessaire de recoder `Status` en variable qualitative puisque l'on va utiliser cette variable conjointement avec `Time` pour l'analyse de survie. Dans un premier temps, pour transformer les données en données de survie, incluant la censure, il est nécessaire de passer par la commande `Surv()` du package `survival` :

```
> library(survival)
> with(prostate, Surv(time=Time, event=Status))
```

La médiane de survie s'obtient à l'aide de `survfit()`, qui est une commande générale permettant de manipuler les données de survie par la méthode de Kaplan-Meier ou le modèle de Cox. Si l'on ne tient pas compte des groupes de traitement (et plus généralement d'un quelconque cofacteur), on l'utilisera ainsi :

```
> survfit(Surv(Time, Status) ~ 1, data=prostate)
```

Pour prendre en compte le facteur traitement dans l'analyse, on utilisera en revanche :

```
> survfit(Surv(Time, Status) ~ Treatment, data=prostate)
```

Pour afficher les courbes de survie correspondant aux deux groupes de traitement, on procèdera comme suit :

```
> plot(survfit(Surv(Time, Status) ~ Treatment, data=prostate))
```

Le test du log-rank peut être obtenu de deux manières différentes. La plus simple consiste à utiliser la commande `survdiff()` pour comparer deux courbes de survie à partir de l'estimateur de Kaplan-Meier :

```
> survdiff(Surv(time=Time, event=Status) ~ Treatment,
           data=prostate)
```

L'autre solution consiste évidemment à utiliser une régression de Cox à l'aide de la commande `coxph()` :

```
> summary(coxph(Surv(time=Time, event=Status) ~ Treatment,
          data=prostate))
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Annexe A

Introduction à RStudio

Cette annexe offre un bref aperçu des fonctionnalités de RStudio. Le lecteur est invité à se référer aux livres consacrés à RStudio [VER 11, GAN 13] pour obtenir plus d'informations.

A.1. Prise en main

L'interface RStudio présente par défaut quatre panneaux arrangés dans une même fenêtre. Il est possible de maximiser le panneau hébergeant la console R afin d'afficher le maximum d'informations concernant les sorties générées par R.

RStudio propose également un navigateur de fichiers simplifié mais qui permet de changer le répertoire de travail (*More > Set As Working Directory*), à l'image de la commande `setwd()`.

A.2. Edition de scripts R

Lors du premier lancement de l'application, l'éditeur de script n'est pas visible et il est nécessaire de créer un nouveau script R afin d'y avoir accès. Pour créer un nouveau script R, il suffit d'aller dans le menu *File > New File > R Script* ou de cliquer sur le bouton correspondant dans la barre d'outils. Tout nouveau script est sauvegardé automatiquement, même si l'application se bloque et qu'il s'avère nécessaire de la redémarrer. L'extension `.R` est ajoutée automatiquement à tout nouveau script enregistré.

L'éditeur de script est composé d'une fenêtre principale et d'une barre d'outils. Les mots-clés du langage R sont surlignés d'une couleur différente et les parenthèses ouvrantes et fermantes sont automatiquement appariées. Un système d'auto-indentation automatique facilite également l'organisation du code R.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Depuis l'éditeur de script, il est possible d'envoyer n'importe quelle commande ou un ensemble de lignes à la console R. Pour cela, il suffit de cliquer sur le bouton *Run* en sélectionnant la ou les lignes à évaluer, ou alors simplement utiliser le raccourci clavier correspondant. Le bouton *Source* permet quant à lui d'envoyer l'intégralité des commandes du script R à la console. Toutes les commandes seront exécutées, mais de manière silencieuse, c'est-à-dire que les résultats ne seront pas nécessairement affichés à moins d'inclure des commandes de type `print()` à l'intérieur du script lui-même. Au contraire, le bouton *Source with echo* permettra d'afficher les résultats intermédiaires générés par R.

L'éditeur de script fournit d'autres fonctionnalités intéressantes. Par exemple, en appuyant sur la touche *TAB*, RStudio complètera automatiquement les noms de commande prédéfinies dans R, ainsi que le nom des variables ou fonctions définies dans le script R. La touche *F1* peut être utilisée pour accéder à l'aide en ligne relative à la commande sur laquelle se trouve le curseur. Il est également possible de commenter automatiquement une ou plusieurs lignes à partir de la barre d'outils ou d'un raccourci clavier dédié.

Une boîte de dialogue spécifique (*Tools > Import Dataset > From Text File*) permet d'importer des fichiers de données et celle-ci offre une prévisualisation des données une fois qu'elles seront importées dans R, à l'aide des options spécifiées dans la boîte de dialogue (séparateur de champ, séparateur décimal, etc.). Il s'agit essentiellement des options que l'on retrouve dans la commande `read.table()`.

A.3. Gestion de la session R

Concernant la gestion de l'espace de travail, RStudio offre des fonctionnalités très utiles. L'espace de travail peut être visualisé directement dans un panneau dédié (*Environment*). Celui-ci fournit essentiellement les mêmes informations que la commande `str()`, à savoir le type d'une variable (vecteur, matrice, data frame), le nombre d'éléments qu'elle contient, ainsi qu'un aperçu des premières valeurs. Dans le cas où la variable est un data frame, une icône en forme de tableur apparaît sur sa droite et il suffit de double cliquer dessus pour visualiser le data frame directement dans un tableur en lecture seule. Ce tableur offre des possibilités de filtrage et de tri des observations par colonnes.

Toutes les commandes R évaluées dans la console sont enregistrées dans un historique accessible *via* un panneau spécifique (*History*). Chacune de ces commandes peut être à nouveau envoyée à la console, voire même dans l'éditeur de script si un script R a été créé. Il suffit pour cela de sélectionner la ou les commandes d'intérêt et de cliquer sur le bouton correspondant dans la mini barre d'outils.

Pour installer des packages, RStudio propose un panneau spécifique (*Packages*) où il est possible de spécifier un ou plusieurs packages à télécharger et installer depuis

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

le site CRAN, ce qui remplace avantageusement l'usage de `install.packages()`. Il est également possible de charger des packages en cliquant simplement sur une case à cocher, ou de mettre à jour l'ensemble des packages déjà installés.

De nombreuses autres fonctionnalités sont disponibles dans RStudio. Mentionnons en particulier la possibilité de gérer des projets comprenant plusieurs fichiers de scripts R, avec un système de gestion de révision (SVN ou Git), ainsi que la création de rapports automatiques à l'aide du langage R Markdown (http://rmarkdown.rstudio.com).

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Annexe B

Les graphiques avec le package lattice

B.1. Pourquoi le package lattice ?

Il existe trois principaux systèmes graphiques sous R :

– les commandes graphiques de base, dans le package `graphics`, inclut la plupart des outils nécessaires pour la représentation uni- ou multivariée des données numériques et catégorielles. Les commandes typiques sont `plot()`, `hist()`, `barplot()`, `dotchart()`, `qqplot()`, etc. Voir pour des illustrations :

```
> demo(graphics)
```

– le package `lattice` (http://lmdvr.r-forge.r-project.org/) fournit des commandes équivalentes aux commandes de base, avec une interface de saisie unifiée, reposant sur l'usage de formules R, des options par défaut plus uniformes et, dans la plupart des cas, largement satisfaisantes pour des graphiques prêts à imprimer, ainsi que la construction automatique de graphique dits « en trellis » ;

– le package `ggplot2` (http://ggplot2.org), qui offre une implémentation de la grammaire des graphiques [WIL 05], se démarque des deux autres systèmes graphiques par l'usage d'une syntaxe et de commandes différentes, tout en offrant un degré de souplesse incomparable pour la création de graphiques statistiques. Les deux derniers systèmes graphiques reposent sur le même package `grid`.

Tout au long de cet ouvrage, nous avons privilégié le package lattice pour plusieurs raisons. Les relations entre variables sont spécifiées à l'aide de formules R (et d'une option `data=`), formules qui sont également utilisées dans les modèles statistiques (ANOVA et régression, entre autres) et certaines commandes de résumé numérique (`aggregate`, et `summary.formula` ou `summarize` du package `Hmisc`, voir chapitre suivant). Les options par défaut (annotation des axes, taille de police et de symboles graphiques, etc.) sont généralement satisfaisantes. Pour une visualisation rapide, il

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

existe des options intéressantes qui évitent d'avoir à combiner plusieurs commandes graphiques. Par exemple :

```
> xyplot(y ~ x, data=d, type=c("p", "g", "r"))
```

permet d'afficher un diagramme de dispersion (`y` en fonction de `x`) avec une droite de régression alors qu'en utilisant les graphiques de base on utiliserait une série de trois commandes :

```
> with(d, plot(x, y))  # ou plot(y ~ x, data=d)
> abline(lm(y ~ x, data=d))
> grid()
```

Les paragraphes qui suivent offrent un panorama des principales options. On se limite dans la plupart des cas à mentionner les principaux mots-clés afin que le lecteur puisse s'orienter dans l'aide en ligne (assez complexe pour ce package).

B.2. Principaux types de graphique

Dans ce qui suit, on présente les principales commandes graphiques pour les distributions uni et bivariées, avec éventuellement un conditionnement des résultats graphiques sur les niveaux d'une ou plusieurs variables catégorielles. Il est également possible de conditionner la relation entre deux variables par une troisième variable numérique mais ce cas ne sera pas traité (voir la notion de `shingle`). Dans chacun des cas, on indique entre parenthèses la commande de base équivalente. Par souci de simplicité, on omettra systématiquement le nom du data frame et on ne considérera aucun facteur de conditionnement ou de stratification explicitement. On utilisera comme variables illustratives `x`, `y` (variables numériques) et `z` (variable catégorielle).

1) `histogram(~ x)` (`hist(x)`) affiche un histogramme de fréquence, d'effectif (`type="count"`) ou de de densité (`type="density"`). Possibilité de conditionnement avec l'opérateur de formule `| z`.

2) `densityplot(~ x)` (`plot(density(x))`) affiche une courbe de densité non paramétrique. Possibilité de conditionnement avec l'opérateur de formule `| z` et `groups=z`.

3) `stripplot(~ x)` ou `stripplot(z ~ x)` (`plot(z ~ x)`) affiche un diagramme de dispersion univarié, éventuellement conditionné sur une variable apparaissant à gauche de l'opérateur `~` dans la formule. Possibilité de conditionnement avec l'opérateur de formule `| z` et `groups=z`.

4) `bwplot(~ x)` (`boxplot(x)`) affiche un diagramme en forme de boîtes à moustaches. Le conditionnement se fait dans la formule directement, par exemple `bwplot(z ~ x)` (barres orientées horizontalement) ou `bwplot(x ~ z)` (barres orientées verticalement). On peut également utiliser l'opérateur de formule `| z`.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

5) `xyplot(y ~ x)` (`plot(y ~ x)` ou `plot(x, y)`) affiche un diagramme de dispersion. Possibilité de conditionnement avec l'opérateur de formule `| z` et `groups=z`.

6) `barchart(xtabs(~ z))` (`barplot(table(x))`) affiche un diagramme en barres. Possibilité de conditionnement avec l'opérateur de formule `| z`, ou en fournissant directement un tableau d'effectifs ou de proportions à deux entrées, par exemple `barchart(xtabs(~ z1 + z2), stack=FALSE)`.

7) `dotplot(xtabs(~ z))` (`dotchart(table(x))`, ou `dotchart(x)` si `x` est déjà une série de données aggrégées, par exemple des moyennes ou des effectifs). Possibilité de conditionnement avec l'opérateur de formule `| z` et `groups=z`. Remarque : `dotplot(z ~ x)` est essentiellement équivalent à `stripplot(z ~ x)`.

8) `splom(data.frame(x, y))` ou `splom(d)`, avec `d` un `data.frame` contenant des variables numériques. Possibilité de conditionnement avec l'opérateur de formule `| z` et `groups=z`.

9) `qqmath(~ x)` affiche un diagramme de type quantile-quantile. Possibilité de conditionnement avec l'opérateur de formule `| z` et `groups=z`.

De manière générale, lorsqu'un facteur de classification supplémentaire est ajouté, le conditionnement peut se faire de deux manières. Si la variable est placée dans la formule R avec un opérateur `| z`, alors R produira autant de graphiques pour la relation que de niveaux pour la variable `z`. Voici une illustration (figure B.1) :

```
> xyplot(bwt ~ lwt | ui, data=birthwt)
```

L'autre option consiste à spécifier une option `groups=` qui permettra de superposer sur le même graphique des données conditionnées sur un facteur de classification. Dans l'exemple précédent, cela reviendrait à afficher l'ensemble des observations de poids dans le même diagramme de dispersion mais en utilisant des symboles différents selon la valeur de `ui`.

Dans certaines situations, il est intéressant de transformer le tableau de données à l'aide de la commande `melt()` du package `reshape2`. C'est le cas en particulier lorsque l'on souhaite afficher dans le même graphique des boîtes à moustaches pour différentes variables numériques d'un data frame mais sans considérer de facteur de classification. Avec les données sur les poids à la naissance, on pourrait vouloir afficher la distribution des variables `bwt`, `lwt` et `age`. Une commande telle que :

```
> bwplot(~ bwt + lwt + age, data=birthwt)
```

ne fonctionnera pas car dans ce cas R va afficher la distribution de la somme des trois variables (pour chaque observation). Au contraire, on utilisera (moyennant le problème de la standardisation des variables qui peut être réglé au préalable avec la commande `scale()`) une instruction de la forme (figure B.2) :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> library(reshape2)
> d <- melt(as.data.frame(sapply(birthwt[,c("bwt","lwt","age")],
                          scale)))
> bwplot(variable ~ value, data=d)
```

Cette notation permet également d'afficher la distribution de chaque variable dans des histogrammes séparés comme illustré dans la figure B.3. On voit donc l'importance de transformer le tableau de données initial pour réexprimer les relations entre variables :

```
> histogram(~ value | variable, data=d)
```

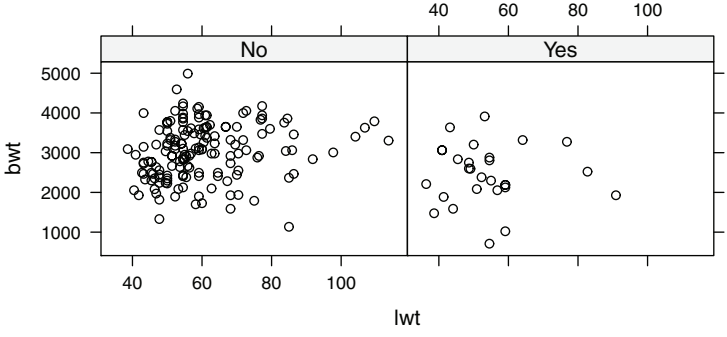


Figure B.1. *Diagramme de dispersion conditionné sur une variable qualitative*

B.3. Options générales pour la personnalisation des graphiques

B.3.1. *Type d'éléments graphiques et annotation*

Concernant l'annotation des axes ou la personnalisation des éléments graphiques (points, lignes, etc.), il est possible d'utiliser les mêmes options que pour les graphiques de base, à savoir :

– `main=`, `xlab=` et `ylab=` pour modifier le titre, et le nom des axes x et y, respectivement. L'option est une chaîne de caractères, par exemple `"Mon titre"` ;

– `pch=` et `cex=`, pour modifier le type (1 = cercle, 2 = carré, etc.) et la taille relative des symboles (les valeurs < 1 signifiant des symboles de taille réduite) ;

– `lty=` et `lwd=`, pour modifier le type (1 = trait plein, 2 = tirets, etc.) et la taille relative des lignes ;

– `col=` pour changer la couleur des points ou des lignes.

On peut se référer à l'aide en ligne pour plus d'informations : `help(par)`.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

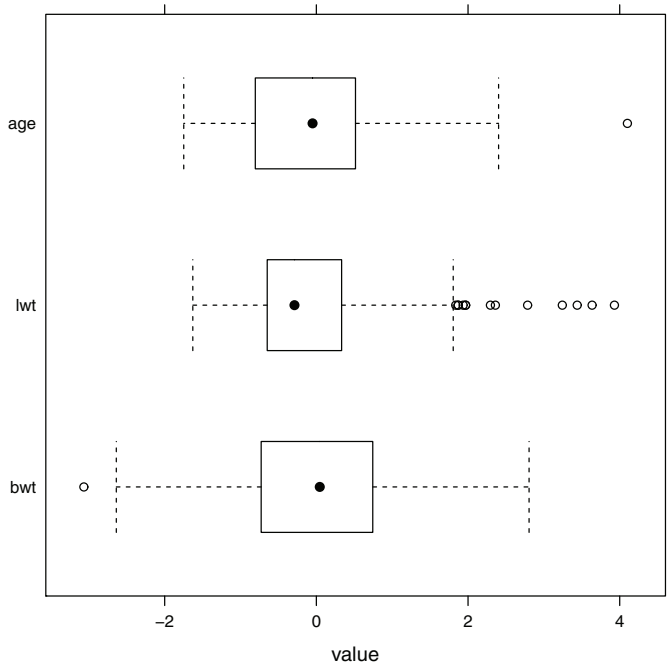


Figure B.2. *Représentation sous forme de boîtes à moustaches pour plusieurs variables*

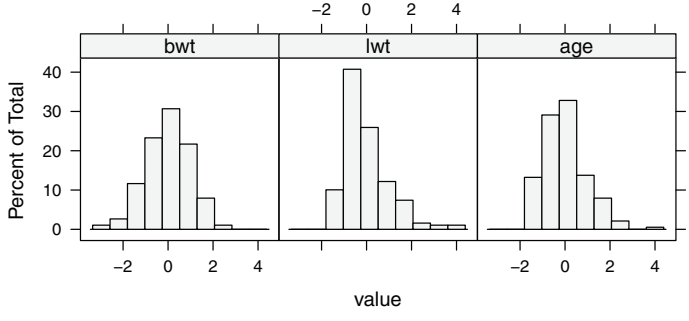


Figure B.3. *Histogrammes pour plusieurs variables*

Par exemple, la commande affichera un diagramme de dipersion où les poids des bébés sont représentés en ordonnées (axe vertical, libellé « Poids des bébés ») en

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

fonction du poids des mères, représentés en abscisses (axe horizontal, libéllé « Poids des mères »). Les observations seront représentées par des carrés (`pch=2`), de taille réduite (`cex=.6`, soit 60 % de la taille par défaut) et de couleur verte :

```
> xyplot(bwt ~ lwt, data=birthwt, pch=2, cex=.6, col="green",
         xlab="Poids des mères (kg)", ylab="Poids des bébés (g)")
```

B.3.2. *Gestion automatique des éléments graphiques à l'aide de thème*

Les options décrites dans le paragraphe précédent sont pratiques pour les graphiques simples, mais dans le cas où il existe un facteur de classification, il est plus commode d'utiliser un « thème » qui permet de gérer automatiquement les symboles associés à chaque niveau du facteur de classification et d'indiquer correctement les options graphiques appliquées dans la légende. Pour cela, on utilise la commande `simpleTheme()` du package `latticeExtra` (à installer au préalable) de la manière suivante :

```
> xyplot(bwt ~ lwt, data=birthwt, groups=ui,
         xlab="Poids des mères (kg)",
         ylab="Poids des bébés (g)",
         par.settings=simpleTheme(pch=c(1,3)),
         auto.key=TRUE)
```

Le thème de couleur global peut également être modifié en utilisant les commandes `trellis.par.set()`. Voici un exemple dans lequel on utilise un thème de couleur spécifique avec des fonds transparents pour les graphiques conditionnels :

```
> library(latticeExtra)
> trellis.par.set(custom.theme.2())
> trellis.par.set("strip.background"=list(col="transparent"))
```

Pour obtenir un thème noir et blanc ou niveaux de gris, on peut utiliser les instructions suivantes :

```
ltheme <- canonical.theme(color=FALSE)
ltheme$strip.background$col <- "transparent"
lattice.options(default.theme=ltheme)
```

On retiendra que ces modifications prennent effet sur le même « device » graphique : lorsque l'on ferme la fenêtre graphique ouverte par défaut par R, elles ne s'appliqueront plus.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

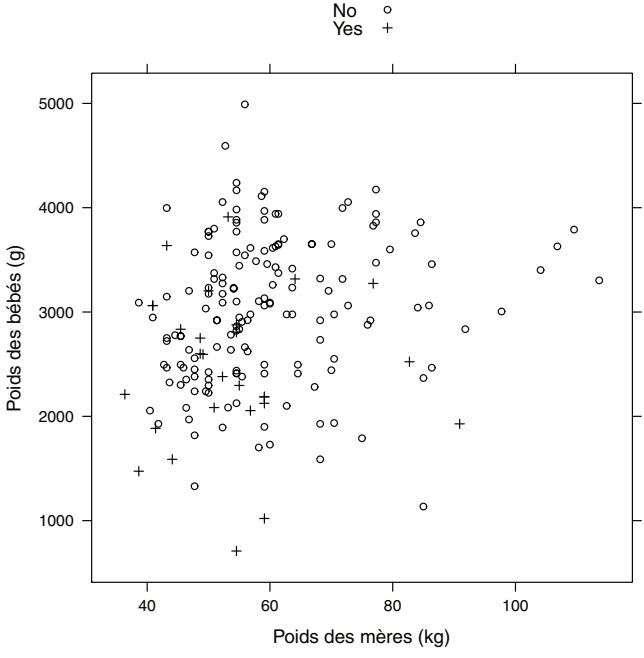


Figure B.4. *Diagramme de dispersion conditionné sur une variable qualitative*

B.3.3. *Légende graphique*

L'option `auto.key=` permet de générer une légende correspondant aux éléments graphiques présents dans la figure. Lorsque l'on utilise `simpleTheme()`, on n'a pas à se soucier de la couleur des points ou du type de ligne, il suffit de gérer le placement de la légende.

Il est possible d'indiquer dans quelle marge afficher la légende (par défaut, à droite), ou d'indiquer des coordonnées relatives (entre 0 et 1, pour chaque axe). Par exemple, `auto.key=list(space="top")` indique à R de placer la légende dans la marge supérieure du graphique. On peut également modifier l'arrangement des différents symboles de légende en ajoutant `columns=2` : les éléments seront ainsi disposés horizontalement sur deux emplacements (s'il y a plus de deux niveaux pour le facteur de conditionnement, les éléments de légende seront empilés sur les deux mêmes emplacements). Concernant le placement en coordonnées relatives, on remplacera `space=` par `corner=` en indiquant une liste pour les coordonnées (x, y) de la partie supérieure gauche du cadre de légende. Il existe plusieurs autres options pour l'option de légende.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Enfin, il est possible d'ajouter un titre de légende avec une instruction du type `title=`. En reprenant l'un des exemples précédents, les commandes suivantes permettent de produire le graphique reproduit dans la figure B.5 :

```
> xyplot(bwt ~ lwt, data=birthwt, groups=smoke,
         par.settings=simpleTheme(pch=c(1,3),
                                  col=c("gray50", "black")),
         auto.key=list(corner=c(0.7,0.95), title="Smoking",
                       cex=.8, cex.title=0.8, border=TRUE))
```

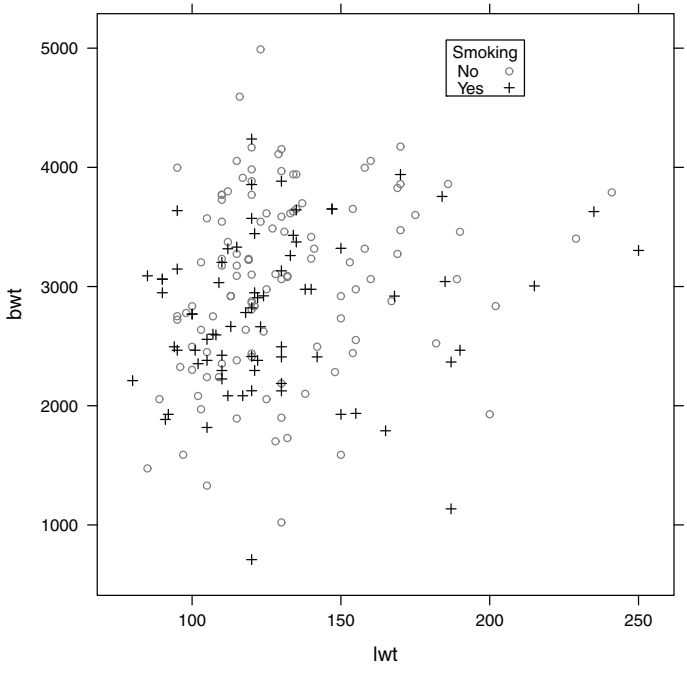


Figure B.5. *Spécification d'une légende dans un graphique lattice*

B.3.4. *Combinaison d'éléments graphiques*

Dans le cas des diagrammes de dispersion produits avec `xyplot()`, l'option `type=` est utile pour combiner différents éléments graphiques : `"p"` affiche les données sous forme de points ; `"l"` permet de relier chaque observation par des lignes ; `"b"` affiche à la fois des points et des lignes ; `"smooth"` permet d'afficher une courbe loess, éventuellement pour chaque niveau d'un facteur de classification indiqué dans une option `groups=` ; `"r"` affiche la droite de régression (éventuellement plusieurs

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

droites de régression selon l'option de conditionnement `groups=`); `"g"` fait apparaître une grille alignée sur les intervalles définis pour les deux axes.

B.3.5. *Organisation spatiale de plusieurs graphiques*

L'option `layout=` permet de spécifier l'arrangement des graphiques multiples en présence d'une variable de conditionnement. On indique *via* cette option sur combien de colonnes et de lignes les graphiques doivent être disposés. Voici un exemple d'utilisation (figure B.6) :

```
> xyplot(bwt ~ lwt | race, data=birthwt, type=c("p", "smooth"),
         layout=c(1,3))
```

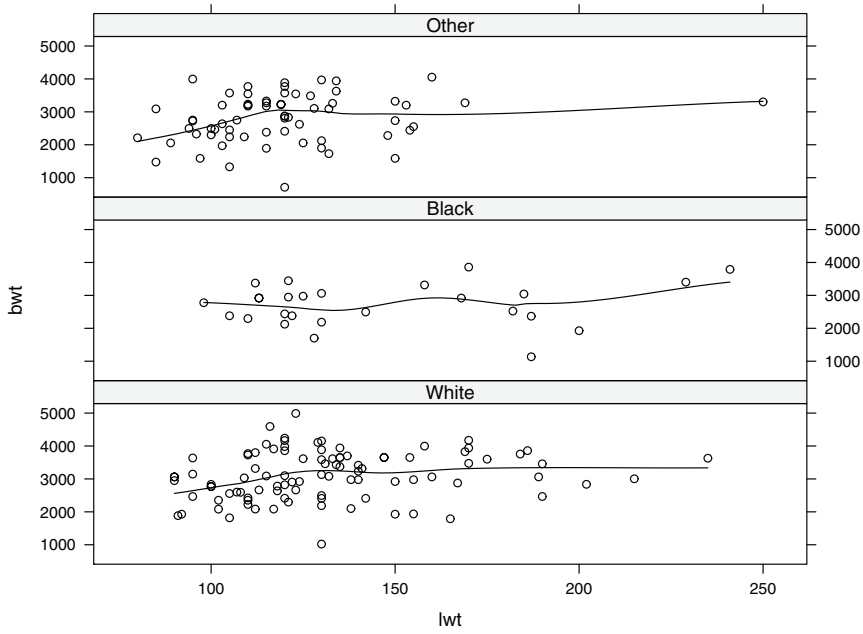


Figure B.6. *Réarrangement des sous-graphiques lattice*

La commande `grid.arrange()` du package `grid` permet quant à elle de combiner différents graphiques sur une même fenêtre.

B.4. Options spécifiques

Certaines commandes graphiques possèdent des options qui leur sont propres. Par exemple, les commandes `stripplot()` et `xyplot()` offrent toutes les deux

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

la possibilité d'ajouter un léger décalage (horizontal et/ou vertical) des points afin d'éviter les problèmes de superposition dans le cas des grands échantillons. Avec `stripplot()`, on utiliserait par exemple (figure B.7) :

```
> stripplot(~ bwt, data=birthwt, jitter.data=TRUE, factor=2)
```

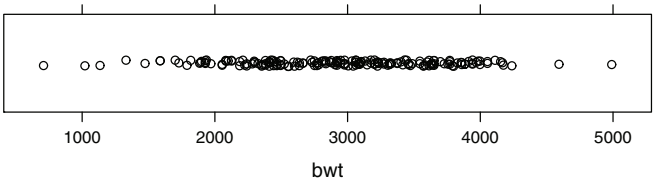


Figure B.7. *Diagramme de dispersion univarié*

La plupart des options spécifiques des principales commandes graphiques peuvent se retrouver en demandant l'aide pour la commande `panel.*` où `*` désigne généralement la commande graphique principale. Par exemple, `help(panel.xyplot)` fournit la liste des options possibles pour une représentation de la relation entre deux variables sous forme de diagramme de dispersion.

B.5. Composition graphique

Une commande telle que :

```
> xyplot(bwt ~ lwt, data=birthwt, type=c("p", "g", "r"))
```

peut en fait se décomposer de la manière suivante : on construit un diagramme de dispersion en affichant des points (`"p"`) pour représenter les paires de valeurs observées pour les variables `bwt` et `lwt`, puis on superpose une grille alignée sur les coordonnées des axes, et enfin on trace une droite de régression correspondant au modèle `lm(bwt ~ lwt)`. Il est possible de reproduire ces étapes à l'aide des instructions suivantes :

```
> xyplot(bwt ~ lwt, data=birthwt,
         panel=function(x, y, ...) {
           panel.xyplot(x, y, ...)
           panel.grid(...)
           panel.lmline(x, y, ...)
         })
```

L'instruction `panel=function(x, y, ...) { ... }` se charge de l'affichage de différents éléments graphiques, à partir des données `x` et `y` correspondant aux

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

variables `lwt` et `btw`, respectivement. Ensuite, on ajoute les éléments graphiques suivants : des points dont les coordonnées sont définies par `x` et `y` (donc les valeurs prises par `lwt` et `btw`) avec `panel.xyplot()`, une grille alignée sur les coordonnées affichées des axes avec `panel.grid()`, et enfin une droite de régression avec la commande `panel.lmline()`.

Il existe des cas plus complexes que l'exemple précédent, notamment lorsque la commande graphique inclut une variable de conditionnement produisant différents graphiques (opérateur `|`) ou différents types de points ou lignes (options `groups=`). Ces cas se gèrent en ajoutant une option `subscripts=` ou `groups=` dans la déclaration de la fonction `panel`. Voir aussi les commandes `packet.number()` ainsi que `panel.number()` dans l'aide en ligne.

B.6. Pour aller plus loin

La référence définitive sur le package `lattice` reste l'ouvrage de Deepayan Sarkar [SAR 08]. On trouvera également des informations utiles sur le site associé, en particulier les commandes R permettant de générer différents types de graphiques : http://lmdvr.r-forge.r-project.org/figures/figures.html.

La package `latticeExtra` (http://latticeextra.r-forge.r-project.org) offre de nombreuses extensions aux commandes `lattice`, en particulier pour les graphiques multivariés et la personnalisation (thème) des graphiques.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Annexe C

Les packages Hmisc et rms

Les packages `Hmisc` et `rms` offrent un ensemble de fonctionnalités additionnelles pour la gestion des données, la production de tableaux et de graphiques de résumé descriptifs de variables numériques et qualitatives et la modélisation (régression linéaire, logistique et analyse de données de survie).

C.1. Les commandes essentielles de Hmisc

C.1.1. *Gestion de données*

Pour la gestion de données, `Hmisc` offre plusieurs fonctionnalités intéressantes. Ce package fournit des commandes permettant d'importer des fichiers CSV (`csv.get()`), SAS (`sas.get()`), SPSS (`spss.get()`) ou Stata (`stata.get`). Ces commandes se chargent de convertir les noms de variable en minuscules, de recoder les dates si besoin. Les deux dernières commandes reposent en fait sur les commandes du package `foreign`. Avant d'utiliser ce package, il est nécessaire de le charger au préalable avec la commande `library(Hmisc)`.

La commande `contents()` fournit un résumé du format de représentation des variables et de la présence éventuelle de valeurs manquantes :

```
> birthwt$age[5] <- NA
> birthwt$ftv[sample(1:nrow(birthwt), 5)] <- NA
> contents(birthwt)

Data frame:birthwt 189 observations and 12 variables
Maximum # NAs:5

      Levels   Class Storage NAs
low        2         integer   0
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
age                  integer   1
lwt                   double   0
race       3         integer   0
smoke      2         integer   0
ptl                  integer   0
ht         2         integer   0
ui         2         integer   0
ftv                  integer   5
bwt                  integer   0
ftv2       3         integer   0
ftv3       3 ordered integer   0

+--------+----------------+
|Variable|Levels          |
+--------+----------------+
|  low   |No,Yes          |
|  smoke |                |
|  ht    |                |
|  ui    |                |
+--------+----------------+
|  race  |White,Black,Other|
+--------+----------------+
|  ftv2  |0,1,2+          |
|  ftv3  |                |
+--------+----------------+
```

On a vu qu'il était possible d'associer des étiquettes aux modalités d'une variable qualitative avec l'option `levels=` de la commande `factor`, ou directement avec la commande `levels()`. Il est également possible d'associer des étiquettes aux variables, et des unités de mesure, lorsque cela s'applique, grâce aux commandes `label()` et `units` :

```
> label(birthwt$age) <- "Age de la mère"
> units(birthwt$age) <- "années"
> label(birthwt$bwt) <- "Poids du bébé à la naissance"
> units(birthwt$bwt) <- "grammes"
```

La commande `label()` peut également servir à fournir des informations sur un data frame, par exemple :

```
> label(birthwt, self=TRUE) <- "Données sur les poids à la
                                naissance de Hosmer et Lemeshow."
```

Ces étiquettes peuvent ensuite être associées automatiquement dans les tableaux ou graphiques produits par `Hmisc`. La commande `describe` fournit une synthèse

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

descriptive de l'ensemble des variables d'un data frame ou d'une liste de variables, à l'image d'un dictionnaire de données ou « codebook » :

```
> describe(subset(birthwt, select=c(age, race, low)))
subset(birthwt, select = c(age, race, low))

 4  Variables      189  Observations
---------------------------------------------------------------
age : Age de la mère [années]
      n missing unique Info  Mean .05 .10 .25 .50 .75 .90 .95
    188       1     24    1 23.27  16  17  19  23  26  31  32

lowest : 14 15 16 17 18, highest: 33 34 35 36 45
---------------------------------------------------------------
race
      n missing  unique
    189       0       3

White (96, 51%), Black (26, 14%), Other (67, 35%)
---------------------------------------------------------------
low
      n missing  unique
    189       0       2

No (130, 69%), Yes (59, 31%)
---------------------------------------------------------------
```

La commande `cut2()` remplace avantageusement la commande `cut()` que l'on trouve souvent combinée à `quantile()` pour convertir des variables numériques en variables qualitatives avec des classes équilibrées (et elle permet d'éviter d'avoir à se rappeler l'option `include.lowest=TRUE`) :

```
> table(cut2(birthwt$lwt, g=4))

[36.4, 50.9) [50.9, 55.5) [55.5, 64.1) [64.1,113.6]
          53           43           46           47
> table(cut2(birthwt$age, g=3, levels.mean=TRUE))

18.074 23.091 30.019
    68     66     54
```

Il est également possible de spécifier le nombre de groupes désirés (`g=`) ou le nombre minimal d'observations par groupe (`m=`). L'option `levels.mean=` permet de renvoyer comme niveau de facteur le centre de classe plutôt que les bornes de l'intervalle.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Pour les données de cohorte ou de survie, la commande `event.chart()` fournit des représentations graphiques intéressantes pour visualiser les durées de suivi, les censures, etc. Voir l'aide en ligne et les exemples fournis :

```
> example(event.chart)
```

Il existe également un certain nombre de commandes permettant d'effectuer de l'imputation simple ou multiple (voir `impute()` et `aregImpute()`). Un exemple d'imputation par la médiane pour les données supprimées dans la variable `ftv` plus haut est donné ci-après :

```
> ftvi <- impute(birthwt$ftv)
> summary(ftvi)

 5 values imputed to 0

   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.0000  0.0000  0.0000  0.7725  1.0000  6.0000
```

Les observations imputées sont signalées d'une astérisque lorsque l'on affiche l'ensemble des valeurs prises par la variable.

C.1.2. *Statistiques descriptives*

La commande `summarize()` fonctionne sur le même principe que `aggregate()` et permet de fournir pour une variable numérique des résumés stratifiés selon une ou plusieurs variables qualitatives. A la différence de `aggregate()`, il est possible de calculer plusieurs statistiques et de les stocker dans des colonnes différentes d'un même tableau de résultats (`aggregate()` stocke en fait tous ses résultats dans une seule et même colonne, ce qui n'est pas gênant pour l'affichage mais ne facilite pas l'exploitation des résultats) :

```
> f <- function(x) c(mean=mean(x), sd=sd(x))
> with(birthwt, summarize(bwt, race, f))
   race      bwt       sd
3 White 3102.719 727.8861
1 Black 2719.692 638.6839
2 Other 2805.284 722.1944
```

La commande `summary()` (en fait, `summary.formula()`) peut s'utiliser avec une notation par formule décrivant les relations entre plusieurs variables. On distingue trois principales méthodes (`method=`) : `"response"`, où l'on résume une variable numérique selon les niveaux d'une ou plusieurs variables (dans le cas de variables numériques, `Hmisc` recodera automatiquement la variable en quatre classes équilibrées), `"cross"` pour calculer les moyennes conditionnelles et marginales d'une

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

ou plusieurs variables réponse en fonction de variables qualitatives ou numériques (trois au maximum), et `"reverse"` pour résumer de manière univariée un ensemble de variables numériques (trois quartiles) ou qualitatives (effectif et proportion) selon les niveaux d'un facteur. La position des variables par rapport à l'opérateur de liaison `~` de la formule joue un rôle particulièrement important pour déterminer le type de résumé à réaliser selon la méthode sélectionnée. Pour les méthodes `"response"` et `"reverse"`, on peut coupler la commande `plot()` directement avec le résultat renvoyé par `summary()` pour avoir une représentation graphique des résultats. On peut également fournir une formule et indiquer à R quelle commande appliquer pour résumer la structure de données. Par exemple, dans le cas `low ~ race + ht`, l'option `fun=table` permettra de construire automatiquement les deux tableaux de contingence correspondant au croisement des variables `low` avec `race` et `low` avec `ht`.

Voici quelques illustrations. Sans préciser d'option, `summary()` permet de résumer sous forme de moyenne une variable numérique en fonction d'une liste de variables explicatives numériques ou catégorielles. Dans le cas où la variable explicative est numérique, `Hmisc` considère les quartiles de cette variable. L'effectif associé à chaque modalité des variables est reporté dans une colonne séparée :

```
> summary(bwt ~ race + ht + lwt, data=birthwt)
Poids du bébé à la naissance    N=189

+-------+------------+---+--------+
|       |            |N  |bwt     |
+-------+------------+---+--------+
|race   |White       | 96|3102.719|
|       |Black       | 26|2719.692|
|       |Other       | 67|2805.284|
+-------+------------+---+--------+
|ht     |No          |177|2972.232|
|       |Yes         | 12|2536.833|
+-------+------------+---+--------+
|lwt    |[36.4, 50.9)| 53|2656.340|
|       |[50.9, 55.5)| 43|3058.721|
|       |[55.5, 64.1)| 46|3074.609|
|       |[64.1,113.6]| 47|3037.957|
+-------+------------+---+--------+
|Overall|            |189|2944.587|
+-------+------------+---+--------+
```

Il est possible de résumer la moyenne de plusieurs variables réponses, associées *via* la commande `cbind()`, et jusqu'à deux variables explicatives en précisant l'option `method="cross"`. `Hmisc` indiquera l'effectif associé à chaque cellule, ainsi que le nombre de valeurs manquantes :

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
> summary(cbind(lwt, age) ~ race + bwt, data=birthwt,
method="cross")

 mean by race, bwt

+-------+
|N      |
|Missing|
|lwt    |
|age    |
+-------+
+-----+-----------+-----------+-----------+-----------+--------+
| race|[ 709,2424)|[2424,3005)|[3005,3544)|[3544,4990]|   ALL  |
+-----+-----------+-----------+-----------+-----------+--------+
|White|   19      |   23      |   20      |   33      | 95     |
|     |  0        |  1        |  0        |  0        |1       |
|     |  55.55024 |  57.66798 |  62.22727 |  63.25069 |60.14354|
|     |  22.73684 |  24.78261 |  24.50000 |  24.90909 |24.35789|
+-----+-----------+-----------+-----------+-----------+--------+
|Black|    9      |    9      |    6      |    2      | 26     |
|     |  0        |  0        |  0        |  0        |0       |
|     |  65.10101 |  59.69697 |  70.83333 |  93.40909 |66.73077|
|     |  23.44444 |  20.88889 |  20.00000 |  20.50000 |21.53846|
+-----+-----------+-----------+-----------+-----------+--------+
|Other|   20      |   16      |   19      |   12      | 67     |
|     |  0        |  0        |  0        |  0        |0       |
|     |  51.22727 |  52.95455 |  58.89952 |  55.34091 |54.55224|
|     |  22.20000 |  22.68750 |  22.26316 |  22.50000 |22.38806|
+-----+-----------+-----------+-----------+-----------+--------+
|ALL  |   48      |   48      |   45      |   47      |188     |
|     |  0        |  1        |  0        |  0        |1       |
|     |  55.53977 |  56.47727 |  61.96970 |  62.51451 |59.06190|
|     |  22.64583 |  23.35417 |  22.95556 |  24.10638 |23.26596|
+-----+-----------+-----------+-----------+-----------+--------+
```

L'option `method="reverse"` permet de fournir le résumé d'un ensemble de variables, numériques ou catégorielles, selon les niveaux d'un `factor` apparaissant à gauche d'une formule. Les variables numériques sont résumées à l'aide des trois quartiles, et les variables catégorielles sont résumées en termes d'effectif et de proportion. Il est possible d'afficher la moyenne et l'écart-type pour les variables numériques en utilisant la commande `print()` et en ajoutant l'option `prmsd=TRUE`. Le nombre de décimales affichées se contrôle *via* la commande `print()` également (par exemple, `round=1`). Enfin, l'option `test=TRUE` associée à `method="reverse"` ajoute automatiquement une colonne faisant figurer des tests de Wilcoxon dans le cas

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

de la comparaison de deux groupes pour les variables numériques, et des tests du χ^2 de Fisher dans le cas des variables catégorielles. Voici un exemple d'utilisation :

```
> summary(low ~ race + age + ui, data=birthwt, method="reverse",
          overall=TRUE)

Descriptive Statistics by low

+------------------+---+---------+---------+---------+
|                  |N  |No       |Yes      |Combined |
|                  |   |(N=130)  |(N=59)   |(N=189)  |
+------------------+---+---------+---------+---------+
|race :            |189|   56%   |   39%   |   51%   |
|White             |   |   (73)  |   (23)  |   (96)  |
+------------------+---+---------+-------------------+
|    Black         |   |   12%   |   19%   |   14%   |
|                  |   |   (15)  |   (11)  |   (26)  |
+------------------+---+---------+-------------------+
|    Other         |   |   32%   |   42%   |   35%   |
|                  |   |   (42)  |   (25)  |   (67)  |
+------------------+---+---------+-------------------+
|Age de            |188|  19.0/  |  19.5/  |  19.0/  |
|la mère           |   |  23.0/  |  22.0/  |  23.0/  |
|[années]          |   |  28.0   |  25.0   |  26.0   |
+------------------+---+---------+-------------------+
|ui : Yes          |189|   11%   |   24%   |   15%   |
|                  |   |   (14)  |   (14)  |   (28)  |
+------------------+---+---------+-------------------+
```

Pour obtenir un tri croisé multiple, il suffit de préciser l'option `fun=table` :

```
> summary(low ~ race + ht, data=birthwt, fun=table)
low    N=189

+-------+-----+---+---+---+
|       |     |N  |No |Yes|
+-------+-----+---+---+---+
|race   |White| 96| 73|23 |
|       |Black| 26| 15|11 |
|       |Other| 67| 42|25 |
+-------+-----+---+---+---+
|ht     |No   |177|125|52 |
|       |Yes  | 12|  5| 7 |
+-------+-----+---+---+---+
|Overall|     |189|130|59 |
+-------+-----+---+---+---+
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Description by low birth weight status (low)

		No	Yes
		$N = 130$	$N = 59$
age	years	19.0 23.0 28.0	19.5 22.0 25.0
lwt	pounds	113.0 123.5 147.0	104.0 120.0 130.0
race : White		56% (73)	39% (23)
Black		12% (15)	19% (11)
Other		32% (42)	42% (25)
smoke : Yes		34% (44)	51% (30)
ptl : 1+		9% (12)	31% (18)
ht : Yes		4% (5)	12% (7)
ui : Yes		11% (14)	24% (14)
ftv : Yes		51% (66)	39% (23)

$a\ b\ c$ represent the three quartiles.

Figure C.1. *Exemple de tableau statistique généré par Hmisc*

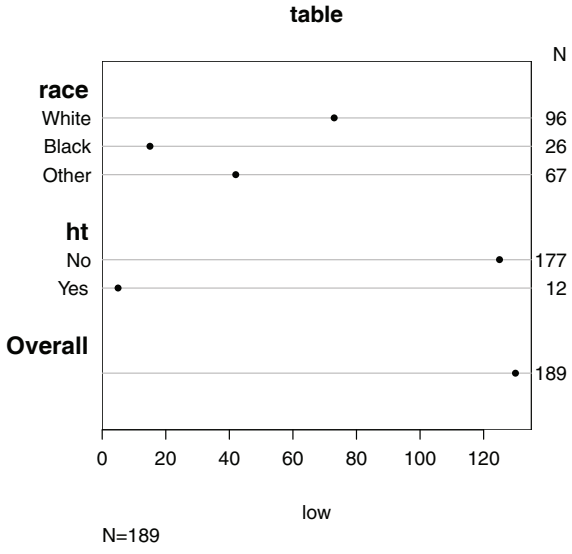


Figure C.2. *Exemple de figure générée par Hmisc à partir d'un tableau*

L'ensemble des tableaux produits par `Hmisc` peuvent être exportés automatiquement au format LaTeX, et donc reproduits en version PDF. Un exemple d'un tel tableau de résumé est donné dans la figure C.1. Par ailleurs, la plupart des tableaux de résumé `Hmisc` peuvent également être représentés sous forme graphique, en utilisant la commande `plot()`, comme illustré dans la figure C.2.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

C.1.3. *Commandes graphiques*

Le package `Hmisc` repose pour l'essentiel sur les graphiques `lattice`, mais il fournit des commandes « plus intégrées ». Parmi les commandes graphiques fournies par `Hmisc`, on distingue les diagrammes en points (`Dotplot()` et `dotchart2()`), les diagrammes en boîtes à moustaches (`panel.bpplot()` à utiliser avec la commande usuelle `bwplot()`) et les diagrammes de dispersion (`xyplot()`). On se contentera de présenter un exemple pour ce dernier cas (figure C.3) :

```
> f <- function(x) c(mean(x), mean(x) + c(-1, 1)
  * sd(x)/sqrt(length(x)))
> bwtmeans <- with(birthwt, summarize(bwt, llist(race,
  smoke), f))
> names(bwtmeans)[4:5] <- c("lwr", "upr")
> bwtmeans
   race smoke      bwt      lwr      upr
5 White    No 3428.750 3321.699 3535.801
6 White   Yes 2826.846 2739.970 2913.722
1 Black    No 2854.500 2699.186 3009.814
2 Black   Yes 2504.000 2302.545 2705.455
3 Other    No 2815.782 2720.133 2911.430
4 Other   Yes 2757.167 2523.327 2991.006
> xYplot(Cbind(bwt, lwr, upr) ~ numericScale(race,
        label="Ethnicité de la mère") | smoke,
        data=bwtmeans, type="b", ylim=c(2200, 3600),
        scales=list(x=list(at=1:3, labels=levels(birthwt$race))))
```

On notera que `Cbind()` prend un c majuscule, pour différencier cette commande de la commande de base, et que si elle existe l'étiquette de la variable remplace automatiquement le nom de la variable (c'est le cas ici pour la variable `bwt`).

Il est également possible d'annoter automatiquement les différentes courbes, segments ou points définis par une variable de conditionnement à l'aide de l'option `keys=`, comme dans l'exemple suivant (figure C.4) :

```
> xYplot(Cbind(bwt, lwr, upr) ~ numericScale(race),
         groups= smoke, data=bwtmeans, type="l", keys="lines")
```

Pour modifier l'affichage de l'axe des abscisses et remplacer les valeurs numériques par les étiquettes du facteur, on pourra ajouter l'option :

```
scales = list(x = list(at = 1:3, labels = levels(birtwht$race)))
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

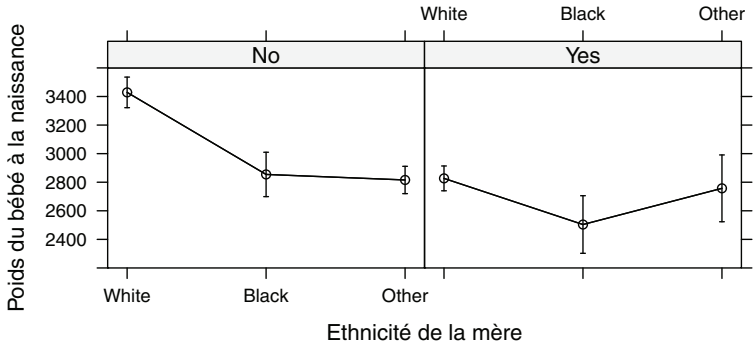


Figure C.3. *Exemple de graphique de moyennes conditionnées sur deux variables*

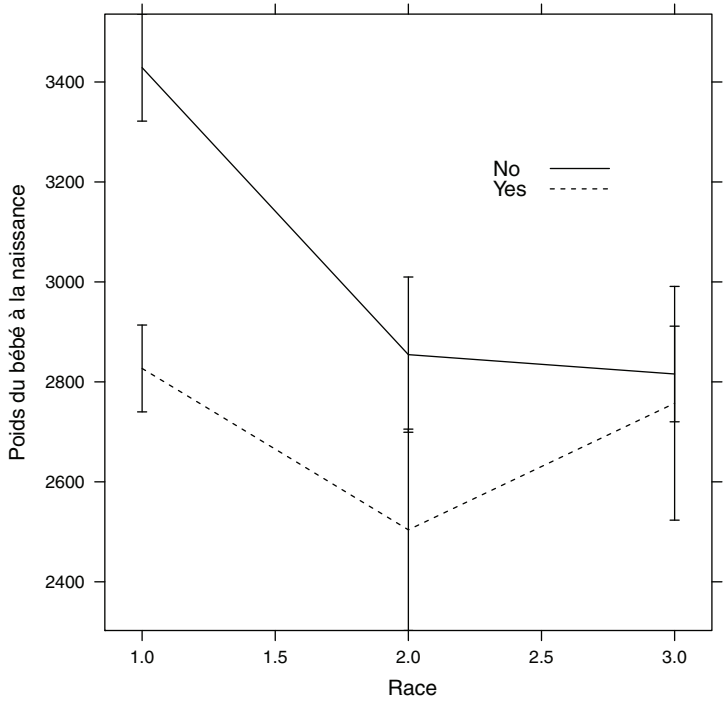


Figure C.4. *Exemple de graphique de moyennes conditionnées sur deux variables avec placement de légende*

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

C.2. Les commandes essentielles de rms

Pour utiliser les commandes du package `rms`, on chargera au préalable le package à l'aide de la commande `library()`. Notons que ce package charge automatiquement le package `Hmisc` :

```
> library(rms)
```

Pour la régression linéaire, il s'agit de la commande `ols()`. Le principe d'utilisation reste le même que pour la commande `lm()` : on spécifie à l'aide d'une formule le rôle de chaque variable (variable réponse ~ variables explicatives) et le data frame dans lequel chercher les données. Dans son usage le plus simple, on se contente d'afficher les résultats renvoyés par `ols()` avec la commande `print()` ou en tapant simplement le nom de la variable auxiliaire dans laquelle on a stocké les résultats du modèle. Pour obtenir des informations additionnelles, en particulier concernant les effets marginaux, les résidus du modèle, et profiter de fonctionnalités graphiques étendues, il est nécessaire d'utiliser la commande `datadist()` pour stocker les prédicteurs du modèle dans une structure à part. On disposera alors de commandes telles que `summary()` ou `plot()`.

Pour modéliser le poids des bébés en fonction de différents indicateurs de risque chez la mère, on utiliserait par exemple :

```
> m <- ols(bwt ~ age + lwt + race + ftv, data=birthwt, x=TRUE)
> m

Linear Regression Model

ols(formula = bwt ~ age + lwt + race + ftv, data = birthwt,
x = TRUE)

Frequencies of Missing Values Due to Each Variable
 bwt  age  lwt race  ftv
   0    1    0    0    5


                   Model Likelihood     Discrimination
                      Ratio Test           Indexes
Obs        183     LR chi2     17.66    R2       0.092
sigma 712.1949     d.f.            5    R2 adj   0.066
d.f.       177     Pr(> chi2) 0.0034    g      254.865

Residuals
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
Poids du bébé à la naissance [grammes]
     Min       1Q   Median       3Q      Max
-2097.25  -440.02    40.41   490.41  1919.63

           Coef      S.E.     t     Pr(>|t|)
Intercept  2482.0826 324.5729  7.65 <0.0001
age          -0.1490  10.5951 -0.01 0.9888
lwt          10.4217   4.0481  2.57 0.0109
race=Black -464.5878 164.4224 -2.83 0.0053
race=Other -263.2467 119.0400 -2.21 0.0283
ftv          12.3299  51.0652  0.24 0.8095
```

Pour la régression logistique (incluant les variables réponse à modalités ordonnées), on utilisera la commande `lrm()`. Cette commande choisit automatiquement le modèle selon le nombre de niveaux de la variable réponse : dans le cas où il y en a plus de deux, un modèle à odds proportionnels est utilisé. Le principe d'utilisation de cette commande est le même que dans le cas de `ols()`. Dans l'exemple suivant, on utilise la commande `datadist()` qui, lorsqu'elle est associée aux modèles de régression de `rms`, permet de calculer automatiquement des tailles d'effet pour les prédicteurs. On peut fournir soit une liste de variables (les prédicteurs entrant dans le modèle), soit directement le data frame sur lequel on travaille. Il est important de faire suivre cette commande par `options()`, comme indiqué ci-après, afin de rendre les données accessibles aux commandes `rms` :

```
> ddist <- datadist(birthwt)
> options(datadist='ddist')
> m <- lrm(low ~ age + lwt + race + ftv, data=birthwt)
> print(m)

Logistic Regression Model

lrm(formula = low ~ age + lwt + race + ftv, data = birthwt)

Frequencies of Missing Values Due to Each Variable
 low  age  lwt race  ftv
   0    1    0    0    5

                  Model Likelihood  Discrimination  Rank Discrim.
                     Ratio Test         Indexes        Indexes
Obs         183   LR chi2     12.26  R2     0.091    C       0.657
 No         125   d.f.            5  g      0.680    Dxy     0.314
 Yes         58   Pr(> chi2) 0.0314  gr     1.974    gamma   0.316
max |deriv| 1e-07                    gp     0.136    tau-a   0.137
                                     Brier  0.203
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
           Coef    S.E.   Wald Z Pr(>|Z|)
Intercept   1.2883 1.0892  1.18  0.2369
age        -0.0226 0.0342 -0.66  0.5088
lwt        -0.0318 0.0146 -2.19  0.0288
race=Black  1.0074 0.5024  2.01  0.0450
race=Other  0.4831 0.3695  1.31  0.1910
ftv        -0.0396 0.1681 -0.24  0.8140
```

Comme on peut le voir, la sortie produite par R lorsque l'on utilise `lrm()` au lieu de `glm()` contient des informations additionnelles, notamment en ce qui concerne la capacité prédictive et discriminante du modèle logistique. Voici une application de la commande `summary()` qui fournira des informations complémentaires en exploitant directement les données stockées à l'aide de `datadist()` :

```
> summary(m)
```

Un test du rapport de vraisemblance peut être utilisé pour comparer un modèle réduit dans lequel on exclut les facteurs `age` et `ftv` au modèle initial à l'aide de `lrtest()`. Dans les instructions suivantes, on construit un deuxième modèle dans lequel on retire les variables `age` et `ftv` à l'aide de la commande `update()`. Cette commande attend un modèle initial, ici `m`, et une formule modifiée par rapport à la formule de ce modèle : dans notre cas, on conserve la même variable réponse, et on enlève du membre droit de la formule les deux variables en question :

```
> m2 <- update(m, . ~ . - age - ftv)
> lrtest(m2, m)
Likelihood ratio test for MLE method
Chi-squared d.f. =  , P value =
```

Enfin, il est également possible d'obtenir un tableau d'analyse de variance ou de déviance pour le modèle de régression à l'aide de la commande `anova()` :

```
> anova(m2)
                Wald Statistics          Response: low

 Factor     Chi-Square d.f. P
 lwt         5.59      1    0.0181
 race        5.40      2    0.0671
 TOTAL      10.13      3    0.0175
```

La commande `Predict()` permet de calculer automatiquement les valeurs prédites par un modèle de régression `rms`. Voici un exemple d'utilisation où l'on souhaite calculer les valeurs de log-odds prédites par le modèle `m2` lorsque le poids de la mère varie entre 40 et 80 kg, par pas de 5 kg :

```
> pm2 <- Predict(m2, lwt=seq(40, 80, by=5), race)
> head(pm2)
```

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

```
  lwt  race       yhat     lower      upper
1  40 White -0.5338795 -1.206583  0.1388239
2  45 White -0.7013337 -1.284054 -0.1186135
3  50 White -0.8687878 -1.383708 -0.3538673
4  55 White -1.0362419 -1.515065 -0.5574193
5  60 White -1.2036960 -1.685304 -0.7220877
6  65 White -1.3711502 -1.893807 -0.8484937

Response variable (y): log odds

Limits are 0.95 confidence limits
```

Comme l'on peut le constater, R fournit les valeurs prédites sur l'échelle du log-odds, avec un intervalle de confiance à 95 %. Il est ensuite relativement aisé de représenter graphqiuement les prédictions de ce modèle à l'aide de `xYplot()` :

```
> xYplot(Cbind(yhat,lower,upper) ~ lwt | race, data=pm2,
         layout=c(3,1), method="filled bands", type="l",
         col.fill=gray(.95))
```

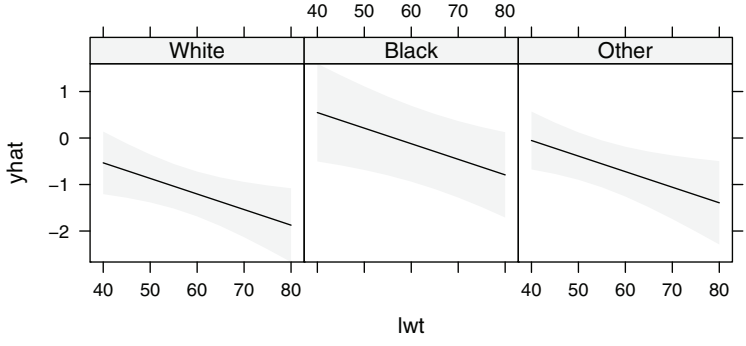


Figure C.5. *Utilisation de la commande xYplot()*

Pour la régression de Cox, il s'agit de la commande `cph()`. La représentation des données de survie passe toujours par une étape préalable de conversion avec la commande `Surv`.

La référence bibliographique concernant le package `rms` est naturellement l'ouvrage de Frank Harrell [HAR 01]. Le cours en ligne suivant est construit autour de cet ouvrage et fournit l'essentiel des idées dans un document PDF : http://biostat.mc.vanderbilt.edu/wiki/Main/CourseBios330. Le livre de Ewout Steyerberg [STE 09] repose en partie sur le package `rms` et constitue un bon complément à l'ouvrage précédent. Le site associé au livre est : www.clinicalpredictionmodels.org.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Bibliographie

[BIL 14] BILDER C., LOUGHIN T., *Analysis of Categorical Data with R*, Chapman & Hall/CRC, Boca Raton, 2014.

[BLI 52] BLISS C., *The Statistics of Bioassay*, Academic Press, New York, 1952.

[BRE 80] BRESLOW N., DAY N., *Statistical Methods in Cancer Research : Vol. 1, The Analysis of Case-Control Studies*, IARC Scientific Publications, Lyon, 1980.

[CLE 79] CLEVELAND W., « Robust locally weighted regression and smoothing scatterplots », *Journal of the American Statistical Association*, vol. 74, p. 829-836, 1979.

[COL 94] COLLETT D., *Modelling survival data in medical research*, Chapman & Hall/CRC, Boca Raton, 1994.

[DRA 98] DRAPER N., SMITH H., *Applied Regression Analysis*, John Wiley and Sons, New York, 3e édition, 1998.

[DUP 09] DUPONT W., *Statistical Modeling for Biomedical Researchers*, Cambridge University Press, Cambridge, 2e édition, 2009.

[EVE 01] EVERITT B., RABE-HESKETH S., *Analyzing Medical Data Using S-PLUS*, Springer, New York, 2001.

[FAR 14] FARAWAY J., *Linear Models with R*, Chapman & Hall/CRC, Boca Raton, 2e édition, 2014.

[FOX 10] FOX J., WEISBERG H., *An R Companion to Applied Regression*, Sage Publications, Los Angeles, 2e édition, 2010.

[FRE 63] FREIREICH E., GEHAN E., FREI E., SCHROEDER L., WOLMAN R., BURGET O., MILLS S., PINKEL D., SELAWRY O., MOON J., GENDEL B., SPURR C., STORRS R., HAURANI F., HOOGSTRATEN B., LEE S., « The effect of 6-mercaptopurine on the duration of steroid-induced remissions in acute leukemia: a model for evaluation of other potentially useful therapy », *Blood*, vol. 21, p. 699-716, 1963.

[GAN 13] GANDRU C., *Reproducible Research with R and RStudio*, Chapman & Hall/CRC, Boca Raton, 2013.

[HAN 93] HAND D., DALY F., MCCONWAY K., OSTROWSKI E. (DIR.), *A Handbook of Small Data Sets*, Chapman & Hall/CRC, Boca Raton, 1993.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

[HAR 01] HARRELL F., *Regression modeling strategies with applications to linear models, logistic regression and survival analysis*, Springer, New York, 2001.

[HOL 73] HOLLANDER M., WOLFE D., *Nonparametric statistical inference*, John Wiley and Sons, New York, 1973.

[HOS 89] HOSMER D., LEMESHOW S., *Applied Logistic Regression*, John Wiley and Sons, New York, 1989.

[HOT 09] HOTHORN T., EVERITT B., *A Handbook of Statistical Analyses Using R*, Chapman and Hall/CRC, Boca Raton, 2e édition, 2009.

[IHA 96] IHAKA R., GENTLEMAN R., « R: A language for data analysis and graphics », *Journal of Computational and Graphical Statistics*, vol. 5, n° 3, p. 299-314, 1996.

[LOP 94] LOPRINZI C., LAURIE J., WIEAND H., KROOK J., NOVOTNY P., KUGLER J., BARTEL J., LAW M., BATEMAN M., KLATT N., « Prospective evaluation of prognostic variables from patient-completed questionnaires », *Journal of Clinical Oncology*, vol. 12, n° 3, p. 601-607, 1994.

[MUR 05] MURRELL P., *R Graphics*, Chapman & Hall/CRC, Boca Raton, 2005.

[PEA 05] PEAT J., BARTON B., *Medical Statistics: A Guide to Data Analysis and Critical Appraisal*, John Wiley and Sons, New York, 2e édition, 2005.

[PEP 80] PEPE M., *The statistical evaluation of medical tests for classification*, Oxford University Press, New York, 1980.

[RAB 04] RABE-HESKETH S., EVERITT B., *A Handbook of Statistical Analyses using Stata*, Chapman & Hall/CRC, Boca Raton, 3e édition, 2004.

[SAR 08] SARKAR D., *Lattice : Multivariate Data Visualization with R*, Springer, New York, 2008.

[SCO 08] SCOTT I., GREENBURG P., POOLE P., « Cautionary tales in the clinical interpretation of studies of diagnostic tests », *Internal Medicine Journal*, vol. 38, p. 120-129, 2008.

[SEL 98] SELVIN S., *Modern Applied Biostatistical Methods using S-PLUS*, Oxford University Press, New York, 1998.

[SHA 12] SHAHBABA B., *Biostatistics with R. An Introduction to Statistics Through Biological Data*, Springer, New York, 2012.

[SPE 08] SPECTOR P., *Data Manipulation with R*, Springer, New York, 2008.

[STE 09] STEYERBERG E., *Clinical Prediction Models: A Practical Approach to Development, Validation, and Updating*, Springer, New York, 2009.

[STU 08] STUDENT, « The probable error of a mean », *Biometrika*, vol. 6, n° 1, p. 1-25, 1908.

[THE 99] THERNEAU T., « A Package for Survival Analysis in S », https://cran.r-project.org/web/packages/survival/citation.html, 1999.

[VEN 02] VENABLES W., RIPLEY B., *Modern Applied Statistics with S*, Springer, New York, 4e édition, 2002.

[VER 11] VERZANI J., *Getting started with RStudio*, O'Reilly Media, Sebastopol, 2011.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

[VIT 05] VITTINGHOFF E., GLIDDEN D., SHIBOSKI S., MCCULLOCH C., *Regression Methods in Biostatistics. Linear, Logistic, Survival, and Repeated Measures Models*, Springer, New York, 2005.

[WIC 11] WICKHAM H., « The Split-Apply-Combine Strategy for Data Analysis », *Journal of Statistical Software*, vol. 40, n° 1, 2011.

[WIC 14] WICKHAM H., « Tidy Data », *Journal of Statistical Software*, vol. 59, n° 1, 2014.

[WIL 73] WILKINSON G., ROGERS C., « Symbolic description of factorial models for analysis of variance », *Applied Statistics*, vol. 22, p. 392-399, 1973.

[WIL 05] WILKINSON L., *The grammar of graphics*, Springer, New York, 2005.

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

Index

A

abline, 145
aggregate, 79, 82, 91, 92, 99, 138
ANOVA, 89
anova, 86, 108, 113, 114, 120, 131, 135
aov, 80, 81, 85, 87-90, 92, 94, 96, 97, 99, 106, 109, 112, 120
apply, 102, 117, 143
apropos, 14
array, 122, 123
as.data.frame, 58, 95
as.numeric, 33, 86, 87, 97
as.ordered, 87, 98
as.table, 125, 136

B

barchart, 44, 58, 70, 72, 82
bartlett.test, 83, 92, 97, 100
binom.test, 70, 75, 126
boxplot.stats, 42
bwplot, 42, 60, 81, 93, 100

C

cbind, 108, 109, 134, 139
chisq.test, 58, 66, 67, 73
coef, 108, 118, 130, 142, 155
colnames, 108, 118, 122, 125
complete.cases, 23, 51
confint, 68, 108, 118, 131, 141, 142
cor, 103, 104, 114-117
cor.test, 104, 105, 114-116
corr.test, 116
cox.zph, 156
coxph, 154, 155, 161, 163
cut, 39, 49, 116, 161

D

data, 28, 71, 72, 98, 147
data.frame, 27, 62, 78, 110, 118, 120, 133, 134, 142, 144, 156, 162
density, 42
densityplot, 42, 51, 56
describeBy, 71
diag, 144
diff, 37
dim, 62
dimnames, 73, 123
dotplot, 44, 58, 70, 72, 81, 95
drop1, 89

E

epi.2by2, 69
epi.conf, 94
epi.tests, 127, 135, 136

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

error.bars.by, 71
exp, 130, 133, 142

F

factor, 19, 20, 29, 63, 85, 98, 115-117, 138, 150, 159, 163
fisher.test, 70, 74
fitted, 83, 109, 111, 132
foreign, 75
ftable, 121
function, 55, 99, 117, 139, 140

G

glm, 128-130, 134, 135, 139, 141-144
grid, 145
grid.arrange, 64, 118

H

head, 24, 25, 27, 28, 32-34, 63, 75, 79, 93, 109-111, 117, 131, 133, 138, 148, 160, 163
help, 14, 42, 67, 103, 127, 149
help.search, 14
histogram, 41, 42, 49, 50, 55, 56, 72, 94, 97, 102, 114, 118

I

I, 114
ifelse, 144
influence.measures, 111
install.packages, 14, 63, 68
interaction, 92, 100
interaction.plot, 100
IQR, 37
is.na, 23, 35, 46, 51, 103, 117

K

kruskal.test, 88
kurtosis, 36

L

legend, 151, 152
length, 16, 23, 31, 45-47
levels, 19, 21, 38, 87, 95, 97, 129, 141
leveneTest, 83, 97
library, 14, 39, 63, 64, 68, 69, 93, 94, 96, 97, 116, 118, 125, 136, 141, 145, 147, 160, 163
list, 73
lm, 86, 97, 98, 106-108, 112-114, 118-120, 128
loess, 103
log, 17, 31, 32, 114, 118, 139, 156
lroc, 135
ls, 16

M

mantelhaen.test, 122, 124, 135
margin.table, 57, 75
matrix, 73, 102, 122, 125, 136, 140
max, 18, 36, 49
mcnemar.test, 67, 75
mean, 17, 18, 30, 33, 35-37, 44, 46, 48, 51, 54, 63, 72, 78, 79, 82, 91, 93, 94, 96, 99, 112, 120, 160
median, 32, 36, 37, 48, 50, 159, 160
melt, 63, 79, 141
merge, 156
min, 18, 30, 36
model.tables, 91, 99

N

NA, 23, 32, 51
names, 25, 27, 28, 34, 49, 141-143, 158
ncol, 28
nlevels, 19
nrow, 28, 47

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

O

oddsratio, 68, 130, 135, 141
options, 119
order, 18
ordered, 87

P

pairs, 115
pairwise.t.test, 84
panel.densityplot, 42
panel.dotplot, 95
panel.segments, 95
pariwise.t.test, 84
pcor.test, 116
performance, 134, 145
plogis, 133
plot, 134, 145, 150-152, 156, 160, 162, 163
pnorm, 45, 46
power.prop.test, 71
power.t.test, 71
predict, 83, 109, 110, 120, 131-133, 139, 140, 143, 144
prediction, 134, 145
prop.table, 38, 44, 47, 58, 65, 75, 96, 159
prop.test, 65, 66, 70, 74, 141
pt, 46

Q

qnorm, 45-48, 51, 133
qt, 46, 94
quantile, 36, 37, 48-50

R

r.test, 118
range, 18, 35, 36
rank, 18
rbind, 94, 118
read.csv, 24, 25, 117
read.csv2, 24, 117
read.dta, 93
read.spss, 75, 96
read.table, 24-26, 34, 115, 137, 142, 158, 163
relevel, 129, 141
rep, 138, 144, 162
replicate, 78
replications, 99
resid, 83, 111
rfs, 111
rnorm, 78
roc.from.table, 135, 136
round, 32, 33, 75, 94, 96, 108, 116, 118, 126, 138
rownames, 95, 118, 122, 125
rstandard, 111
rstudent, 111

S

sample, 20
sapply, 78, 102, 133
save, 26
scale, 113, 114, 119
scan, 24, 32, 50
sd, 37, 45, 46, 48, 51, 72, 93, 96, 99
segplot, 95
seq, 36, 41, 72, 110, 149
set.seed, 20
shapiro.test, 83
skewness, 36
sort, 18, 87
splom, 115
sqrt, 45-47, 51, 94, 126
stdres, 111
str, 32, 67, 75, 96, 98, 115, 117, 137, 138, 150, 163
stripplot, 40, 64, 97
studres, 111
subset, 102, 116, 117, 123, 148, 159
sum, 18, 35, 46, 51, 94, 117, 126, 138, 143, 144

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677

summary, 29, 30, 36, 44, 48, 62, 63, 66, 78-81, 85, 87-90, 92, 94, 96-99, 102, 107, 108, 114, 115, 118-120, 128-130, 135, 138, 139, 141-143, 149, 150, 155, 157, 160
summary.aov, 80
summary.lm, 88, 120
Surv, 148, 154-156, 160-163
survdiff, 153, 154, 157, 160, 163
survfit, 149, 150, 155, 157, 160, 162, 163

T

t.test, 47, 58, 60-63, 72, 84
table, 20, 21, 30, 34, 37-39, 44, 47-51, 57, 65-67, 70, 75, 85, 94, 96, 115, 117, 121-123, 129, 144, 159, 160, 163
tapply, 54, 55, 60, 63, 70, 72, 79, 92-94, 96, 112, 120, 144, 159

U

unique, 83, 87
unlist, 94

V

var, 18, 37, 49, 60
var.test, 60

W

which, 23, 85
which.max, 38
wilcox.test, 69, 70
with, 30, 45, 54, 55, 57, 60, 62, 63, 65, 66, 69, 72, 79, 84, 93, 94, 96, 103, 105, 112, 115-117, 120, 122, 123, 144, 148, 159, 163
within, 29
woolf.test, 124, 125, 135
write.csv, 26
write.table, 26, 33

X

xtabs, 38, 39, 44, 57, 58, 66-68, 70, 98, 105, 121-124, 134, 138
xYplot, 95
xyplot, 64, 85, 90, 99, 103, 107, 109, 111, 116, 117, 119, 132, 140, 143, 144, 152, 156, 159

Document protégé électroniquement. Réservé à l'usage unique de Wilguy DOISY(wdoisy@gmail.com) Transaction : #57677